



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Máster Universitario en Inteligencia Artificial

Trabajo Fin de Máster

**SmolVLA-MD: Integración de Memoria a
Corto Plazo y Percepción de la
Profundidad en Modelos VLA para
Manipulación Robótica**

Autor: Juan Esteban Rincón Marín

Tutor: Javier de Lope Asiaín

Madrid, julio 2026

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster

Máster Universitario en Inteligencia Artificial

Título: SmoVLA-MD: Integración de Memoria a Corto Plazo y Percepción de la Profundidad en Modelos VLA para Manipulación Robótica

julio 2026

Autor: Juan Esteban Rincón Marín

Tutor: Javier de Lope Asiaín

Departamento de Inteligencia Artificial

ETSI Informáticos

Universidad Politécnica de Madrid

*A Santi
que cuando crezcas te explicaré todo esto*

Agradecimientos

«Una vez que sabes lo que hay que hacer, es mejor hacerlo que vivir con el miedo a ello.»

—Joe Abercrombie, *La Voz de las Espadas*

«Solo los sacerdotes y los locos no tienen miedo a nada, y yo nunca me he llevado muy bien con Dios.»

—Patrick Rothfuss, *El Nombre del Viento*

Quiero dar las gracias de corazón a toda la gente que me apoya, especialmente cuando me lanzo de cabeza a retos imposibles. A los que conocen y saben que siempre me embarco en aventuras sin garantías, pero confían en mi empeño. Gracias a quienes me han dado una mano para sacar este proyecto adelante y, sobre todo, a los que han llenado este camino de risas y buenos momentos. Porque, al final, siempre es más importante el viaje que el destino.

Resumen

Los modelos Vision-Language-Action (VLA) permiten generar acciones robóticas directamente a partir de observaciones visuales e instrucciones en lenguaje natural. Sin embargo, la mayoría de las políticas actuales dependen exclusivamente de imágenes RGB del instante actual, lo que limita su capacidad en tareas con oclusiones temporales o requerimientos de geometría tridimensional. Este Trabajo Fin de Máster investiga la integración de memoria visual a corto plazo e información de profundidad en SmolVLA, un modelo VLA ligero diseñado para entornos robóticos con recursos de cómputo limitados. La propuesta se implementa sobre un brazo robótico SO-101 mediante aprendizaje por imitación a partir de demostraciones teleoperadas. Se evalúan y comparan cuatro variantes experimentales: SmolVLA-Vanilla (referencia), SmolVLA-M (atención temporal causal), SmolVLA-D (características geométricas inyectadas en capas del experto de acción seleccionadas tras un estudio de ablación) y SmolVLA-MD (fusión conjunta). Las pruebas físicas en escenarios con autooclusiones y distractores fuera de distribución revelan que la incorporación aislada de profundidad (SmolVLA-D) incrementa la puntuación normalizada media de 0,55 a 0,86 en entornos controlados, reduciendo los errores de agarre. Sin embargo, esta variante decae sustancialmente ante distractores visuales, mientras que la memoria actúa como un estabilizador temporal sin resolver la pérdida total de visibilidad. Asimismo, la combinación directa (SmolVLA-MD) no produce una sinergia positiva e interfiere con la dinámica del experto de acción, provocando bloqueos y trayectorias erráticas. Se concluye que la ampliación sensorial en modelos VLA compactos requiere mecanismos de fusión multimodal adaptativos y un corpus de datos de entrenamiento más diverso.

Abstract

Vision-Language-Action (VLA) models enable robotic action generation from visual inputs and text instructions, yet reliance on single-frame RGB observations limits performance under occlusions and 3D geometric demands. This Master’s Thesis evaluates integrating short-term visual memory and depth perception into SmolVLA, a lightweight VLA model. Deployed on an SO-101 robotic arm, four variants were tested: SmolVLA-Vanilla (baseline), SmolVLA-M (causal temporal attention), SmolVLA-D (point cloud features injected into ablation-selected action expert layers), and SmolVLA-MD (joint fusion). Real-world benchmarks show that depth alone (SmolVLA-D) increases mean normalized grasping scores from 0.55 to 0.86 in controlled setups, though it degrades under visual distractors. Temporal memory mainly serves as a trajectory stabilizer, while joint fusion (SmolVLA-MD) disrupts action expert dynamics, inducing freezes and erratic motions. Expanding sensory modalities in compact VLAs requires adaptive fusion mechanisms rather than direct concatenation.

Keywords: Robotics, Vision-Language-Action, SmolVLA, Visual Memory, Depth Perception, Imitation Learning.

Tabla de contenidos

Agradecimientos	iii
Abstract	vii
Índice de figuras	xi
Índice de tablas	xiv
Lista de abreviaturas	xvii
1. Introducción	1
1.1. Objetivo principal	2
1.2. Objetivos específicos	2
1.3. Hipótesis de trabajo	2
1.4. Metodología y plan de trabajo	3
1.5. Estructura del documento	3
2. Estado del arte	5
2.1. Precursores del paradigma actual	5
2.2. Nacimiento y consolidación del paradigma VLA	9
2.3. Generalización a nuevos entornos y plataformas	14
2.4. Modelos abiertos y eficiencia	18
2.4.1. SmoIVLA: robótica asequible	21
2.5. Percepción táctil, memoria y tareas complejas	23
2.6. Representación geométrica tridimensional	28
2.7. Síntesis y posicionamiento del trabajo	33
3. Arquitectura de Referencia y Fundamentos del Sistema	35
3.1. Aprendizaje por imitación	35
3.2. Flow Matching	36
3.3. Arquitectura de SmoIVLA	37
3.4. Representación tridimensional e inyección de características	38
3.5. Memoria visual de corto plazo	39
4. Metodología	41
4.1. Entorno experimental	41
4.2. Toma y preparación de datos	43
4.3. Variantes de SmoIVLA	44
4.3.1. SmoIVLA-Vanilla	45

4.3.2. SmoIVLA-M	45
4.3.3. SmoIVLA-D	46
4.3.4. SmoIVLA-MD	48
4.3.5. Entrenamientos	48
4.4. Estrategia experimental	50
4.4.1. Test de ablación de capas	50
4.4.2. Evaluación final	51
5. Resultados y discusión	53
5.1. Resultados del test de ablación de capas	53
5.2. Resultados de la evaluación final	55
5.3. Discusión de la evaluación final	57
6. Conclusiones y trabajo futuro	61
6.1. Conclusiones	61
6.2. Trabajo futuro	63
Bibliografía	65

Índice de figuras

2.1. Ejemplos cualitativos de observaciones voxelizadas y predicciones de acción en PerAct. La figura muestra distintas escenas de manipulación, junto con la acción predicha por el modelo y la acción experta de referencia, ambas señaladas sobre la representación tridimensional del entorno. Los ejemplos de la izquierda corresponden a tareas simuladas y los de la derecha a tareas reales, adaptado de [1].	6
2.2. Proceso de aprendizaje por imitación utilizado en ALOHA y ACT. Las demostraciones se recopilan mediante un sistema de teleoperación bi-manual y se emplean para entrenar una política que genera secuencias de acciones a partir de observaciones visuales y propioceptivas [2]. . . .	7
2.3. Funcionamiento del action chunking y del temporal ensembling en ACT. La política predice bloques de k acciones futuras y combina las predicciones solapadas correspondientes a un mismo instante para obtener la acción finalmente ejecutada [2].	8
2.4. Esquema conceptual de RT-2. Las acciones robóticas continuas se discretizan y se representan mediante tokens, que se generan de forma autorregresiva y posteriormente se convierten en comandos de control ejecutables [3].	10
2.5. Representación conceptual de Octo. Las instrucciones, las observaciones visuales y propioceptivas y las representaciones empleadas para generar acciones se integran mediante una arquitectura Transformer común, adaptable a diferentes configuraciones robóticas [4].	11
2.6. Esquema conceptual de Π_0 . El backbone visión-lenguaje procesa las imágenes y la instrucción, mientras que un experto de acción condicionado por sus representaciones genera bloques de acciones continuas [5].	12
2.7. Arquitectura dual de GR00T N1. El módulo visión-lenguaje, denominado System 2, interpreta las observaciones y la instrucción, mientras que System 1 genera acciones motoras mediante un Diffusion Transformer condicionado por las representaciones anteriores [7].	15
2.8. Pirámide de datos utilizada en GR00T N1. Los datos web y los vídeos humanos proporcionan la base de mayor escala; los datos sintéticos y simulados ocupan el nivel intermedio; y las trayectorias de robots físicos constituyen la fuente de mayor especificidad encarnada [7].	15
2.9. Proceso de entrenamiento e inferencia de $\Pi_{0.5}$. El preentrenamiento integra fuentes heterogéneas mediante acciones discretizadas y tareas multimodales; posteriormente, un experto de acción basado en flow matching genera bloques de acciones continuas para el control robótico [8].	17

2.10 Fuentes heterogéneas utilizadas en $\Pi_{0.5}$. El modelo combina trayectorias robóticas, predicción de subtarefas, instrucciones verbales, detección de objetos y ejemplos multimodales procedentes de la web para favorecer la transferencia hacia entornos domésticos no observados [8].	18
2.11 Arquitectura de OpenVLA. Las representaciones visuales extraídas mediante DINOv2 y SigLIP se proyectan al espacio de Llama 2, que genera tokens discretizados posteriormente convertidos en comandos continuos de control [6].	19
2.12 Arquitectura y proceso de adaptación de TinyVLA. Un modelo visión-lenguaje compacto se combina con una política de difusión que genera secuencias continuas de acciones, evitando la predicción autorregresiva de tokens de control [9].	20
2.13 Esquema general de SmolVLA. El modelo combina un VLM compacto con un experto de acción entrenado mediante flow matching y utiliza conjuntos de datos públicos recopilados sobre plataformas robóticas asequibles [10].	22
2.14 Pila de inferencia asíncrona de SmolVLA. La generación de nuevos bloques de acciones se solapa con la ejecución de los bloques disponibles, reduciendo los periodos de espera entre la percepción, la predicción y el control [10].	22
2.15 Visión general de VTLA. El modelo procesa observaciones visuales y táctiles junto con una instrucción lingüística para generar acciones en tareas de inserción peg-in-hole [11].	24
2.16 Proceso de entrenamiento de VTLA. En la primera fase, el modelo se ajusta mediante predicción del siguiente token. En la segunda, DPO introduce una señal basada en preferencias para aproximar mejor la naturaleza continua de las acciones robóticas [11].	25
2.17 Visión general de MEM. El sistema combina una memoria visual a corto plazo, basada en secuencias de vídeo, con una memoria textual a largo plazo que resume los acontecimientos semánticos relevantes para la tarea [12].	27
2.18 Arquitectura funcional de MEM. La política de alto nivel actualiza la memoria textual y selecciona subtarefas, mientras que la política de bajo nivel utiliza observaciones recientes, memoria visual y contexto semántico para generar acciones continuas [12].	27
2.19 Arquitectura de 3D Diffusion Policy. La información de profundidad se transforma en una nube de puntos, se procesa mediante un codificador ligero y condiciona una política de difusión encargada de generar secuencias continuas de acciones [13].	29
2.20 Arquitectura general de PointVLA. El modelo conserva el procesamiento visión-lenguaje del VLA preentrenado e incorpora una representación de la nube de puntos mediante un módulo tridimensional conectado con determinados bloques del experto de acción [14].	31
2.21 Análisis de bloques utilizado en PointVLA. La importancia de las distintas capas del experto de acción se evalúa mediante su omisión durante la inferencia. Los bloques menos críticos se seleccionan para recibir la información tridimensional, reduciendo la interferencia sobre las representaciones preentrenadas [14].	32

3.1. Evolución conceptual del proceso de flow matching. Elaboración propia a partir de [15].	36
3.2. Arquitectura simplificada de SmoVLA. El modelo de visión y lenguaje procesa las imágenes RGB, la instrucción y la propiocepción para generar un contexto multimodal que condiciona de forma constante al experto de acción en la generación de acciones. Elaboración propia a partir de [10].	38
4.1. Brazos SO-101 para teleoperación: seguidor con pinza (izquierda) y líder (derecha). Imágenes de https://huggingface.co/docs/lerobot/so101	42
4.2. Cámaras del sistema: RGB superior fija (izquierda), RGB monocular de muñeca (centro) y estéreo SVPRO con su calibración y captura side-by-side (derecha).	42
4.3. Discretización del área de trabajo: rejilla 3×3 (posiciones A1-C3) para origen de objetos y zona fija de destino.	43
4.4. Flujo de procesamiento para el dataset con profundidad: rectificación de vistas estéreo, cálculo del mapa de disparidad y reproyección a nube de puntos.	44
4.5. Relación y jerarquía de la familia de modelos SmoVLA desarrollada a partir del checkpoint base.	44
4.6. Vista general de la arquitectura de SmoVLA-Vanilla.	46
4.7. Arquitectura de SmoVLA-M: integración de atención temporal causal en las capas 3, 7 y 11 del codificador visual.	47
4.8. Arquitectura de SmoVLA-D.	47
4.9. Arquitectura de SmoVLA-MD.	48
4.10 Evolución de la pérdida y de la tasa de aprendizaje durante el entrenamiento de las cuatro configuraciones finales. Las líneas verticales indican el punto de finalización o interrupción de cada ejecución.	49
4.11 Evolución de la pérdida de entrenamiento y de la tasa de aprendizaje durante el ajuste del modelo base para el estudio de ablación.	51
5.1. Puntuación normalizada media y tasa de éxito completo obtenidas al desactivar individualmente cada capa del experto de acción. Las barras de error representan la desviación típica de la puntuación.	54
5.2. Comparación entre el rendimiento esperado a partir de las ablaciones individuales y el observado al desactivar varias capas contiguas.	54
5.3. Distribución de los modos de fallo observados en el modelo base, las ablaciones individuales y las combinaciones de capas contiguas.	55
5.4. Comparación de la puntuación normalizada media por modelo y condición experimental (sin foto y con foto).	56
5.5. Puntuación normalizada de cada ejecución física por modelo. La línea vertical separa los episodios sin fotografía de los episodios con fotografía. La línea horizontal representa la media de ese conjunto de ejecuciones	56
5.6. Métricas conductuales de la evaluación final: éxito completo, timeout, intervención de emergencia, recuerdo del objeto ocluido, finalización sobre el objeto ocluido y rechazo de la fotografía. Derivadas de las dinámicas de ejecución y modos de fallo, estas tasas permiten identificar cómo se comporta y por qué falla cada modelo en detalle.	57

Índice de tablas

2.1. Síntesis de los principales trabajos revisados en el estado del arte. . . .	34
4.1. Resumen de la configuración y ejecución de los entrenamientos de los modelos finales (NVIDIA H100 80GB).	49
4.2. Configuración del entrenamiento utilizado como base para la ablación. .	50
4.3. Desglose de la rúbrica de evaluación según la condición del escenario. .	52

Lista de abreviaturas

Abreviatura	Significado
BC	Behavioral Cloning (Clonación de comportamiento).
CNN	Convolutional Neural Network (Red neuronal convolucional).
DOF	Degrees of Freedom (Grados de libertad).
DPO	Direct Preference Optimization (Optimización directa de preferencias).
ID	In-Distribution (Dentro de la distribución de entrenamiento).
LLM	Large Language Model (Gran modelo de lenguaje).
LoRA	Low-Rank Adaptation (Adaptación de bajo rango).
MEM	Multi-Scale Embodied Memory (Memoria corpórea multiescala).
OOD	Out-of-Distribution (Fuera de la distribución de entrenamiento).
RGB-D	Red, Green, Blue and Depth (Imagen con información de color y profundidad).
SO-101	Plataforma robótica de seis grados de libertad empleada en este trabajo.
VGTE	Vision-Guided Temporally Enhanced Tokens (Tokens mejorados temporalmente y guiados por visión).
ViT	Vision Transformer (Arquitectura Transformer aplicada a imágenes).
VLA	Vision-Language-Action (Modelo de visión, lenguaje y acción).
VLM	Vision-Language Model (Modelo de visión y lenguaje).

Capítulo 1

Introducción

A lo largo de las últimas décadas, la robótica industrial ha alcanzado un nivel de madurez tecnológica impresionante, siendo capaz de operar con el máximo nivel de precisión y velocidad en entornos estructurados con tareas preprogramadas. Las líneas de montaje automatizadas son un claro ejemplo de sistemas extraordinariamente optimizados, donde los brazos robot ejecutan trayectorias milimétricas de forma predecible. Sin embargo, el desafío actual en la robótica se encuentra en crear sistemas de control capaces de manejar la incertidumbre de un entorno menos estructurado. El verdadero obstáculo ahora mismo no recae en mejorar las capacidades físicas del robot, o dotarle de innumerables reglas rígidas para reaccionar a su entorno. El obstáculo está en dotarle de las capacidades de interpretar el entorno, comprender las tareas a llevar a cabo y adaptarse ante la variabilidad del mundo real.

Los avances para abordar este desafío se apoyan en los recientes desarrollos del deep learning, entre los que destaca la arquitectura Transformer por su capacidad para procesar y relacionar secuencias de datos a gran escala. Concebida inicialmente para el procesamiento del lenguaje natural, esta arquitectura se extendió al dominio visual mediante los Vision Transformers (ViT), que representan las imágenes como secuencias de tokens. Siguiendo este mismo enfoque, la arquitectura se trasladó a la robótica para entrenar generadores de acciones condicionados por las observaciones del entorno y la tarea asignada. De esta manera surgieron los modelos Vision-Language-Action (VLA), los cuales permiten desarrollar políticas de control más flexibles y capaces de adaptarse a entornos no estructurados y a una mayor diversidad de tareas.

Sin embargo, la robótica plantea desafíos a los que la arquitectura Transformer no se había enfrentado aún, como la comprensión tridimensional de la escena o su coherencia temporal. La manipulación de objetos en el mundo real exige algo más que interpretar el entorno y entender el objetivo; requiere dominar el espacio tridimensional por el que se desplaza el robot, estimar el volumen de los objetos y mantener un registro de las acciones previas cuando la tarea exige una secuencia de pasos. Al tratarse de un enfoque relativamente reciente, la cuestión de qué mejoras sensoriales pueden ayudar a cerrar esta brecha sigue abierta.

En este contexto surge la pregunta central de este trabajo: ¿puede una mejora en la percepción de la profundidad y en la memoria a corto plazo optimizar la destreza de un modelo VLA en tareas de manipulación de objetos? Para responder a esta

pregunta, se analizan distintas variantes desarrolladas a partir del modelo base en un entorno físico real. De este modo, es posible evaluar el impacto de cada modificación y atribuir sus mejoras de forma aislada.

1.1. Objetivo principal

El objetivo principal de este trabajo consiste en diseñar, implementar y evaluar una extensión modular de SmolVLA que incorpore memoria visual de corto plazo e información de profundidad. A través de esta ampliación se pretende estudiar el efecto de ambas fuentes de información sobre el comportamiento de una política de manipulación robótica, especialmente en situaciones de observabilidad parcial, autooclusión y ambigüedad espacial.

La propuesta, denominada SmolVLAMD, parte de un modelo Vision-Language-Action ligero y previamente entrenado, evitando el desarrollo de una política fundacional desde cero. El trabajo se centra, por tanto, en analizar cómo pueden integrarse nuevas capacidades perceptivas y temporales sin sustituir el conocimiento visual, lingüístico y motor adquirido por el modelo base.

1.2. Objetivos específicos

Para alcanzar el objetivo principal se plantean los siguientes objetivos específicos:

- Analizar la arquitectura de SmolVLA mediante un estudio de ablación en su experto de acción para determinar las regiones óptimas de integración de los nuevos componentes.
- Configurar el entorno experimental con el robot SO-101 para recopilar un conjunto de datos propio mediante demostraciones por teleoperación.
- Diseñar e implementar el mecanismo de memoria visual a corto plazo y el módulo de procesamiento e inyección de datos de profundidad.
- Desarrollar las cuatro variantes experimentales del modelo necesarias para aislar y comparar el impacto de cada mejora sensorial.
- Establecer un protocolo de evaluación común que contemple escenarios de prueba tanto ID como OOD.
- Comparar el comportamiento de las distintas configuraciones con el modelo base para evaluar si la información de profundidad y la memoria realmente mejoran la destreza del robot.

1.3. Hipótesis de trabajo

Para guiar la evaluación experimental de este trabajo, se plantean las siguientes hipótesis:

- **H1 (Memoria temporal):** La incorporación de memoria visual a corto plazo permitirá retener información de observaciones pasadas, mejorando el desempeño de la política en situaciones de autooclusión, pérdida temporal de visibilidad y observabilidad parcial.
- **H2 (Percepción espacial):** La integración de información de profundidad proporcionará una representación geométrica explícita de la escena, aumentando la robustez del modelo en tareas sensibles a la distancia, el volumen y la ubicación espacial de los objetos.
- **H3 (Sinergia de capacidades):** La combinación simultánea de memoria visual e información de profundidad ofrecerá un rendimiento superior tanto al modelo base como a sus variantes individuales, al aportar información temporal y geométrica complementaria.

1.4. Metodología y plan de trabajo

En primer lugar, se realiza una revisión del estado del arte de los modelos VLA para entender de dónde vienen, qué técnicas usan para generar acciones y qué mejoras sensoriales se están aplicando actualmente. Esto se complementa con los fundamentos teóricos necesarios para comprender cómo integrar la información temporal y de profundidad, además de cómo obtener los datos de entrenamiento.

Una vez revisadas las bases del problema e identificadas las limitaciones en los modelos actuales, se aborda el desarrollo de la propuesta principal de este trabajo. Primero se configura la plataforma experimental, a partir de la cual se realiza la recopilación de datos por teleoperación, y se diseña el protocolo de pruebas en el entorno físico.

Tras la recolección del conjunto de datos, se procede al diseño y entrenamiento de las mejoras sobre el modelo base. Para ello, se incluye un estudio de ablación previo con el fin de encontrar los puntos óptimos donde integrar los nuevos componentes. Finalmente, se evalúan las cuatro variantes en la plataforma experimental para analizar, de forma aislada, el impacto de cada mejora sensorial en la destreza del robot.

1.5. Estructura del documento

El resto del documento se organiza del siguiente modo:

- El **Capítulo 2** presenta el estado del arte. En él se revisan los antecedentes del paradigma Vision-Language-Action, desde el aprendizaje por imitación y las políticas basadas en Transformers hasta los modelos VLA contemporáneos. También se analizan las líneas relacionadas con la eficiencia, la memoria, la multimodalidad y la percepción tridimensional.
- El **Capítulo 3** desarrolla los fundamentos teóricos necesarios para comprender el trabajo. Entre ellos se incluyen el aprendizaje por imitación, la técnica de difusión conocida como flow matching, la memoria temporal y la estimación de profundidad.

- El **Capítulo 4** describe los materiales y métodos empleados. Se presentan la plataforma robótica, el proceso de adquisición de datos, el estudio jerárquico de ablación, las variantes del modelo original desarrolladas, las configuraciones experimentales, el procedimiento de entrenamiento y el protocolo de evaluación.
- El **Capítulo 5** presenta y discute los resultados experimentales tanto del estudio de capas como de la evaluación final. Posteriormente se comparan las cuatro variantes del modelo para contrastar las hipótesis de trabajo.
- Finalmente, el **Capítulo 6** resume las principales conclusiones del trabajo, expone sus limitaciones y plantea posibles líneas de trabajo futuro.

Capítulo 2

Estado del arte

Este capítulo revisa los principales antecedentes y líneas de evolución del paradigma VLA. La revisión parte de sus antecedentes metodológicos y examina la transición hacia políticas generalistas, abiertas y computacionalmente eficientes.

La selección de trabajos presta especial atención a las propuestas que amplían la información disponible para la política, como la percepción táctil, la estructura tridimensional del entorno y la memoria del modelo. Con esto, se pretende delimitar el vacío abordado en este trabajo y justificar la elección de SmolVLA como arquitectura de referencia.

2.1. Precursores del paradigma actual

Antes de la formulación explícita de los modelos VLA, diferentes trabajos ya investigan cómo relacionar instrucciones, observaciones visuales y acciones robóticas mediante modelos aprendidos. Dentro de esta etapa pueden identificarse dos líneas especialmente relevantes. La primera estudia el uso de representaciones tridimensionales y arquitecturas Transformer para la manipulación condicionada por lenguaje. La segunda se centra en el aprendizaje de tareas de manipulación fina a partir de demostraciones humanas y en la predicción conjunta de secuencias de acciones. Aunque ambas líneas abordan problemas diferentes, sus aportaciones convergen posteriormente en el desarrollo de políticas capaces de interpretar el entorno, comprender la tarea y generar acciones de forma integrada.

Una de las dificultades fundamentales de la manipulación robótica consiste en traducir una instrucción general, como abrir un cajón o colocar un objeto en una posición determinada, en una acción físicamente ejecutable. Para ello, la política debe identificar los elementos relevantes de la escena, localizar su posición y determinar cómo debe situarse el efector final para interactuar con ellos. Esta relación entre percepción, lenguaje y acción requiere representar no solo la apariencia de los objetos, sino también su disposición espacial.

PerAct constituye un antecedente representativo de esta línea de investigación [1]. El modelo aborda tareas de manipulación condicionadas mediante instrucciones lingüísticas y utiliza observaciones visuales con información de profundidad para construir una representación tridimensional del entorno. Su planteamiento parte de que

2.1. Precursores del paradigma actual

la predicción de acciones de seis grados de libertad resulta más adecuada cuando la política dispone de una descripción explícita de la geometría de la escena.

Para ello, PerAct transforma las observaciones obtenidas desde una o varias cámaras en una cuadrícula tridimensional formada por vóxeles. Cada vóxel representa una región del espacio e incorpora información visual y geométrica. Esta representación permite que la política determine dónde debe actuar el robot sobre una reconstrucción espacial del entorno, en lugar de inferir la posición de los objetos únicamente a partir de imágenes bidimensionales. Como se muestra en la Figura 2.1, esta formulación permite expresar la escena en un espacio tridimensional estructurado y situar sobre él tanto la predicción de la política como la acción experta de referencia.

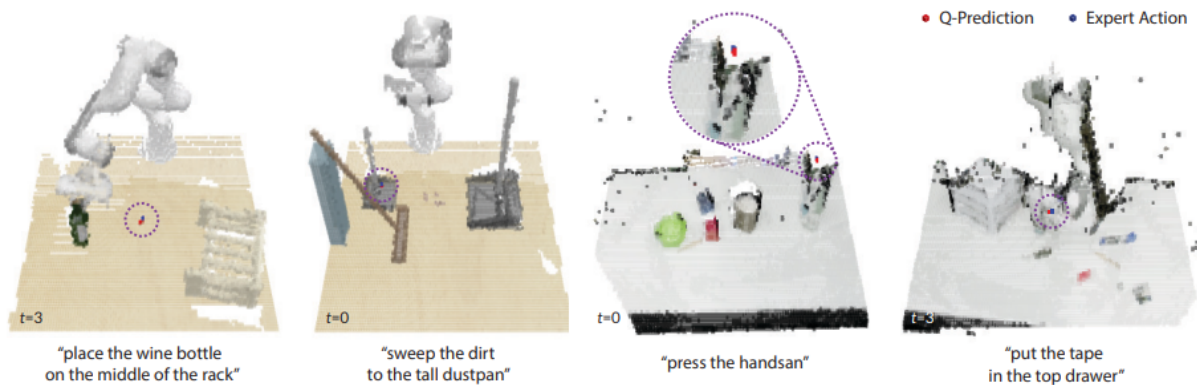


Figura 2.1: Ejemplos cualitativos de observaciones voxelizadas y predicciones de acción en PerAct. La figura muestra distintas escenas de manipulación, junto con la acción predicha por el modelo y la acción experta de referencia, ambas señaladas sobre la representación tridimensional del entorno. Los ejemplos de la izquierda corresponden a tareas simuladas y los de la derecha a tareas reales, adaptado de [1].

La representación voxelizada se procesa mediante un Perceiver Transformer condicionado por la instrucción lingüística. A partir de ella, el modelo predice una acción discretizada de seis grados de libertad, que incluye la posición, la orientación y el estado de la pinza. Esta formulación se plantea como la selección del siguiente vóxel de acción más adecuado, denominado en el trabajo next best voxel action [1].

El uso de una representación voxelizada aporta varias ventajas. En primer lugar, permite fusionar de forma natural observaciones procedentes de diferentes cámaras. En segundo lugar, introduce información explícita sobre profundidad, posición y orientación, variables directamente relacionadas con la ejecución de acciones de manipulación. Finalmente, facilita la aplicación de transformaciones espaciales durante el aumento de datos, lo que mejora el aprovechamiento de conjuntos de demostraciones relativamente reducidos.

PerAct no se considera un VLA en el sentido adoptado posteriormente por modelos construidos sobre grandes modelos de visión y lenguaje preentrenados. Su comprensión lingüística permanece estrechamente vinculada a las tareas y demostraciones utilizadas durante el entrenamiento, y la política actúa de forma reactiva a partir de la observación actual, sin mantener una representación explícita del historial [1]. No obstante, el trabajo anticipa tres elementos que adquieren una importancia central en desarrollos posteriores: el condicionamiento mediante lenguaje, el uso de Transformers para relacionar percepción y acción, y la incorporación de estructura espacial

explícita en políticas de manipulación.

Esta última contribución resulta especialmente relevante para la línea tridimensional revisada más adelante en este capítulo. La formulación de PerAct muestra que la representación empleada por la política constituye una parte fundamental del problema. Cuando la tarea requiere razonar sobre geometría, profundidad y orientación, proporcionar una estructura tridimensional explícita puede favorecer un aprendizaje más eficiente que el uso exclusivo de imágenes RGB.

Sin embargo, una representación adecuada del entorno no resuelve por sí sola otro de los principales retos del aprendizaje robótico: la adquisición de comportamientos precisos a partir de datos reales y su ejecución estable a lo largo del tiempo. Incluso cuando la política dispone de información espacial suficiente, las tareas de manipulación fina requieren demostraciones de calidad, coordinación visual continua y mecanismos capaces de limitar la acumulación de errores entre acciones consecutivas. Esta segunda línea de antecedentes está representada por ALOHA y Action Chunking with Transformers (ACT) [2].

La propuesta aborda tareas bimanuales de manipulación fina que exigen coordinación visual continua, movimientos precisos y un control adecuado de los contactos entre el robot y los objetos. En estas tareas, un pequeño error en la posición de los efectores finales puede modificar el estado de la escena y condicionar las decisiones posteriores. Como resultado, la política puede recibir observaciones cada vez más alejadas de las utilizadas durante el entrenamiento, lo que favorece la acumulación progresiva de errores.

Para abordar este problema desde la adquisición de datos, ALOHA proporciona una plataforma de teleoperación bimanual construida con componentes de bajo coste y diseñada para recopilar demostraciones humanas en robots físicos. El operador controla dos brazos líderes, cuyos movimientos se trasladan a los brazos seguidores encargados de ejecutar la tarea. Como se muestra en la Figura 2.2, las observaciones visuales y propioceptivas registradas durante estas demostraciones se utilizan posteriormente para entrenar una política extremo a extremo.

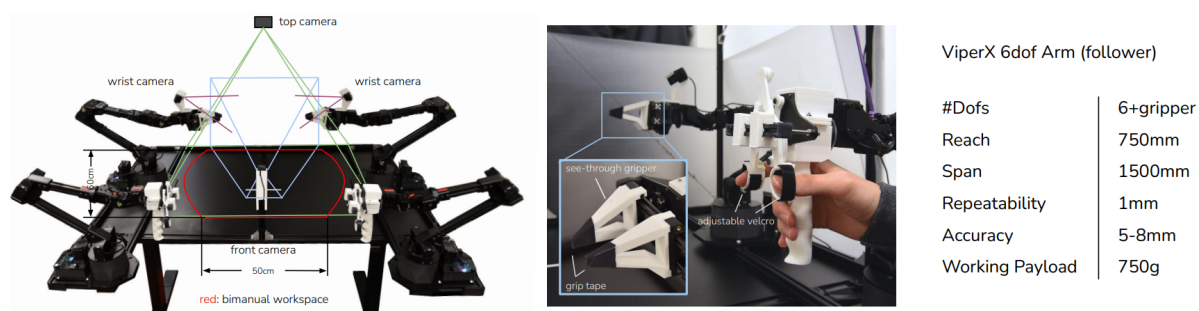


Figura 2.2: Proceso de aprendizaje por imitación utilizado en ALOHA y ACT. Las demostraciones se recopilan mediante un sistema de teleoperación bimanual y se emplean para entrenar una política que genera secuencias de acciones a partir de observaciones visuales y propioceptivas [2].

Esta infraestructura aborda uno de los principales condicionantes del aprendizaje robótico, ya que la obtención de datos reales resulta costosa y requiere sistemas de teleoperación suficientemente precisos para capturar comportamientos complejos. La propuesta permite recopilar demostraciones de tareas como la inserción de objetos,

la apertura de recipientes y la manipulación coordinada de elementos deformables o de pequeñas dimensiones [2].

La disponibilidad de demostraciones precisas permite abordar el segundo componente del problema: cómo generar acciones estables a partir de ellas. Sobre estos datos se entrena ACT, una política basada en Transformers que busca reducir la acumulación de errores propia del aprendizaje por imitación. Las políticas que predicen una única acción en cada instante deben tomar un número elevado de decisiones consecutivas. Cada error modifica el estado observado en el siguiente paso y puede desplazar progresivamente al robot fuera de la distribución cubierta por las demostraciones.

ACT aborda este problema mediante el action chunking, o agrupación de acciones. En lugar de generar una única acción, la política predice un bloque de posiciones articulares futuras. Este mecanismo reduce el horizonte efectivo de decisión y permite representar movimientos coordinados como una unidad temporal. La arquitectura se entrena como un autoencoder variacional condicionado, lo que permite modelar parte de la variabilidad presente en las demostraciones humanas [2].

No obstante, la generación de bloques completos puede producir discontinuidades al sustituir una secuencia por la siguiente. Para reducir este efecto, ACT incorpora un mecanismo de temporal ensembling, o ensamblado temporal. La política se consulta de forma recurrente, por lo que distintos bloques contienen predicciones para un mismo instante futuro. La acción ejecutada se obtiene mediante una combinación ponderada de estas predicciones solapadas.

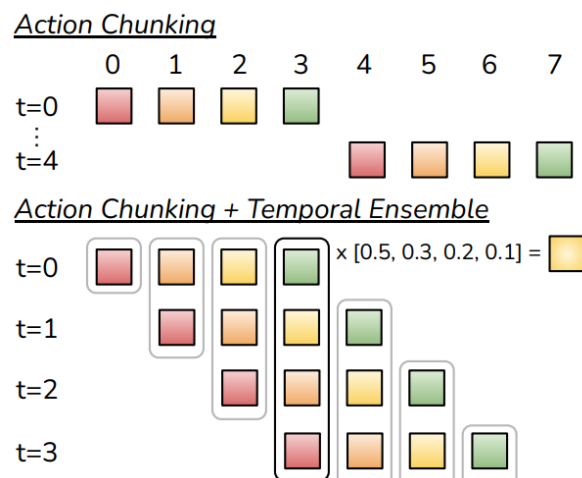


Figura 2.3: Funcionamiento del action chunking y del temporal ensembling en ACT. La política predice bloques de k acciones futuras y combina las predicciones solapadas correspondientes a un mismo instante para obtener la acción finalmente ejecutada [2].

La Figura 2.3 representa ambos mecanismos. La predicción por bloques reduce la longitud efectiva de la secuencia de decisiones, mientras que el ensamblado temporal suaviza la transición entre predicciones consecutivas. Esta combinación permite obtener trayectorias más estables y limitar la propagación de errores durante la ejecución.

Las aportaciones de PerAct, ALOHA y ACT son complementarias. PerAct se centra en cómo representar el entorno y situar espacialmente la acción, mientras que ALOHA y

ACT abordan cómo obtener demostraciones precisas y convertirlas en secuencias de control estables. En conjunto, estos trabajos anticipan varios de los componentes que posteriormente se integran en las políticas VLA: el condicionamiento lingüístico, el procesamiento de observaciones mediante Transformers, la representación espacial del entorno, el aprendizaje por imitación y la generación de bloques de acciones.

De este modo, PerAct demuestra la utilidad de combinar instrucciones lingüísticas, arquitecturas Transformer y representaciones espaciales tridimensionales para predecir acciones de manipulación. ALOHA refuerza el papel de las demostraciones humanas y facilita su recopilación mediante una plataforma robótica accesible. ACT, por su parte, muestra que la generación conjunta de secuencias de acciones puede reducir la acumulación de errores en tareas que requieren control fino.

Estos trabajos establecen tres bases relevantes para el desarrollo posterior de los modelos VLA: la representación espacial explícita introducida por PerAct, la recopilación de demostraciones mediante ALOHA y la generación estable de secuencias de acciones propuesta por ACT. La combinación posterior de estas ideas con modelos de visión y lenguaje preentrenados permite avanzar hacia políticas más generales, precisas y adaptables.

2.2. Nacimiento y consolidación del paradigma VLA

Sobre los antecedentes descritos anteriormente, los modelos Vision-Language-Action introducen un cambio relevante en el aprendizaje robótico al reutilizar modelos de visión y lenguaje preentrenados sobre grandes colecciones de imágenes y textos. De este modo, las políticas pueden aprovechar representaciones visuales y semánticas adquiridas previamente y adaptarlas a la generación de acciones robóticas, en lugar de depender exclusivamente de datos específicos de una tarea o plataforma.

El desarrollo del paradigma se orienta progresivamente hacia la integración de percepción, lenguaje y control dentro de una misma arquitectura, así como hacia la generalización entre tareas y robots, el uso de grandes conjuntos de demostraciones, la apertura de modelos y procedimientos de entrenamiento, y la generación de acciones continuas mediante módulos especializados. Esta evolución establece las bases de las arquitecturas VLA actuales.

El primer paso en esta consolidación consiste en incorporar las acciones robóticas dentro de la misma interfaz generativa empleada por los modelos de visión y lenguaje. RT-2 introduce de forma explícita el término Vision-Language-Action para describir modelos que procesan observaciones visuales e instrucciones lingüísticas y generan directamente acciones robóticas [3]. La propuesta parte de un modelo de visión y lenguaje (VLM) preentrenado con datos procedentes de Internet y lo adapta al control mediante un entrenamiento conjunto sobre tareas visión-lenguaje y trayectorias robóticas.

Los modelos visión-lenguaje convencionales generan secuencias de texto. Sin embargo, una política robótica debe producir valores que representen desplazamientos, rotaciones y comandos de apertura o cierre de la pinza. Para hacer compatibles ambas salidas, RT-2 aplica una tokenización de acciones: cada dimensión continua de la acción se discretiza y se representa mediante tokens pertenecientes al vocabulario del modelo.

2.2. Nacimiento y consolidación del paradigma VLA

De este modo, la respuesta a una instrucción robótica adopta formalmente la misma estructura que una respuesta textual. Ante una imagen y una instrucción, el modelo genera una secuencia de tokens mediante generación autorregresiva. Posteriormente, estos tokens se convierten de nuevo en valores numéricos que pueden enviarse al controlador del robot.

La Figura 2.4 resume este procedimiento. La principal aportación de la arquitectura consiste en integrar las acciones dentro de la misma interfaz secuencial utilizada para representar el lenguaje. Esta unificación permite entrenar conjuntamente ejemplos de preguntas visuales, instrucciones lingüísticas y control robótico sin introducir un decodificador de acciones completamente independiente.

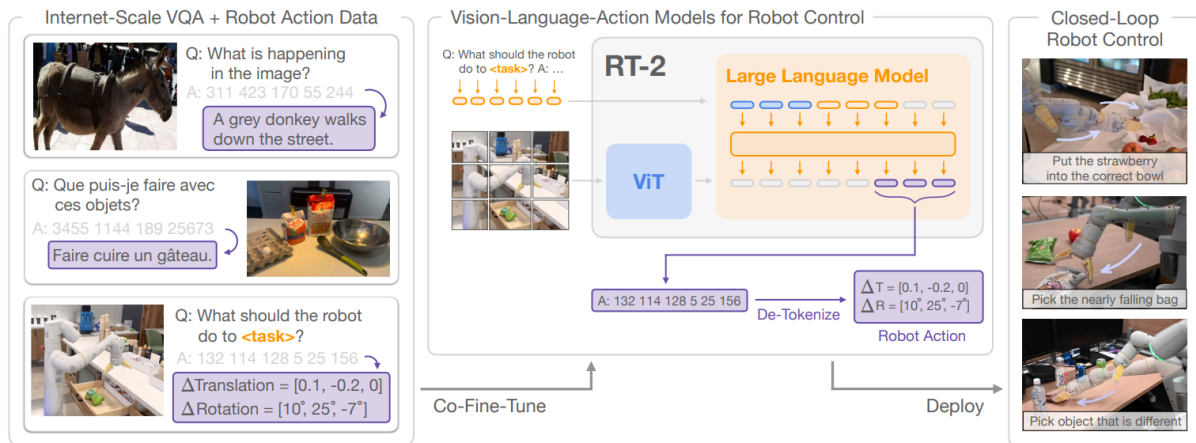


Figura 2.4: Esquema conceptual de RT-2. Las acciones robóticas continuas se discretizan y se representan mediante tokens, que se generan de forma autorregresiva y posteriormente se convierten en comandos de control ejecutables [3].

Este planteamiento presenta dos consecuencias relevantes. En primer lugar, permite reutilizar directamente arquitecturas e infraestructuras desarrolladas para grandes modelos de lenguaje y visión. En segundo lugar, facilita la transferencia de conocimiento semántico desde los datos web hacia las tareas robóticas. RT-2 muestra, por ejemplo, capacidad para relacionar instrucciones con propiedades de objetos, interpretar relaciones espaciales y aplicar habilidades de manipulación conocidas a indicaciones que no aparecen literalmente en las demostraciones robóticas [3].

No obstante, la representación discreta también introduce limitaciones. La precisión de las acciones queda condicionada por el nivel de discretización y la generación autorregresiva requiere producir los tokens de manera secuencial. Este proceso puede aumentar la latencia y dificultar la representación de distribuciones continuas complejas. Estas restricciones motivan posteriormente el desarrollo de decodificadores de acciones basados en difusión o flow matching.

A pesar de estas limitaciones, RT-2 constituye un punto de inflexión porque proporciona una formulación clara del paradigma VLA. La percepción visual, la comprensión de instrucciones y la generación de acciones dejan de tratarse como módulos completamente separados y se integran dentro de un mismo modelo generativo.

Una vez demostrada la viabilidad de esta integración, el siguiente reto consiste en ampliar su alcance más allá de una configuración robótica concreta. La consolidación del aprendizaje robótico generalista no depende únicamente de combinar visión,

lenguaje y acción, sino también de que la política pueda reutilizarse en aplicaciones con sensores, instrucciones, robots y espacios de acción diferentes. Octo aborda este problema mediante una política generalista basada en Transformers y preentrenada sobre aproximadamente 800.000 trayectorias de Open X-Embodiment [4].

Octo no utiliza exactamente la misma formulación que RT-2. En lugar de adaptar un gran VLM para generar tokens de acción, construye una política Transformer en la que las observaciones, la especificación de la tarea y las salidas se representan mediante diferentes grupos de tokens. La arquitectura admite instrucciones lingüísticas o imágenes objetivo y puede procesar distintas combinaciones de cámaras y estados propioceptivos.

Como se representa en la Figura 2.5, el núcleo Transformer recibe los tokens disponibles para cada configuración y genera representaciones que se conectan con cabezas específicas de acción. Esta organización permite modificar las entradas o las salidas sin sustituir completamente el modelo preentrenado.

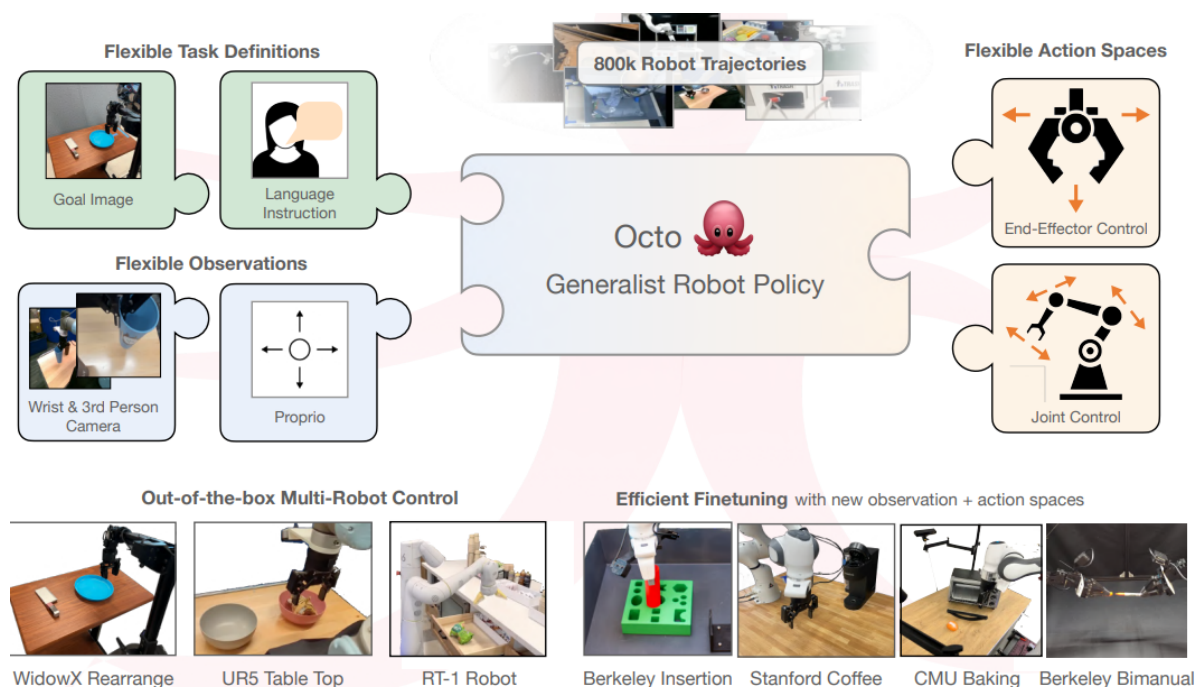


Figura 2.5: Representación conceptual de Octo. Las instrucciones, las observaciones visuales y propioceptivas y las representaciones empleadas para generar acciones se integran mediante una arquitectura Transformer común, adaptable a diferentes configuraciones robóticas [4].

Una de las principales contribuciones de Octo es su capacidad de adaptación a nuevos dominios. Mediante la incorporación de codificadores o cabezas de acción específicas, la política puede ajustarse a robots con observaciones, morfologías o espacios de control diferentes de los utilizados durante el preentrenamiento. Este procedimiento reduce la necesidad de entrenar una política completa desde una inicialización aleatoria.

Octo también muestra la importancia de la diversidad de los datos de preentrenamiento. La combinación de trayectorias procedentes de diferentes plataformas per-

2.2. Nacimiento y consolidación del paradigma VLA

mite aprender representaciones compartidas entre tareas y robots, aunque las configuraciones sensoriales y los espacios de acción no sean idénticos. El trabajo indica que las políticas entrenadas con mezclas de datos más amplias presentan una mayor capacidad de reutilización que aquellas desarrolladas sobre distribuciones más restringidas [4].

Aunque Octo no responde estrictamente a la formulación autorregresiva empleada por RT-2, comparte el objetivo central de los modelos VLA: desarrollar una política multimodal capaz de interpretar diferentes especificaciones de tarea y generar acciones para múltiples configuraciones robóticas. Asimismo, establece propiedades que adquieren una importancia creciente en los trabajos posteriores, como la adaptación a nuevas observaciones, espacios de acción y morfologías.

La evolución desde RT-2 hasta Octo amplía, por tanto, el paradigma desde la transferencia de conocimiento visión-lenguaje hacia el control robótico hasta el preentrenamiento de políticas reutilizables entre múltiples plataformas. Sin embargo, ambos enfoques mantienen abierta una cuestión fundamental: cómo representar y generar acciones continuas con suficiente precisión y velocidad. Los modelos basados en acciones tokenizadas proporcionan una integración directa con los modelos de lenguaje, pero su salida discreta puede limitar el control. Π_0 aborda esta dificultad mediante una arquitectura que combina un backbone visión-lenguaje con un módulo especializado en la generación de acciones continuas [5].

Este módulo recibe el nombre de experto de acción (action expert). El VLM procesa las imágenes y la instrucción para obtener una representación contextual de la tarea, mientras que el experto utiliza esta información, junto con el estado propioceptivo y una secuencia de acciones perturbadas por ruido, para generar un bloque continuo de acciones.

La organización general se presenta en la Figura 2.6. El backbone visión-lenguaje proporciona las representaciones que condicionan las capas de atención cruzada del experto de acción. Las entradas correspondientes al estado y a las acciones se procesan en un flujo especializado, lo que permite conservar el conocimiento semántico del VLM sin exigir que el propio modelo lingüístico genere directamente cada dimensión del control.

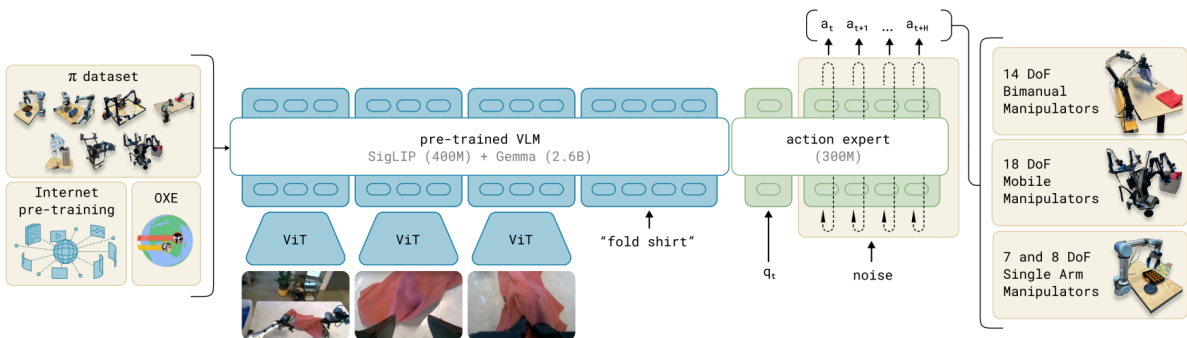


Figura 2.6: Esquema conceptual de Π_0 . El backbone visión-lenguaje procesa las imágenes y la instrucción, mientras que un experto de acción condicionado por sus representaciones genera bloques de acciones continuas [5].

El experto se entrena mediante flow matching, un procedimiento generativo que transforma una secuencia inicial de ruido en un bloque continuo de acciones condi-

cionado por las observaciones, la instrucción y el estado del robot. Este mecanismo permite modelar directamente acciones continuas sin recurrir a su discretización y se desarrolla con mayor detalle en la Sección 3.2.

Esta formulación permite representar distribuciones de acción continuas y multimodales con mayor flexibilidad que la discretización uniforme. También facilita la generación conjunta de bloques temporales, recuperando la idea de agrupación de acciones presentada anteriormente en ACT. La diferencia reside en que el bloque no se obtiene mediante un autoencoder variacional condicionado, sino mediante un proceso generativo condicionado por las representaciones del modelo visión-lenguaje.

π_0 se entrena con datos procedentes de distintas plataformas, entre ellas robots de un brazo, sistemas bimanuales y manipuladores móviles. Esta diversidad busca que el modelo aprenda representaciones compartidas entre tareas y morfologías, aunque cada plataforma disponga de espacios de estado y acción diferentes. Posteriormente, el modelo puede ajustarse con demostraciones específicas de una tarea para mejorar la precisión y la robustez de su ejecución.

La incorporación del experto de acción modifica la relación entre la comprensión multimodal y el control robótico. En RT-2, las acciones se integran directamente en la salida generativa del modelo de visión y lenguaje. En π_0 , el VLM proporciona el contexto semántico y visual, mientras que un módulo especializado modela la distribución continua de acciones. Esta separación funcional influye en arquitecturas posteriores, entre ellas GR00T N1 y SmoVLA, que también combinan un módulo de visión y lenguaje con un componente específico para la generación de movimiento.

En conjunto, RT-2, Octo y π_0 representan distintas etapas en la consolidación de las políticas robóticas generalistas. RT-2 establece una interfaz común para la visión, el lenguaje y las acciones tokenizadas. Octo demuestra la viabilidad de preentrenar una política flexible con datos procedentes de múltiples robots. Por su parte, π_0 introduce un experto de acción capaz de generar bloques continuos mediante flow matching, separando parcialmente la comprensión multimodal de la generación del movimiento.

Junto a esta evolución arquitectónica, OpenVLA constituye un hito transversal al trasladar el paradigma hacia un ecosistema abierto, reproducible y adaptable [6]. La publicación de sus pesos, su código de entrenamiento y sus herramientas de ajuste facilita la reutilización del modelo en nuevas tareas y plataformas, además de proporcionar una base común para desarrollos posteriores. Dado que su principal aportación se relaciona con la apertura, la adaptación eficiente y la accesibilidad de los modelos VLA, estos aspectos se analizan con mayor detalle en la sección dedicada a los modelos abiertos y eficientes.

A partir de estas contribuciones, la investigación deja de centrarse exclusivamente en demostrar la viabilidad de los modelos VLA. Los trabajos posteriores estudian su aplicación en plataformas robóticas más complejas, la ejecución de tareas prolongadas en entornos no observados, la reducción del coste computacional y la incorporación de nuevas fuentes de información, como la memoria, el tacto y la geometría tridimensional.

2.3. Generalización a nuevos entornos y plataformas

Una vez demostrada la viabilidad de los modelos Vision-Language-Action, la investigación comienza a estudiar su aplicación en condiciones más exigentes. Los primeros trabajos muestran que estas políticas pueden seguir instrucciones y ejecutar tareas de manipulación, pero su rendimiento continúa dependiendo en gran medida de la proximidad entre los escenarios de entrenamiento y los de despliegue. La presencia de nuevos entornos, diferentes plataformas robóticas o tareas de mayor duración plantea dificultades adicionales de generalización.

Dentro de esta etapa pueden distinguirse dos direcciones complementarias. GR00T N1 extiende el paradigma hacia plataformas robóticas con morfologías más complejas, especialmente humanoides, y propone una estrategia de entrenamiento basada en fuentes de datos heterogéneas. Por su parte, $\Pi_{0.5}$ estudia la generalización en entornos domésticos no observados durante el entrenamiento y combina predicción semántica de alto nivel con generación continua de acciones. Ambas propuestas muestran que la expansión de los modelos VLA no depende únicamente de aumentar la escala del modelo, sino también de adaptar la arquitectura y la composición de los datos a contextos de ejecución más diversos.

La primera de estas direcciones se encuentra representada por GR00T N1, un modelo fundacional abierto orientado al control de robots humanoides [7]. La transición desde brazos robóticos convencionales o sistemas bimanuales de sobremesa hacia plataformas humanoides incrementa la complejidad del problema. Estas plataformas presentan un mayor número de grados de libertad, espacios de acción más amplios y requisitos de coordinación entre diferentes partes del cuerpo.

Para responder a estas exigencias, GR00T N1 activa una arquitectura compuesta por dos sistemas funcionales. El módulo denominado System 2 utiliza un modelo visión-lenguaje para procesar las observaciones visuales y la instrucción, interpretar el entorno y obtener una representación semántica de la tarea. A continuación, System 1 genera las acciones motoras mediante un Diffusion Transformer entrenado con flow matching.

Ambos componentes se encuentran estrechamente acoplados y se optimizan conjuntamente. La información producida por el módulo visión-lenguaje condiciona directamente la generación de movimiento, evitando una separación completa entre razonamiento y control. Como se muestra en la Figura 2.7, el modelo mantiene flujos diferenciados para la interpretación multimodal y la producción de acciones, pero los integra dentro de un único proceso de entrenamiento extremo a extremo.

La segunda aportación relevante del trabajo se encuentra en la organización de los datos de entrenamiento. La obtención de grandes cantidades de trayectorias reales resulta especialmente costosa en robots humanoides, debido a la dificultad de teleoperar simultáneamente los brazos, las manos y el resto de articulaciones. GR00T N1 aborda esta limitación mediante una combinación de fuentes con diferentes niveles de escala y especificidad.

Esta estrategia se representa mediante una pirámide de datos. En su base se sitúan grandes volúmenes de datos web y vídeos humanos, que proporcionan conocimiento visual y conductual general. El nivel intermedio incorpora datos sintéticos, trayectorias de simulación y secuencias generadas mediante otros modelos. En la parte

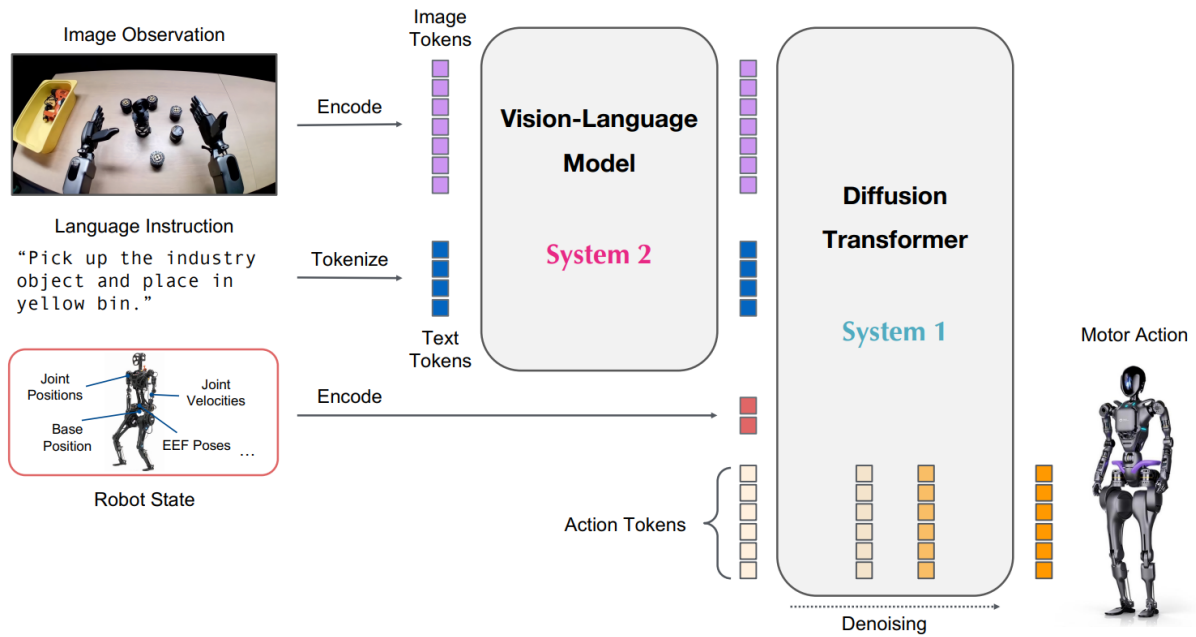


Figura 2.7: Arquitectura dual de GR00T N1. El módulo visión-lenguaje, denominado System 2, interpreta las observaciones y la instrucción, mientras que System 1 genera acciones motoras mediante un Diffusion Transformer condicionado por las representaciones anteriores [7].

superior se encuentran las demostraciones recopiladas directamente sobre robots físicos, de menor volumen pero estrechamente relacionadas con la ejecución real.

La Figura 2.8 resume esta distribución. A medida que se asciende en la pirámide disminuye la cantidad de información disponible, pero aumenta su correspondencia con la plataforma y el espacio de acción del robot.



Figura 2.8: Pirámide de datos utilizada en GR00T N1. Los datos web y los vídeos humanos proporcionan la base de mayor escala; los datos sintéticos y simulados ocupan el nivel intermedio; y las trayectorias de robots físicos constituyen la fuente de mayor especificidad encarnada [7].

La propuesta también incorpora vídeos que no contienen acciones robóticas explíci-

2.3. Generalización a nuevos entornos y plataformas

tas. Para utilizarlos durante el entrenamiento, el trabajo obtiene acciones latentes o pseudoacciones mediante modelos de dinámica inversa [7]. De este modo, secuencias humanas o sintéticas pueden aportar información sobre cambios visuales y comportamientos, aunque no dispongan originalmente de comandos articulares asociados.

Esta formulación amplía el concepto de demostración empleado en el aprendizaje por imitación. El entrenamiento deja de depender exclusivamente de trayectorias robóticas completamente etiquetadas y pasa a integrar información procedente de distintos dominios. GR00T N1 muestra así que la extensión de los VLA hacia robots humanoides requiere no solo adaptar la arquitectura, sino también diseñar procedimientos capaces de aprovechar fuentes de datos con diferentes grados de relación con el control final.

Sin embargo, ampliar el paradigma a nuevas morfologías no resuelve por sí solo el problema de la generalización. Incluso cuando una política puede controlar plataformas más complejas y aprovechar fuentes heterogéneas de datos, sigue siendo necesario comprobar si ese conocimiento permite actuar en escenarios distintos de los observados durante el entrenamiento. Esta cuestión adquiere especial relevancia en entornos domésticos, donde la distribución de los objetos, las condiciones visuales y la secuencia de acciones necesarias pueden variar de forma considerable entre viviendas.

En este contexto, $\Pi_{0.5}$ se centra en la generalización en entornos domésticos no incluidos en los datos de entrenamiento [8]. El objetivo consiste en controlar un manipulador móvil durante tareas prolongadas en viviendas con distribuciones, objetos y condiciones visuales diferentes de las utilizadas durante el aprendizaje.

La propuesta combina datos procedentes de múltiples robots, tareas de percepción, anotaciones semánticas, instrucciones verbales y ejemplos multimodales obtenidos de la web. El conjunto más directamente relacionado con la aplicación final contiene aproximadamente 400 horas de manipulación móvil en hogares. Sin embargo, durante la primera fase de entrenamiento, el 97,6

Esta composición responde a la dificultad de recopilar suficientes trayectorias para cubrir todas las configuraciones que pueden aparecer en una vivienda. En lugar de intentar representar la variabilidad del mundo real exclusivamente mediante demostraciones del robot objetivo, el modelo transfiere conocimiento desde tareas y dominios relacionados. De este modo, $\Pi_{0.5}$ comparte con GR00T N1 la idea de que los datos robóticos específicos deben complementarse con fuentes más amplias, aunque orienta esta estrategia hacia la transferencia entre escenarios y no hacia la adaptación a una morfología concreta.

El entrenamiento se organiza en dos etapas. Durante la primera, $\Pi_{0.5}$ utiliza una formulación autorregresiva en la que las acciones se representan mediante tokens discretizados con la tokenización FAST. En esta fase se combinan ejemplos de predicción de acciones con tareas como generación de subtareas semánticas, localización mediante cajas delimitadoras, descripción de imágenes y pregunta-respuesta visual.

Esta interfaz permite tratar diferentes fuentes de supervisión como secuencias multimodales dentro de un mismo procedimiento de entrenamiento. Las tareas perceptivas y semánticas contribuyen a la interpretación del entorno, mientras que las trayectorias robóticas proporcionan la relación entre observaciones y acciones.

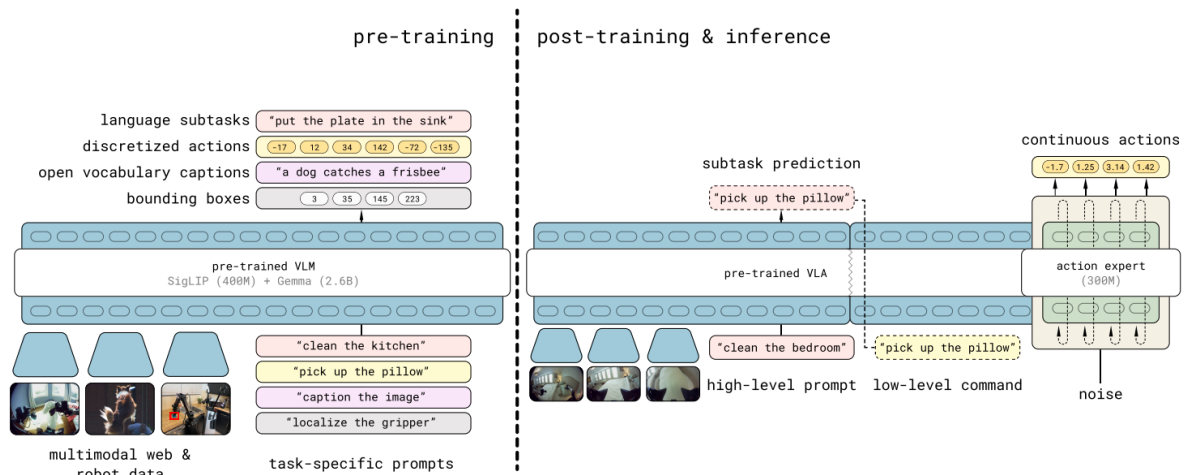


Figura 2.9: Proceso de entrenamiento e inferencia de $\Pi_{0.5}$. El preentrenamiento integra fuentes heterogéneas mediante acciones discretizadas y tareas multimodales; posteriormente, un experto de acción basado en flow matching genera bloques de acciones continuas para el control robótico [8].

En una segunda etapa, el modelo incorpora el experto de acción continuo utilizado en Π_0 . Este componente se entrena mediante flow matching para generar bloques de acciones continuas. La secuencia completa se representa en la Figura 2.9: el pre-entrenamiento permite integrar datos heterogéneos mediante una interfaz discreta, mientras que el postentrenamiento adapta la política a las exigencias del control continuo.

La combinación de ambas etapas permite aprovechar de forma diferenciada el conocimiento semántico y las capacidades motoras. Durante la inferencia, esta separación se mantiene mediante una organización jerárquica. El modelo predice primero una subtarea semántica de alto nivel, como seleccionar o desplazar un objeto concreto. A continuación, el experto de acción genera los comandos de bajo nivel condicionados por dicha subtarea.

Esta división permite aprovechar de forma diferenciada las fuentes de entrenamiento. Los datos web, las anotaciones visuales y las instrucciones verbales refuerzan la interpretación semántica y la selección de subtarear. Las demostraciones de diferentes robots, por su parte, contribuyen al aprendizaje de habilidades de manipulación y secuencias de movimiento.

La Figura 2.10 muestra las principales fuentes utilizadas por el modelo y su relación con los dos niveles de la política.

El trabajo muestra que esta estrategia permite ejecutar tareas de entre diez y quince minutos, como la limpieza de cocinas y dormitorios, en viviendas que no aparecen durante el entrenamiento [8]. La principal contribución no reside únicamente en la duración de estas ejecuciones, sino en demostrar que los datos directamente relacionados con la plataforma final pueden complementarse con información procedente de otros robots y tareas multimodales.

GR00T N1 y $\Pi_{0.5}$ representan, por tanto, dos formas complementarias de ampliar el paradigma VLA. GR00T N1 adapta la arquitectura y los datos al control de pla-

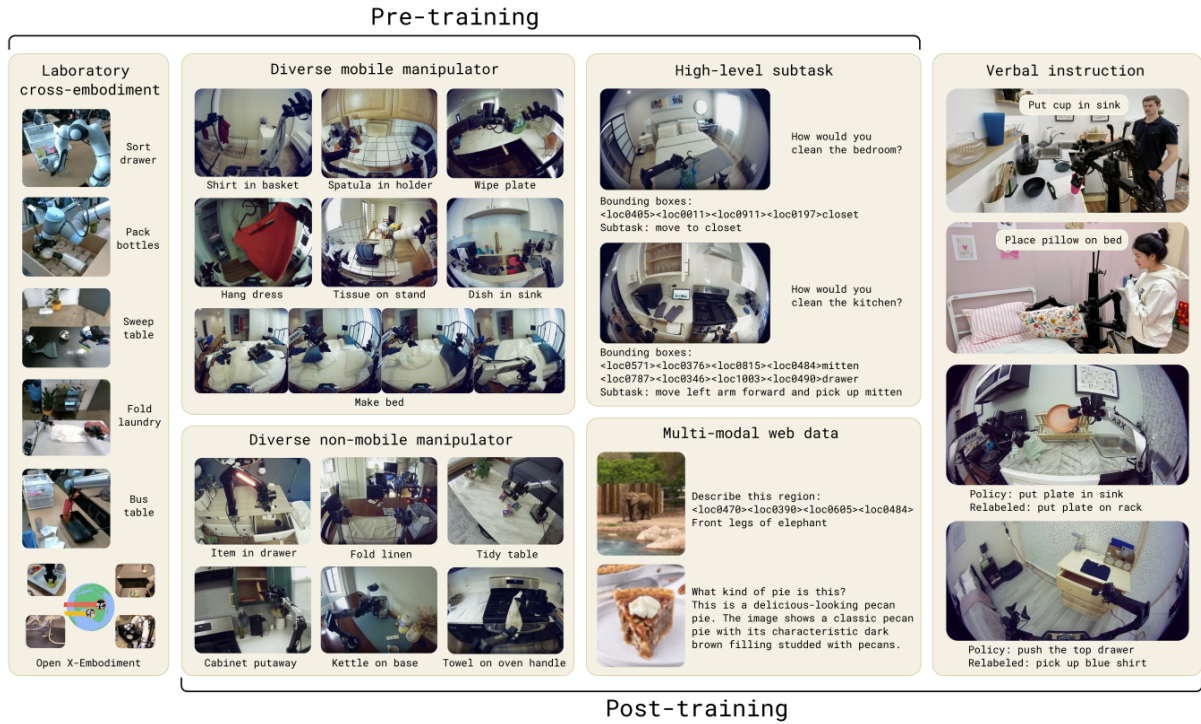


Figura 2.10: Fuentes heterogéneas utilizadas en $\pi_{0.5}$. El modelo combina trayectorias robóticas, predicción de subtareas, instrucciones verbales, detección de objetos y ejemplos multimodales procedentes de la web para favorecer la transferencia hacia entornos domésticos no observados [8].

taformas humanoides, donde aumenta la complejidad de la morfología y del espacio de acción. $\pi_{0.5}$ utiliza coentrenamiento heterogéneo y una política jerárquica para mejorar la transferencia hacia entornos domésticos no observados y tareas de mayor duración.

En ambos casos, la capacidad de generalización depende no solo del tamaño del modelo, sino también de la composición de los datos y de la forma en que se relacionan la comprensión semántica y el control motor. Esta evolución desplaza el foco desde la ejecución de tareas en configuraciones conocidas hacia políticas capaces de actuar sobre plataformas y escenarios cada vez más diversos.

2.4. Modelos abiertos y eficiencia

La consolidación de un paradigma de aprendizaje robótico requiere que sus modelos puedan estudiarse, reproducirse y adaptarse a nuevas plataformas. En el caso de los VLA, esta necesidad adquiere especial importancia debido al coste asociado a los grandes modelos multimodales, a la recopilación de demostraciones robóticas y al entrenamiento sobre infraestructuras especializadas.

A partir de esta necesidad surge una línea orientada a construir un ecosistema más abierto y accesible. Las propuestas incluidas en este ámbito abordan la publicación de pesos y código, el ajuste eficiente, la reducción del tamaño de las arquitecturas y el uso de plataformas robóticas de menor coste. Estos avances permiten ampliar

la participación en el desarrollo de modelos VLA y facilitan su aplicación fuera de entornos industriales con grandes recursos.

Como se menciona en la Sección 2.2, OpenVLA traslada la formulación VLA hacia un entorno abierto y reproducible [6]. El modelo contiene aproximadamente 7.000 millones de parámetros y se entrena con 970.000 episodios de Open X-Embodiment.

La arquitectura combina dos codificadores visuales con funciones complementarias. DINOv2 aporta características relacionadas con la estructura espacial de las imágenes, mientras que SigLIP proporciona representaciones con mayor alineación semántica respecto al lenguaje. Las salidas de ambos codificadores se fusionan y se proyectan al espacio de entrada de un backbone Llama 2.

La instrucción y las características visuales se procesan conjuntamente para generar tokens de acción. Como en RT-2, las dimensiones continuas del control se discretizan, se asocian a elementos del vocabulario y se producen de forma autorregresiva. Finalmente, los tokens se convierten de nuevo en desplazamientos, rotaciones y comandos para la pinza.

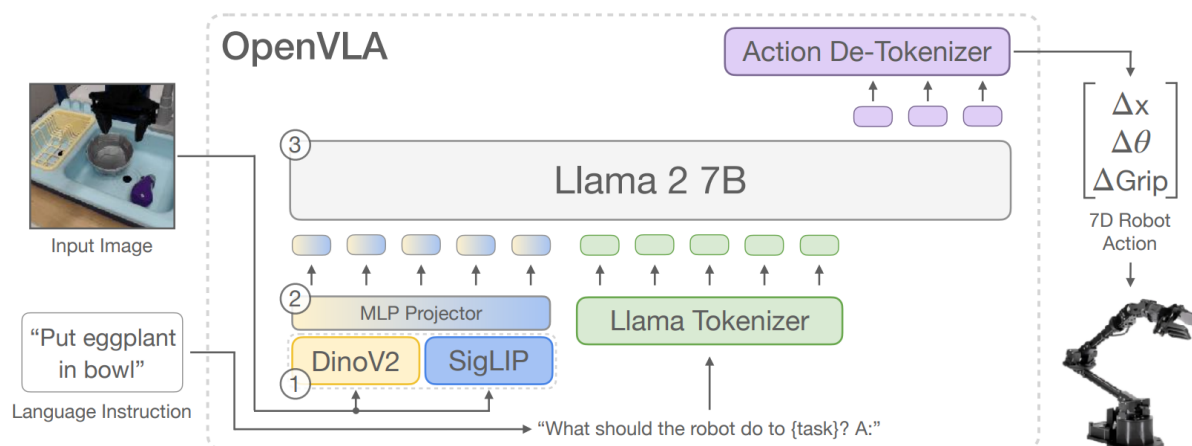


Figura 2.11: Arquitectura de OpenVLA. Las representaciones visuales extraídas mediante DINOv2 y SigLIP se proyectan al espacio de Llama 2, que genera tokens discretizados posteriormente convertidos en comandos continuos de control [6].

La principal aportación de OpenVLA dentro de esta línea consiste en publicar los pesos, el código de entrenamiento y las herramientas necesarias para adaptar el modelo. Esta disponibilidad permite examinar la composición de la arquitectura, modificar los procedimientos de aprendizaje y comparar nuevas propuestas sobre una base compartida.

El trabajo estudia además la adaptación del modelo mediante Low-Rank Adaptation (LoRA). Esta técnica introduce matrices entrenables de bajo rango en determinados componentes de la red y mantiene congelada la mayor parte de los parámetros originales. Según los resultados presentados en el trabajo, este procedimiento permite aproximar el rendimiento del ajuste completo actualizando únicamente una fracción reducida del modelo [6].

También se analiza el uso de cuantización a 4 bits durante la inferencia. La reducción de la precisión numérica disminuye el consumo de memoria y facilita el despliegue

en hardware con menor capacidad, sin producir una degradación significativa del rendimiento descrito en el estudio [6].

OpenVLA convierte así la accesibilidad en una dimensión central del desarrollo de los VLA. La publicación del proceso completo y la evaluación de técnicas de ajuste eficiente permiten utilizar el modelo como punto de partida para nuevas tareas sin repetir el coste de su preentrenamiento generalista.

Sin embargo, la apertura del modelo y la reducción del número de parámetros actualizados durante su adaptación no eliminan el coste asociado a una arquitectura de varios miles de millones de parámetros. El ajuste eficiente facilita la reutilización de OpenVLA, pero la inferencia continúa condicionada por el tamaño de su backbone y por la generación autorregresiva de las acciones. La siguiente evolución dentro de esta línea consiste, por tanto, en abordar la eficiencia desde el propio diseño de la arquitectura.

TinyVLA se orienta a estas dos limitaciones: el elevado coste de inferencia de los grandes VLA y la cantidad de datos robóticos necesaria para su entrenamiento [9]. La propuesta sustituye los backbones de varios miles de millones de parámetros por modelos visión-lenguaje compactos de menos de 1.000 millones de parámetros.

Además de reducir el tamaño del componente multimodal, TinyVLA modifica la forma de generar las acciones. En lugar de utilizar una salida autorregresiva basada en tokens discretizados, incorpora un decodificador de política basado en difusión. Este módulo produce secuencias de acciones continuas y evita generar por separado cada componente del control.

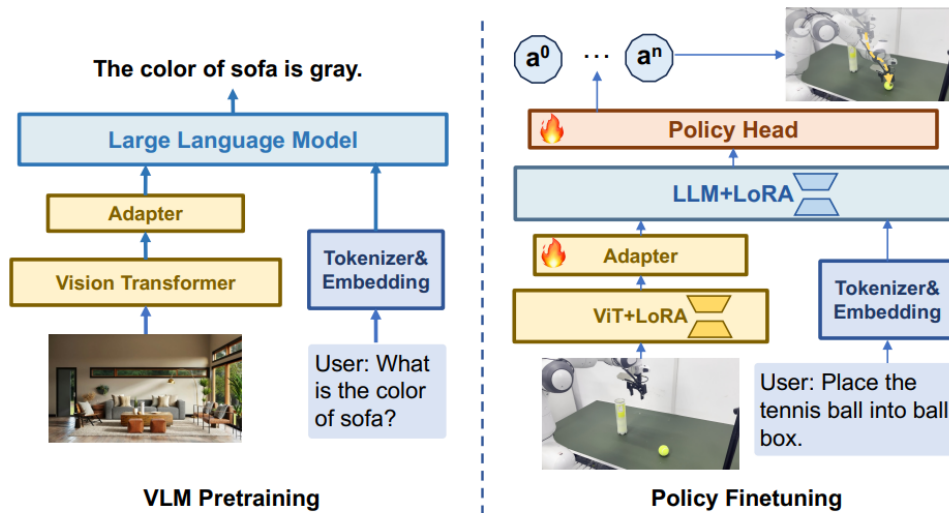


Figura 2.12: Arquitectura y proceso de adaptación de TinyVLA. Un modelo visión-lenguaje compacto se combina con una política de difusión que genera secuencias continuas de acciones, evitando la predicción autorregresiva de tokens de control [9].

Esta formulación busca conservar las representaciones aprendidas durante el preentrenamiento visión-lenguaje sin depender de un preentrenamiento adicional sobre grandes conjuntos de datos robóticos. La política se adapta directamente a las demostraciones correspondientes a las tareas objetivo.

Los resultados descritos en el trabajo muestran mejoras de velocidad y eficiencia de

datos respecto a OpenVLA, junto con un rendimiento comparable o superior en diferentes configuraciones de simulación y robots reales [9]. También se estudian tareas bimanuales, para las que la utilización de un modelo compacto y un decodificador continuo evita depender de que la distribución de preentrenamiento robótico contenga exactamente la misma morfología.

TinyVLA muestra que la reducción de escala no tiene que limitarse a comprimir un modelo existente. La arquitectura, el objetivo de entrenamiento y la representación de las acciones pueden modificarse conjuntamente para reducir la latencia y mejorar el aprovechamiento de un conjunto limitado de demostraciones.

La transición de OpenVLA a TinyVLA refleja dos niveles complementarios de accesibilidad. OpenVLA permite estudiar, adaptar y desplegar una política generalista previamente entrenada mediante herramientas abiertas y técnicas de ajuste eficiente. TinyVLA traslada esta preocupación al diseño interno del modelo, reduciendo el tamaño del backbone y sustituyendo la generación autorregresiva por un decodificador continuo. Sobre esta evolución se desarrolla SmolVLA, que profundiza en la eficiencia y añade una orientación explícita hacia los datos comunitarios, las plataformas asequibles y la continuidad del control durante la inferencia.

2.4.1. SmolVLA: robótica asequible

SmolVLA continúa la línea de modelos compactos, pero incorpora además una orientación explícita hacia plataformas robóticas asequibles y conjuntos de datos aportados por la comunidad [10]. El modelo combina un VLM de tamaño reducido con un experto de acción entrenado mediante flow matching.

El componente visión-lenguaje procesa la instrucción y las imágenes, mientras que el experto recibe sus representaciones junto con el estado del robot y una secuencia de acciones con ruido. El experto alterna capas de autoatención y atención cruzada para generar un bloque de acciones continuas.

La arquitectura introduce varias decisiones dirigidas a reducir el coste computacional. Entre ellas se encuentran la eliminación de las últimas capas del VLM, la utilización de un número reducido de tokens visuales y el empleo de capas de atención cruzada más ligeras dentro del experto de acción.

SmolVLA también modifica la ejecución de la política mediante una pila de inferencia asíncrona. En un esquema síncrono, el robot debe esperar a que finalicen la captura de la observación, su procesamiento y la generación del siguiente bloque antes de continuar con la ejecución. Esta dependencia puede introducir pausas cuando el tiempo de inferencia es elevado.

La inferencia asíncrona desacopla la producción de nuevos bloques de acciones de la ejecución del bloque actual. Mientras el robot consume las acciones ya disponibles, el modelo procesa observaciones recientes y prepara la siguiente secuencia. La Figura 2.14 representa esta superposición entre predicción y control.

Este desacoplamiento se produce en el flujo de ejecución y no implica que las modalidades se procesen de forma independiente dentro del modelo. El experto de acción continúa condicionado por las representaciones obtenidas del VLM. La diferencia

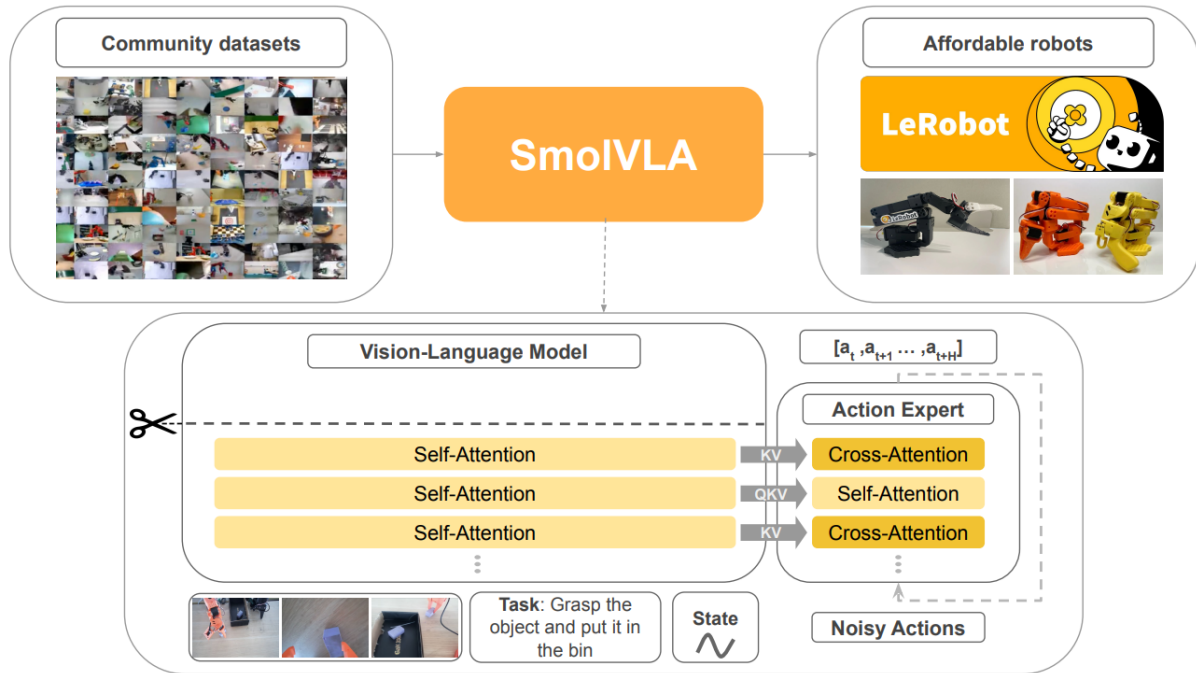


Figura 2.13: Esquema general de SmoIVLA. El modelo combina un VLM compacto con un experto de acción entrenado mediante flow matching y utiliza conjuntos de datos públicos recopilados sobre plataformas robóticas asequibles [10].

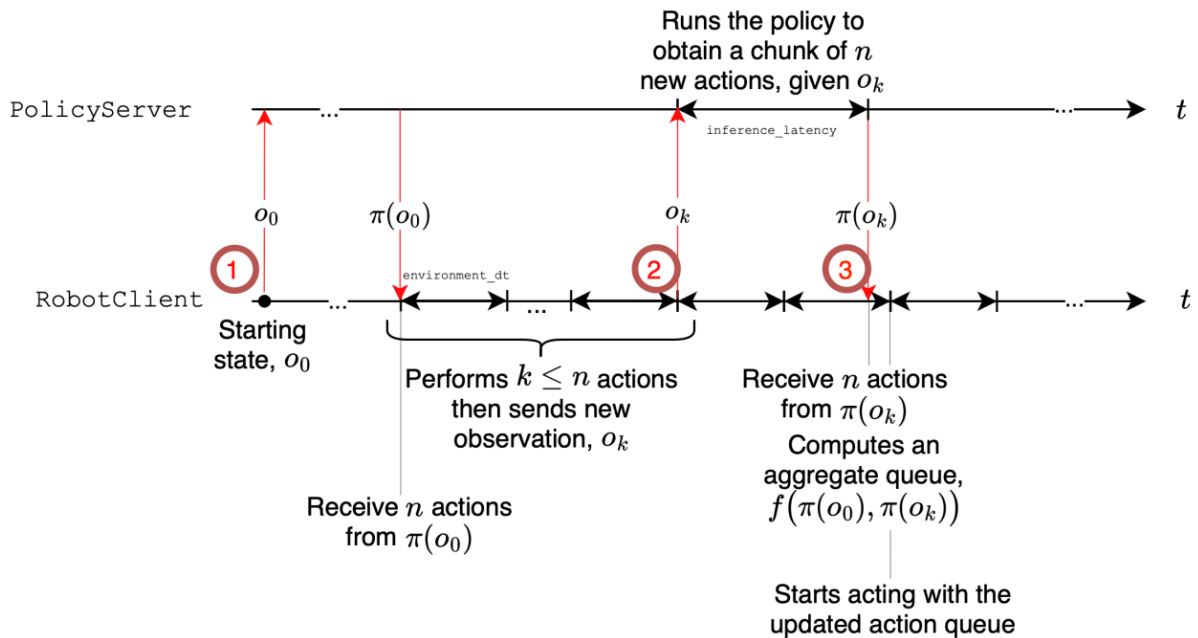


Figura 2.14: Pila de inferencia asíncrona de SmoIVLA. La generación de nuevos bloques de acciones se solapa con la ejecución de los bloques disponibles, reduciendo los periodos de espera entre la percepción, la predicción y el control [10].

consiste en que la predicción y la actuación no deben finalizar de forma estrictamente secuencial en cada instante.

Esta característica contrasta con el acoplamiento empleado en GR00T N1. En ambos modelos, la información visión-lenguaje condiciona directamente la generación de acciones. Sin embargo, SmoVLA introduce una separación temporal durante la inferencia para mejorar la continuidad del control, mientras que GR00T N1 se orienta a la integración conjunta de razonamiento y actuación en plataformas humanoides [7], [10].

OpenVLA, TinyVLA y SmoVLA representan tres niveles complementarios de accesibilidad. OpenVLA proporciona una base abierta y procedimientos de ajuste eficiente. TinyVLA reduce el tamaño y sustituye la generación autorregresiva por una política continua basada en difusión. SmoVLA combina una arquitectura compacta, datos comunitarios e inferencia asíncrona para facilitar el despliegue sobre plataformas y recursos más limitados.

Esta evolución muestra que el desarrollo de los VLA no depende únicamente de aumentar el número de parámetros o el volumen de datos. La apertura del código, la posibilidad de adaptación, la latencia de inferencia y la disponibilidad de plataformas de recogida de datos constituyen también factores determinantes para su utilización práctica.

2.5. Percepción táctil, memoria y tareas complejas

La consolidación de los modelos VLA permite desplazar parte de la investigación desde la integración básica de visión, lenguaje y acción hacia la adquisición de capacidades perceptivas y temporales más amplias. En tareas reales, una única observación visual puede resultar insuficiente para describir el estado del entorno, especialmente cuando existen contactos difíciles de interpretar, oclusiones o dependencias entre distintas etapas de la ejecución.

Por este motivo, las arquitecturas recientes incorporan modalidades adicionales y mecanismos capaces de conservar información a lo largo del tiempo. Estas extensiones permiten representar tanto aspectos físicos de la interacción como acontecimientos que ya no son visibles, proporcionando a la política un contexto más completo para seleccionar sus acciones. Dentro de esta línea, VTLA amplía la percepción instantánea mediante información táctil, mientras que MEM incorpora memoria para conservar el contexto relevante de observaciones y acciones anteriores.

La mayoría de los modelos VLA revisados anteriormente utilizan imágenes RGB como principal fuente de percepción del entorno. Esta información permite reconocer objetos, interpretar instrucciones y estimar relaciones espaciales visibles. Sin embargo, existen tareas en las que el estado relevante de la interacción depende también de fuerzas, contactos o pequeñas desalineaciones que no pueden inferirse con suficiente precisión a partir de la imagen.

Esta limitación resulta especialmente importante en operaciones de inserción. Cuando un objeto entra en contacto con una superficie o con los bordes de un orificio, diferencias espaciales de pequeña magnitud pueden determinar si la acción progresa correctamente o produce una colisión. En estas situaciones, la información táctil

2.5. Percepción táctil, memoria y tareas complejas

proporciona señales locales sobre el contacto que complementan la percepción visual global.

VTLA amplía el paradigma mediante un modelo Vision-Tactile-Language-Action diseñado para tareas de inserción precisa, en las que una pieza debe alinearse e introducirse en una cavidad con la geometría correspondiente [11]. La política procesa una instrucción lingüística, una imagen obtenida mediante una cámara situada en la muñeca y secuencias de imágenes táctiles capturadas por sensores ubicados en los dedos de la pinza.

La Figura 2.15 presenta la organización general de la propuesta. Las modalidades visual y táctil se integran dentro de un modelo visión-lenguaje, que genera una acción destinada a corregir la desalineación existente entre la pieza y el orificio.

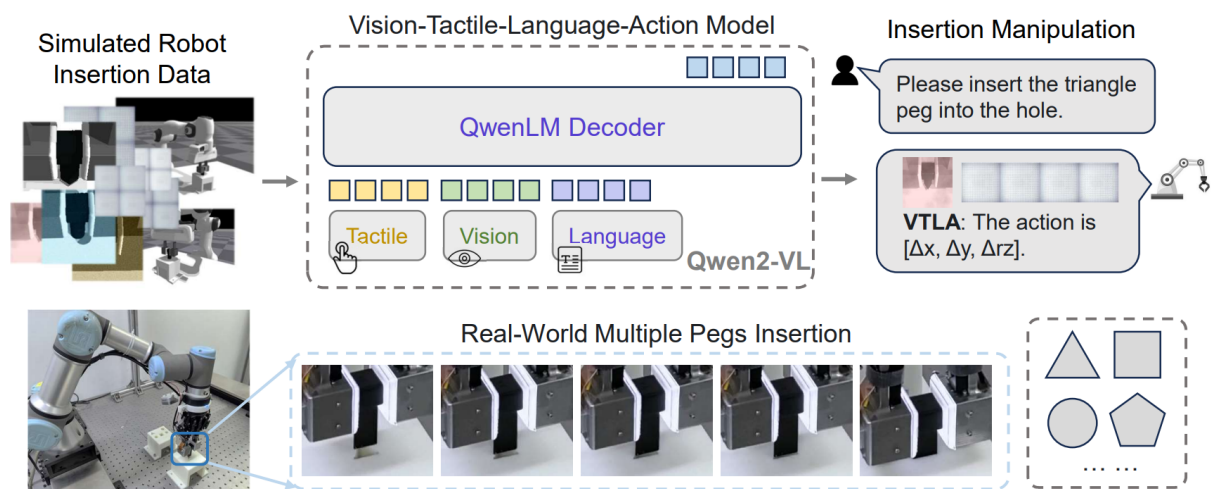


Figura 2.15: Visión general de VTLA. El modelo procesa observaciones visuales y táctiles junto con una instrucción lingüística para generar acciones en tareas de inserción peg-in-hole [11].

La integración de las observaciones táctiles presenta una dificultad temporal. Mientras que la imagen visual proporciona una descripción global del estado actual, la información táctil se obtiene como una secuencia breve cuyos cambios reflejan la evolución del contacto. Un modelo visión-lenguaje convencional puede procesar imágenes individuales, pero no está necesariamente preparado para representar las dependencias temporales de este tipo de señales.

Para abordar este problema, VTLA introduce Vision-Guided Temporally Enhanced Tokens (VGTE). Este mecanismo transforma las secuencias táctiles en representaciones sensibles a su evolución temporal antes de incorporarlas al modelo. La información visual actúa como referencia global, mientras que los tokens táctiles aportan detalles locales sobre el contacto y su variación durante los intentos de inserción [11].

El entrenamiento se organiza en dos fases. En la primera, el modelo se ajusta mediante predicción del siguiente token sobre un conjunto de datos formado por observaciones visuales, observaciones táctiles, instrucciones y acciones. Las acciones continuas se representan mediante tokens, manteniendo una formulación similar a la empleada por los VLA autorregresivos.

Esta representación introduce una diferencia entre el objetivo de entrenamiento y

la naturaleza del problema. La predicción del siguiente token se plantea como una tarea de clasificación sobre elementos discretos, mientras que el control robótico requiere aproximar valores continuos. Para reducir esta discrepancia, la segunda fase incorpora Direct Preference Optimization (DPO).

El procedimiento genera distintas predicciones de acción para una misma observación y las ordena según su proximidad respecto a la acción de referencia. Las acciones más cercanas se consideran preferidas y las más alejadas se utilizan como ejemplos rechazados. De este modo, el modelo recibe una señal adicional que aproxima el comportamiento de una supervisión de regresión sin abandonar completamente su formulación generativa [11].

La Figura 2.16 resume ambas fases. El ajuste supervisado establece inicialmente la relación entre las modalidades y las acciones, mientras que DPO refina posteriormente las predicciones utilizando pares de acciones preferidas y rechazadas.

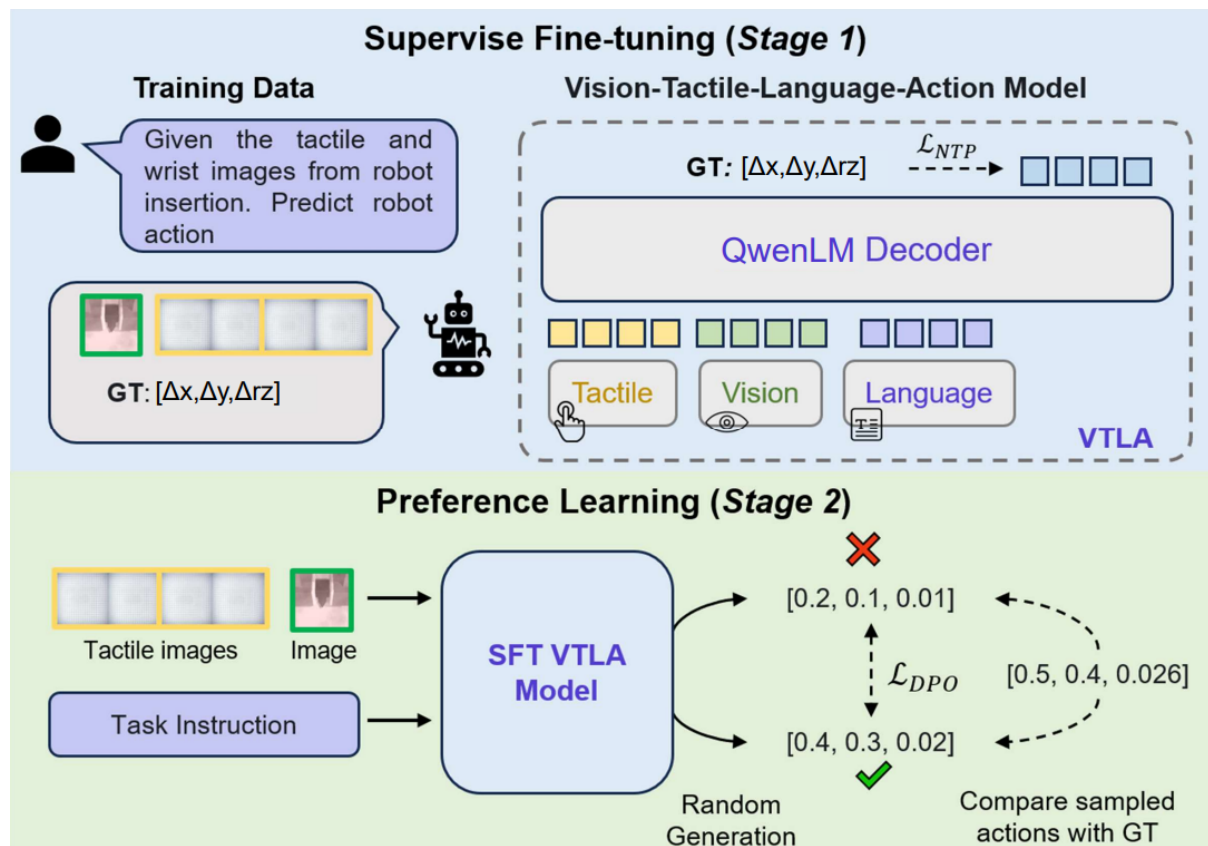


Figura 2.16: Proceso de entrenamiento de VTLa. En la primera fase, el modelo se ajusta mediante predicción del siguiente token. En la segunda, DPO introduce una señal basada en preferencias para aproximar mejor la naturaleza continua de las acciones robóticas [11].

El conjunto de entrenamiento se obtiene en simulación mediante tareas de inserción con diferentes formas geométricas y niveles de holgura. Posteriormente, el modelo se evalúa sobre un sistema físico sin realizar un nuevo entrenamiento con datos reales. Los resultados presentados muestran tasas de éxito superiores al 90

Las comparaciones también indican que la combinación de visión y tacto supera a las

2.5. Percepción táctil, memoria y tareas complejas

variantes que utilizan únicamente una de las dos modalidades. La visión proporciona información global sobre la posición y la geometría de los elementos, mientras que el tacto permite interpretar el estado local del contacto. La aportación de VTLA reside, por tanto, en mostrar que la multimodalidad de los VLA puede extenderse más allá de la relación entre imagen y lenguaje cuando la tarea exige información física que no se encuentra completamente representada en los datos visuales.

La incorporación de señales táctiles amplía la información disponible sobre la interacción actual. Sin embargo, disponer de más modalidades en cada instante no resuelve las tareas en las que parte del contexto necesario pertenece al pasado. Una política puede interpretar con mayor precisión el contacto presente y, aun así, perder información sobre objetos temporalmente ocultos, acciones ya realizadas o etapas completadas durante una ejecución prolongada.

Muchas políticas robóticas actúan únicamente a partir de la observación actual o de una ventana temporal reducida. Esta formulación resulta insuficiente cuando un objeto queda oculto, cuando una acción anterior modifica el estado de la tarea o cuando la ejecución requiere recordar qué etapas ya se han completado.

Una posible solución consiste en introducir todas las observaciones anteriores dentro del contexto de la política. Sin embargo, este procedimiento incrementa el coste computacional a medida que avanza la ejecución. Además, no toda la información pasada requiere el mismo nivel de detalle ni debe conservarse durante el mismo intervalo.

MEM parte de esta diferencia y propone una arquitectura de memoria multiescala para modelos VLA [12]. La propuesta distingue entre información visual reciente, necesaria para interpretar movimientos y oclusiones, e información semántica de mayor duración, utilizada para representar el progreso global de la tarea.

La primera componente es una memoria visual a corto plazo (short-term video memory). Esta memoria conserva una secuencia densa de observaciones recientes y la comprime mediante un codificador de vídeo eficiente. Su objetivo consiste en mantener detalles visuales y temporales que pueden resultar necesarios para interpretar la dinámica de la escena.

Esta representación permite, por ejemplo, recordar la posición de un objeto cuando el brazo del robot lo oculta temporalmente, detectar que un agarre no se ha completado o modificar una estrategia de manipulación después de un intento fallido. Al utilizar una secuencia de imágenes próximas, la política dispone de información sobre los cambios recientes y no depende exclusivamente de una imagen aislada.

La segunda componente es una memoria textual a largo plazo (long-term text memory). En lugar de conservar imágenes durante toda la ejecución, el sistema mantiene un resumen lingüístico de los acontecimientos pasados que continúan siendo relevantes. Esta representación puede indicar qué ingredientes ya se han utilizado, qué zonas se han limpiado o qué subtareas permanecen pendientes [12].

La Figura 2.17 muestra esta separación. La memoria visual conserva información densa durante un horizonte reducido, mientras que la memoria textual comprime el progreso de la tarea mediante conceptos semánticos que pueden mantenerse durante intervalos más amplios.

Esta organización evita aplicar una única representación a todos los horizontes tem-

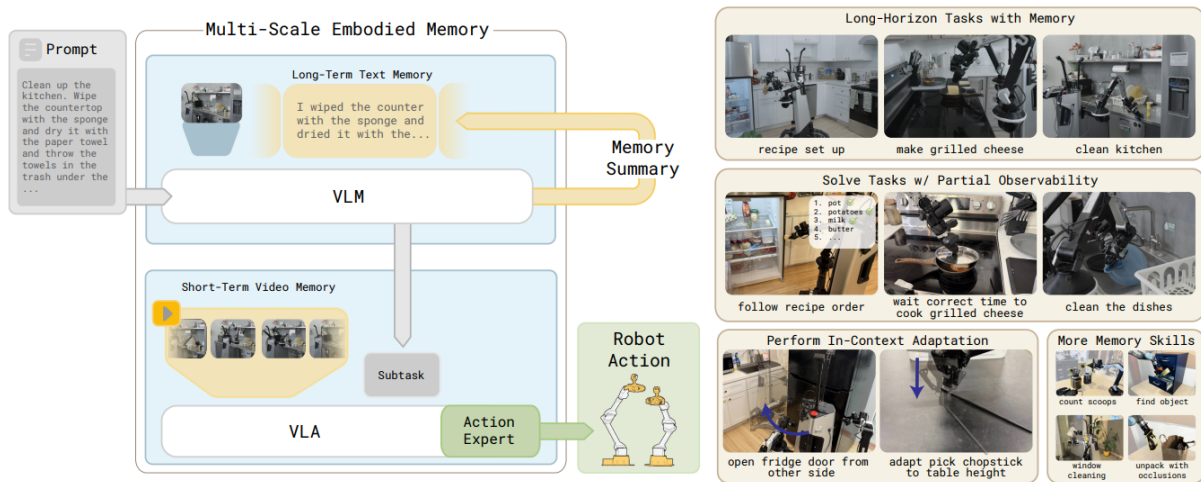


Figura 2.17: Visión general de MEM. El sistema combina una memoria visual a corto plazo, basada en secuencias de vídeo, con una memoria textual a largo plazo que resume los acontecimientos semánticos relevantes para la tarea [12].

porales. Las imágenes preservan detalles espaciales y dinámicos, pero su almacenamiento y procesamiento resultan costosos. El lenguaje proporciona una representación más compacta, aunque no conserva con la misma precisión la geometría o el movimiento. MEM asigna cada formato al tipo de información para el que resulta más adecuado.

La arquitectura se divide funcionalmente en una política de alto nivel y otra de bajo nivel. La política de alto nivel actualiza la memoria lingüística y genera la subtarea que debe ejecutarse a continuación. La política de bajo nivel recibe dicha subtarea, las observaciones actuales, la memoria visual reciente y el resumen textual para producir las acciones continuas mediante un experto de acción.

La Figura 2.18 representa esta relación. El nivel superior mantiene el estado semántico de la tarea, mientras que el inferior utiliza información visual más detallada para controlar el robot. Ambas escalas de memoria se combinan en la generación de las acciones, pero cumplen funciones diferentes dentro del sistema.

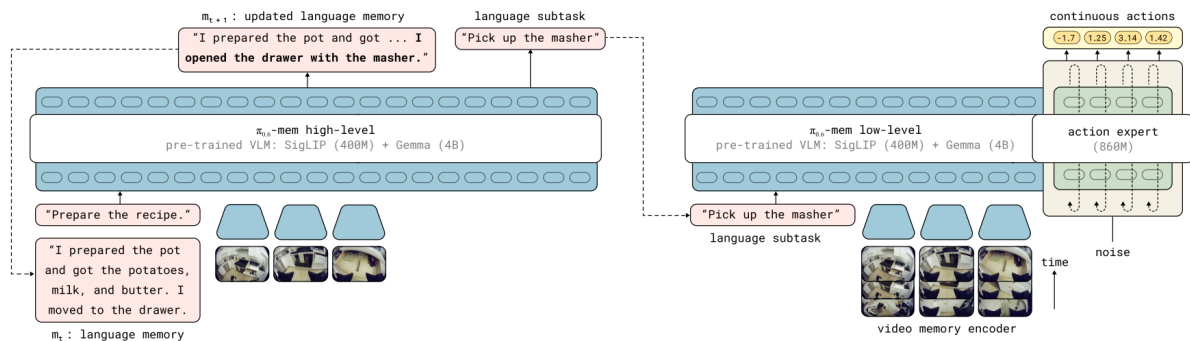


Figura 2.18: Arquitectura funcional de MEM. La política de alto nivel actualiza la memoria textual y selecciona subtarear, mientras que la política de bajo nivel utiliza observaciones recientes, memoria visual y contexto semántico para generar acciones continuas [12].

2.6. Representación geométrica tridimensional

El codificador de vídeo resulta necesario para mantener la latencia dentro de los límites requeridos por el control robótico. Procesar cada imagen histórica de forma independiente mediante el backbone completo del VLA incrementaría progresivamente el tiempo de inferencia. La compresión temporal permite utilizar varias observaciones recientes sin procesarlas todas como entradas visuales independientes [12].

Los resultados presentados muestran que MEM puede conservar información durante tareas de hasta quince minutos. Entre las evaluaciones se incluyen actividades prolongadas, como limpiar una cocina o preparar una receta, en las que la política debe coordinar múltiples etapas y recordar acontecimientos que ya no son visibles en la observación actual [12].

La memoria a corto plazo también mejora la respuesta ante observabilidad parcial y autooclusiones. Asimismo, permite realizar una adaptación in context de las estrategias de manipulación: después de observar un fallo, la política puede utilizar la secuencia reciente para modificar el siguiente intento sin actualizar sus parámetros.

MEM muestra que añadir memoria a un VLA no consiste únicamente en ampliar su ventana de entrada. La política debe seleccionar qué información conservar, durante cuánto tiempo y mediante qué representación. La separación entre memoria visual y textual permite combinar precisión local y compresión semántica sin exigir que todo el historial se procese con el mismo nivel de detalle.

VTLA y MEM amplían el paradigma en direcciones diferentes, pero responden a una limitación común: la observación visual actual no contiene siempre toda la información necesaria para actuar. VTLa incorpora señales táctiles para representar el estado físico local de una interacción. MEM conserva información procedente de instantes anteriores para resolver oclusiones, adaptar estrategias y mantener el progreso de tareas prolongadas.

Estas propuestas muestran que la evolución de los VLA no se limita al aumento de escala. Una política más general también debe integrar modalidades adecuadas al tipo de interacción y mantener representaciones temporales compatibles con la duración de la tarea. La siguiente línea de investigación aborda otra limitación relacionada con la información disponible para el control: la ausencia de una representación geométrica tridimensional explícita en modelos construidos principalmente sobre observaciones bidimensionales.

2.6. Representación geométrica tridimensional

Además de ampliar las modalidades perceptivas y el contexto temporal, las políticas robóticas deben representar con suficiente precisión la estructura espacial del entorno. Las imágenes RGB aportan información visual y semántica relevante, pero reducen una escena tridimensional a una proyección bidimensional. Esta pérdida de información puede generar ambigüedades en la estimación de la profundidad, la altura, la orientación o la distancia entre objetos, aspectos directamente relacionados con la ejecución de movimientos en tareas de manipulación.

La incorporación de estructura tridimensional al aprendizaje robótico cuenta con antecedentes anteriores al desarrollo de los modelos VLA. Como se describe en la Sección 2.1, PerAct utiliza observaciones RGB-D para construir una representación

voxelizada del entorno y predecir acciones discretizadas de seis grados de libertad [1]. Esta formulación muestra que proporcionar una representación geométrica explícita puede facilitar la relación entre percepción y acción. A partir de esta base, la investigación posterior estudia cómo trasladar la información tridimensional a políticas generativas y a modelos VLA preentrenados, procurando conservar las representaciones visuales y semánticas adquiridas previamente.

Un primer paso en esta evolución consiste en comprobar si la geometría tridimensional también puede mejorar políticas que generan secuencias continuas de acciones. Las políticas visuales basadas exclusivamente en imágenes bidimensionales pueden necesitar un número elevado de demostraciones y presentar dificultades para generalizar cuando cambian la posición de los objetos, el punto de vista o la apariencia de la escena. DP3, abreviatura de 3D Diffusion Policy, aborda esta limitación mediante una política de difusión condicionada por una representación tridimensional compacta [13].

El modelo obtiene información de profundidad y la transforma en una nube de puntos. Esta representación conserva la posición tridimensional de un conjunto de puntos observados en la escena, proporcionando información explícita sobre la geometría y la distribución espacial de los objetos. Antes de utilizarse como condición de la política, la nube se recorta y se submuestraa para reducir su tamaño.

A continuación, un codificador ligero transforma la nube de puntos en una representación latente de dimensión reducida. Esta representación se combina con el estado del robot y condiciona una política de difusión encargada de generar secuencias continuas de acciones.

La arquitectura se resume en la Figura 2.19. La profundidad no se procesa simplemente como un canal adicional de una imagen, sino que se convierte en una estructura tridimensional sobre la que se obtiene una descripción compacta de la escena. Esta descripción se utiliza posteriormente para guiar la generación de movimiento.

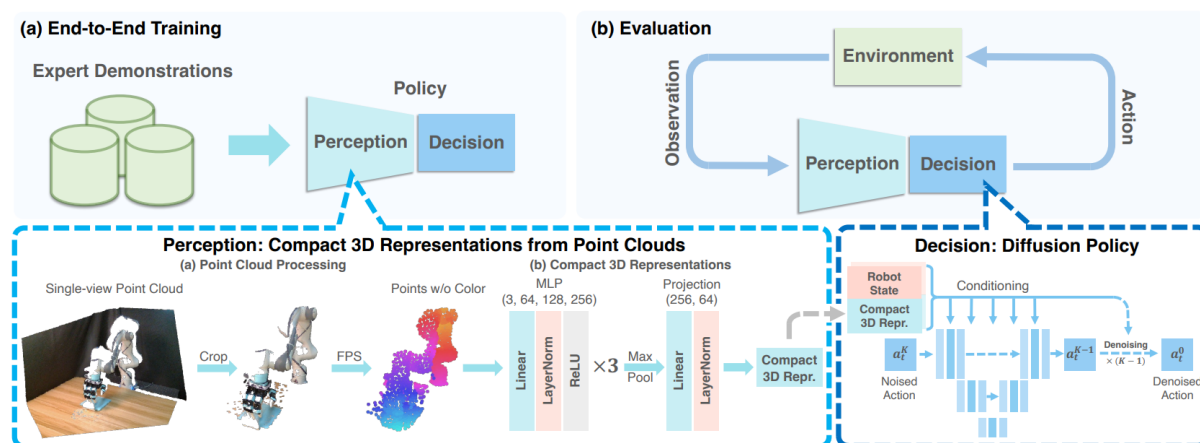


Figura 2.19: Arquitectura de 3D Diffusion Policy. La información de profundidad se transforma en una nube de puntos, se procesa mediante un codificador ligero y condiciona una política de difusión encargada de generar secuencias continuas de acciones [13].

DP3 permite comprobar que las ventajas de la estructura tridimensional no dependen de una única familia arquitectónica. En PerAct, la representación 3D condiciona un

2.6. Representación geométrica tridimensional

Transformer que selecciona acciones discretizadas. En DP3, la misma idea se aplica a una política generativa que modela directamente secuencias de control continuas.

Los resultados presentados en el trabajo indican mejoras en precisión, eficiencia de datos y generalización respecto a políticas visuales bidimensionales [13]. Las evaluaciones incluyen cambios simultáneos en la posición de los objetos, el punto de vista, la apariencia y las instancias presentes en la escena.

Los estudios de ablación también analizan diferentes formas de incorporar información espacial. En la configuración experimental de DP3, la representación mediante nubes de puntos obtiene mejores resultados que alternativas basadas directamente en profundidad, imágenes RGB-D o vóxeles [13]. Asimismo, un codificador compacto puede proporcionar resultados superiores a representaciones tridimensionales más pesadas.

Estas observaciones indican que la incorporación de una modalidad adicional no garantiza por sí sola una mejora. El efecto depende de la forma en que se representa la geometría y de cómo se conecta dicha representación con la política de acciones. DP3 utiliza una estructura suficientemente informativa para describir el espacio, pero evita que su procesamiento domine el coste total de la arquitectura.

Sin embargo, tanto PerAct como DP3 se entrenan como políticas específicas y no aprovechan directamente el conocimiento contenido en grandes modelos visión-lenguaje preentrenados. Esta diferencia resulta relevante porque una parte importante de la capacidad de los VLA procede precisamente de las representaciones semánticas adquiridas con grandes volúmenes de imágenes y texto. El siguiente avance consiste, por tanto, en incorporar geometría tridimensional dentro de un modelo ya preentrenado, evitando tanto el coste de repetir todo el proceso como la pérdida del conocimiento adquirido previamente.

PointVLA aborda esta cuestión mediante la integración de nubes de puntos en modelos VLA que ya han sido preentrenados principalmente con observaciones bidimensionales [14]. La propuesta parte de una diferencia importante entre ambas fuentes de datos: los conjuntos de imágenes y trayectorias RGB disponibles para el preentrenamiento son considerablemente más amplios que los conjuntos robóticos que incluyen información tridimensional.

Reentrenar completamente un modelo fundacional utilizando una nueva modalidad supondría repetir una parte sustancial del coste computacional y podría alterar las representaciones aprendidas anteriormente. Por otro lado, sustituir los datos bidimensionales por conjuntos tridimensionales de menor escala implicaría renunciar a información visual y semántica adquirida durante el preentrenamiento.

PointVLA adopta una estrategia modular. El backbone visión-lenguaje y la mayor parte del experto de acción conservan las representaciones ya aprendidas, mientras que un nuevo módulo procesa la nube de puntos y añade sus características a determinados bloques del experto. El objetivo no consiste en reemplazar la percepción RGB, sino en complementarla con información geométrica explícita.

La Figura 2.20 muestra la arquitectura general. Las imágenes y la instrucción continúan procesándose mediante el VLM original. Paralelamente, la nube de puntos se transforma en una representación compatible con el espacio interno del experto de acción. Esta representación se incorpora mediante módulos adicionales en una parte

seleccionada de la red.

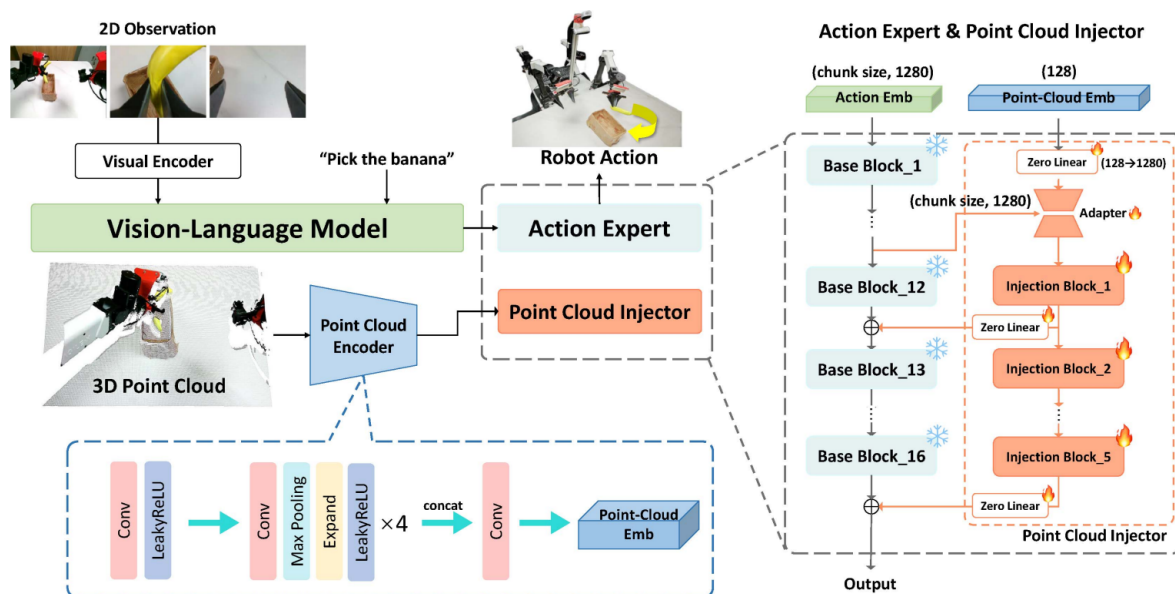


Figura 2.20: Arquitectura general de PointVLA. El modelo conserva el procesamiento visión-lenguaje del VLA preentrenado e incorpora una representación de la nube de puntos mediante un módulo tridimensional conectado con determinados bloques del experto de acción [14].

La selección de estos bloques constituye una parte central de la propuesta. Inyectar la información tridimensional en todas las capas aumentaría el coste computacional y modificaría de forma generalizada las representaciones internas del experto. Esta estrategia podría provocar interferencias con las capacidades aprendidas previamente a partir de datos bidimensionales.

Para reducir este riesgo, PointVLA utiliza un análisis de omisión de bloques (skip-block analysis). El procedimiento evalúa la importancia de los distintos bloques del experto de acción omitiéndolos durante la inferencia y observando el efecto sobre el rendimiento. Los bloques cuya ausencia produce una degradación menor se consideran candidatos más adecuados para incorporar la nueva modalidad.

Como se representa en la Figura 2.21, el análisis permite diferenciar las partes más críticas del experto de aquellas que presentan una mayor capacidad de adaptación. La inyección se concentra en estas últimas, mientras que el resto del experto permanece preservado.

Una vez seleccionados los bloques, las características de la nube de puntos se adaptan a su dimensión interna y se incorporan mediante una operación aditiva. Esta formulación mantiene separada la extracción de características tridimensionales del codificador visual bidimensional y limita el número de componentes que deben entrenarse con los nuevos datos.

La propuesta convierte la incorporación del 3D en un problema de compatibilidad entre representaciones. No resulta suficiente introducir información de profundidad en cualquier punto de la arquitectura. La nueva modalidad debe añadirse de manera que contribuya a la generación de acciones sin degradar el conocimiento visual y semántico adquirido anteriormente.

2.6. Representación geométrica tridimensional

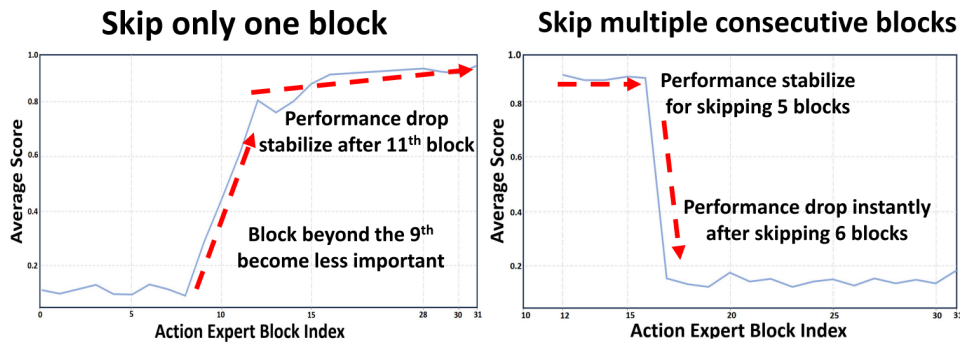


Figura 2.21: Análisis de bloques utilizado en PointVLA. La importancia de las distintas capas del experto de acción se evalúa mediante su omisión durante la inferencia. Los bloques menos críticos se seleccionan para recibir la información tridimensional, reduciendo la interferencia sobre las representaciones preentrenadas [14].

Los experimentos descritos en el trabajo comparan PointVLA con políticas bidimensionales y tridimensionales en escenarios simulados y reales [14]. Los resultados muestran mejoras frente a diferentes métodos de aprendizaje por imitación, entre ellos OpenVLA, Diffusion Policy y DexVLA, en las configuraciones evaluadas.

Más allá del rendimiento agregado, las evaluaciones estudian situaciones donde la geometría explícita proporciona una ventaja diferenciada. Una de ellas consiste en distinguir un objeto físico de una fotografía del mismo. Desde una vista RGB concreta, ambas observaciones pueden presentar una apariencia similar, pero la nube de puntos permite identificar que la fotografía no ocupa el volumen esperado en el espacio.

También se analiza la adaptación a cambios en la altura de la superficie de trabajo. Los modelos basados únicamente en observaciones bidimensionales pueden asociar la apariencia aprendida con una configuración espacial concreta y mantener trayectorias inadecuadas cuando cambia la altura. PointVLA utiliza la geometría observada para ajustar la acción a la nueva disposición.

El trabajo incluye además escenarios de aprendizaje con pocos ejemplos y tareas de mayor duración [14]. En conjunto, estas evaluaciones sugieren que algunas limitaciones de los VLA bidimensionales no se resuelven únicamente incrementando el volumen de datos, ya que proceden de la ausencia de una descripción geométrica suficientemente explícita.

La progresión desde PerAct hasta PointVLA permite identificar tres etapas. PerAct muestra que una representación tridimensional estructurada puede favorecer el aprendizaje de acciones 6-DoF. DP3 demuestra que las nubes de puntos también pueden condicionar políticas generativas continuas con eficiencia. PointVLA traslada finalmente esta información a un VLA preentrenado mediante una arquitectura modular que busca preservar sus capacidades previas.

Esta última etapa resulta especialmente relevante para el presente trabajo. En lugar de desarrollar una política tridimensional desde una inicialización independiente, PointVLA plantea que la geometría puede incorporarse como una fuente complementaria dentro de una base VLA existente. Esta estrategia permite aprovechar simultáneamente el conocimiento semántico aprendido mediante visión y lenguaje y la

información espacial necesaria para interactuar con mayor precisión en el entorno físico.

2.7. Síntesis y posicionamiento del trabajo

La evolución revisada a lo largo de este capítulo muestra la convergencia de distintas líneas de investigación. El aprendizaje por imitación, las demostraciones humanas, las arquitecturas Transformer, la generación de secuencias de acciones y las representaciones espaciales proporcionan las bases sobre las que los modelos VLA integran visión, lenguaje y control robótico. A partir de esta formulación inicial, el paradigma evoluciona hacia políticas más generales, abiertas y eficientes, capaces de utilizar diferentes mecanismos de generación de acciones y nuevas fuentes de información perceptiva.

El Cuadro 2.1 resume los principales trabajos analizados, atendiendo a sus entradas, al procedimiento empleado para generar las acciones y a su contribución principal. La comparación no pretende establecer una clasificación de rendimiento, ya que cada propuesta se evalúa sobre plataformas, tareas y conjuntos de datos diferentes.

La comparación permite identificar dos tendencias principales. La primera corresponde a la transición desde acciones discretizadas y generadas como tokens hacia bloques continuos obtenidos mediante difusión o flow matching. La segunda consiste en ampliar la información disponible para la política mediante señales táctiles, memoria temporal y representaciones tridimensionales. Estas líneas muestran que la evolución de los VLA no depende únicamente del tamaño del modelo, sino también de cómo se representan el movimiento, el contexto temporal y la geometría del entorno.

El presente trabajo se sitúa en la intersección entre la memoria y la percepción tridimensional. Sobre la arquitectura de SmolVLA se incorporan memoria visual de corto plazo e información de profundidad, tanto de forma independiente como conjunta. Esta estrategia mantiene las capacidades visuales y lingüísticas del modelo base e introduce componentes especializados para representar mejor las oclusiones y la estructura espacial de la escena, siguiendo la orientación modular observada en trabajos como MEM [12] y PointVLA [14].

Cuadro 2.1: Síntesis de los principales trabajos revisados en el estado del arte.

Modelo	Año	Entrada principal	Generación de acciones	Contribución principal
PerAct [1]	2022	Lenguaje y observaciones RGB-D voxelizadas.	Selección discretizada de acciones 6-DoF mediante next best voxel action.	Introduce una representación tridimensional estructurada y condicionada por lenguaje para manipulación con Transformers.
ALOHA/ACT [2]	2023	Imágenes, estado propioceptivo y demostraciones humanas bimanuales.	Bloques de acciones generados mediante un Transformer condicionado y combinados con ensamblado temporal.	Propone una plataforma asequible de teleoperación y desarrolla la agrupación de acciones para reducir la acumulación de errores.
RT-2 [3]	2023	Imagen e instrucción lingüística.	Generación autorregresiva de acciones discretizadas como tokens.	Formaliza el paradigma VLA y transfiere conocimiento adquirido mediante preentrenamiento visión-lenguaje al control robótico.
DP3 [13]	2024	Nube de puntos y estado propioceptivo.	Secuencias continuas de acciones mediante una política de difusión.	Demuestra la eficiencia de una representación tridimensional compacta para políticas generativas de manipulación.
Octo [4]	2024	Imágenes, propiocepción e instrucciones lingüísticas o imágenes objetivo.	Bloques de acciones continuas mediante una cabeza de difusión.	Desarrolla una política generalista abierta y adaptable a nuevas observaciones, espacios de acción y plataformas.
OpenVLA [6]	2024	Imagen, instrucción lingüística y estado del robot.	Generación autorregresiva de tokens de acción discretizados.	Proporciona un VLA abierto, reproducible y compatible con procedimientos de ajuste eficiente y cuantización.
TinyVLA [9]	2024	Imágenes, lenguaje y estado del robot.	Secuencias de acciones continuas mediante una política de difusión.	Reduce el tamaño y la latencia de los VLA y mejora su eficiencia de datos sin depender de preentrenamiento robótico a gran escala.
Π_0 [5]	2024	Imágenes, instrucción lingüística y propiocepción.	Bloques continuos generados por un experto de acción entrenado con flow matching.	Separa el procesamiento visión-lenguaje de la generación continua de acciones mediante un módulo especializado.
PointVLA [14]	2025	Imagen, lenguaje, estado del robot y nube de puntos.	Acciones continuas mediante un experto que recibe características tridimensionales inyectadas.	Integra geometría 3D en un VLA preentrenado mediante módulos ligeros y un análisis de omisión de bloques.
GR00T N1 [7]	2025	Imágenes, lenguaje, estado robótico y datos heterogéneos de distintas fuentes.	Acciones continuas mediante un Diffusion Transformer con flow matching.	Extiende los modelos VLA hacia múltiples morfologías, especialmente plataformas humanoides.
$\Pi_{0.5}$ [8]	2025	Imágenes, lenguaje, subtareas semánticas y datos robóticos y web heterogéneos.	Preentrenamiento con acciones discretizadas y postentrenamiento de un experto continuo con flow matching.	Mejora la generalización en entornos domésticos no observados mediante coentrenamiento e inferencia jerárquica.
VTLA [11]	2025	Visión, secuencias táctiles e instrucción lingüística.	Acciones tokenizadas refinadas posteriormente mediante DPO.	Incorpora percepción táctil a un VLA para resolver tareas de inserción con contacto intensivo.
SmoVLA [10]	2025	Múltiples imágenes RGB, instrucción lingüística y estado propioceptivo.	Bloques continuos mediante un experto de acción entrenado con flow matching.	Combina una arquitectura compacta, datos comunitarios, plataformas asequibles e inferencia asíncrona.
MEM [12]	2026	Observación actual, memoria visual reciente, memoria textual e instrucción.	Acciones continuas condicionadas por memoria multiescala y subtareas semánticas.	Introduce memoria visual a corto plazo y memoria lingüística a largo plazo para tareas prolongadas y parcialmente observables.

Capítulo 3

Arquitectura de Referencia y Fundamentos del Sistema

En este capítulo se presentan los elementos teóricos necesarios para comprender el funcionamiento del modelo SmolVLA [10] y justificar las propuestas desarrolladas en este trabajo. En primer lugar, se aborda el aprendizaje por imitación, principio sobre el que se apoya la recopilación de demostraciones robóticas. A continuación, se describe la técnica de flow matching empleada para entrenar el experto de acción. Posteriormente, se detalla la arquitectura general de SmolVLA, analizando la integración entre el módulo de visión y lenguaje y la generación de acciones continuas. Por último, se presentan los principios de la percepción de profundidad y de la memoria visual a corto plazo, elementos clave para la formulación de las variantes descritas en el Capítulo 4.

3.1. Aprendizaje por imitación

El aprendizaje por imitación permite entrenar una política de control a partir de demostraciones proporcionados por un operador humano, o incluso por una política ya entrenada. Estas demostraciones suelen obtenerse mediante teleoperación. El operador controla el robot mientras se registran de forma sincronizada las imágenes de las cámaras, el estado interno del sistema y las acciones ejecutadas. También se asocia a cada demostración una instrucción que describe la tarea realizada.

Cada demostración constituye un episodio formado por una secuencia temporal de observaciones y acciones. Las imágenes describen el entorno y la posición de los objetos, mientras que el estado propioceptivo recoge información interna del robot, como la posición de sus articulaciones o la apertura de la pinza. Estos datos constituyen el conjunto de entrenamiento de la política, con las acciones como función objetivo y los demás datos como contexto para condicionar esta función.

Durante el despliegue, el robot funciona en bucle cerrado. La política recibe una observación, genera un bloque de acciones, ejecuta una parte y vuelve a observar la escena antes de continuar. Esta actualización permite adaptar el movimiento a los cambios producidos durante la ejecución.

Sin embargo, aquí surge una diferencia importante entre la fase de entrenamiento y la de inferencia. Durante la recogida de datos, la política solo observa movimientos correctos realizados por el operador. En cambio, cuando el robot funciona por su cuenta, un pequeño error al ejecutar una acción cambia ligeramente la escena. Si el modelo no ha visto esa situación antes, este pequeño fallo se va acumulando y puede hacer que el robot se desvíe por completo de la tarea.

Por este motivo, la calidad y sobre todo la variedad de las demostraciones son fundamentales. No basta con grabar la misma trayectoria repetida; conviene incluir diferentes posiciones de los objetos y ejemplos donde el propio operador corrija un agarre fallido o una desviación. Así, el robot aprende a reaccionar y recuperarse cuando la ejecución no sale perfectamente como en la teoría [2], [10].

3.2. Flow Matching

El flow matching es una técnica generativa que permite transformar de manera progresiva una distribución de datos simple (como el ruido gaussiano) en una distribución objetivo que se desea replicar [15].

Para entender la idea de forma intuitiva, el flow matching funciona definiendo un campo vectorial que cambia a lo largo del tiempo. En la Figura 3.1 se puede observar un esquema simplificado de este proceso: las muestras iniciales (los puntos azules que representan el ruido) se van desplazando y adoptan la forma de la distribución objetivo (la figura roja) a medida que avanza el tiempo, guiadas por los vectores en cada punto. En cada etapa intermedia, el modelo calcula hacia dónde debe mover las muestras para acercarlas a la forma final deseada.

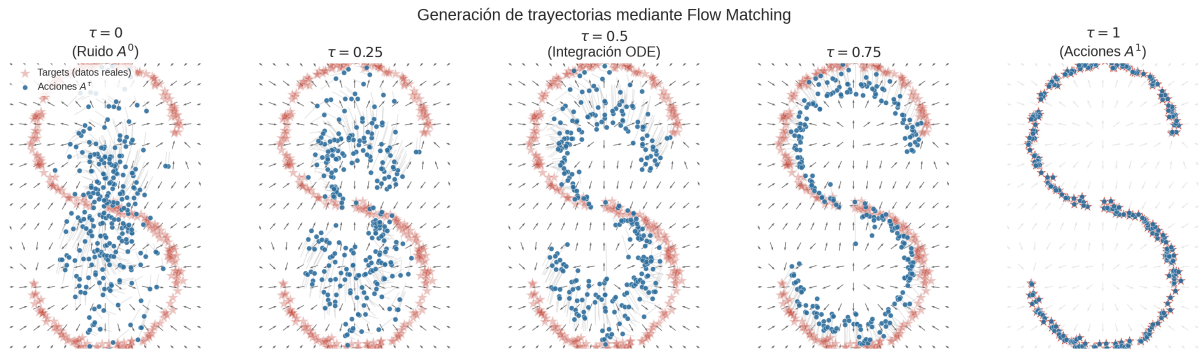


Figura 3.1: Evolución conceptual del proceso de flow matching. Elaboración propia a partir de [15].

Sin embargo, en el ámbito de los VLA se utiliza una adaptación de este concepto. En un robot no se busca generar siempre la misma distribución objetivo, sino una respuesta concreta según lo que esté ocurriendo en cada momento. Por ello, el experto de acción aplica un flow matching condicionado, que se representa como:

$$v_{\psi} = f_{\psi}(x_{\tau}, c, \tau), \quad (3.1)$$

donde x_{τ} es el estado intermedio del bloque de acciones, τ indica la etapa o avance del proceso de transformación (el «tiempo» en la explicación inicial), c es el vector de

contexto que contiene la información visual y la instrucción lingüística, y v_ψ es el campo vectorial predicho por el modelo f_ψ con el que obtendríamos el siguiente x_τ .

En los sistemas de control robótico, la distribución objetivo está formada por las secuencias de acciones de las demostraciones reales (cuya recolección se explica mejor en la Sección 3.1). Durante el entrenamiento, el modelo recibe el contexto y combina las acciones reales con distintos niveles de ruido. De este modo, aprende a predecir las correcciones necesarias para transformar ese ruido en movimientos coherentes con la tarea.

SmolVLA aplica este principio directamente en su experto de acción [10]. El modelo visión-lenguaje procesa las imágenes y la instrucción para construir el contexto c , mientras que el experto recibe esta información junto con el estado propioceptivo del robot. Durante la inferencia, el proceso comienza con un bloque de acciones completamente aleatorio que el experto de acción va corrigiendo de forma iterativa siguiendo los vectores predichos. Al completar estos pasos de refinamiento, el ruido inicial se transforma en una secuencia de movimientos suave y continua, lista para ser ejecutada por el robot.

3.3. Arquitectura de SmolVLA

SmolVLA es una arquitectura VLA que combina un modelo compacto de visión y lenguaje con un experto especializado en la generación de acciones continuas [10], tal y como se ilustra en la Figura 3.2. Para ello, un codificador visual basado en SigLIP procesa las imágenes de las cámaras, mientras que el modelo de lenguaje SmolLM2 recibe y sintetiza estas características visuales junto con la instrucción textual y el estado propioceptivo del robot. En cada paso de inferencia, este bloque construye un contexto multimodal unificado en una sola pasada al inicio del ciclo, lo que permite optimizar el tiempo de cómputo y desacoplar la percepción densa de la escena de la generación de acciones.

Por su parte, el experto de acción se entrena mediante flow matching y opera de manera paralela sobre las representaciones obtenidas. En lugar de predecir una única acción, genera una secuencia continua de acciones futuras aplicando action chunking (concepto introducido en la Figura 2.3 del Capítulo 2). Su entrada inicial es una muestra de ruido gaussiano que se refina de forma progresiva a lo largo de diez iteraciones. Para guiar esta transformación, el experto de acción intercala capas de autoatención y atención cruzada. Ambas etapas emplean el mismo operador de atención, diferenciándose únicamente en el origen de las variables de entrada.

En la autoatención, las consultas, claves y valores se extraen de la propia representación latente del bloque de acciones, denotada como H_A , permitiendo relacionar los distintos elementos de la trayectoria entre sí:

$$H_{\text{self}} = \text{Attention}(Q = H_A, K = H_A, V = H_A). \quad (3.2)$$

En la atención cruzada, se utiliza la misma función pero variando la fuente de las claves y los valores. Las consultas (Q) se derivan de H_A , mientras que las claves (K) y valores (V) proceden del contexto multimodal generado por el VLM, denotado como H_{VLM} :

3.4. Representación tridimensional e inyección de características

$$H_{\text{cross}} = \text{Attention}(Q = H_A, K = H_{\text{VLM}}, V = H_{\text{VLM}}). \quad (3.3)$$

De este modo, la autoatención coordina internamente el flujo del bloque de acciones, mientras que la atención cruzada actúa como el mecanismo de condicionamiento del proceso de flow matching, permitiendo a dichas acciones extraer la información visual, semántica y propioceptiva necesaria para guiar la generación de la acción.

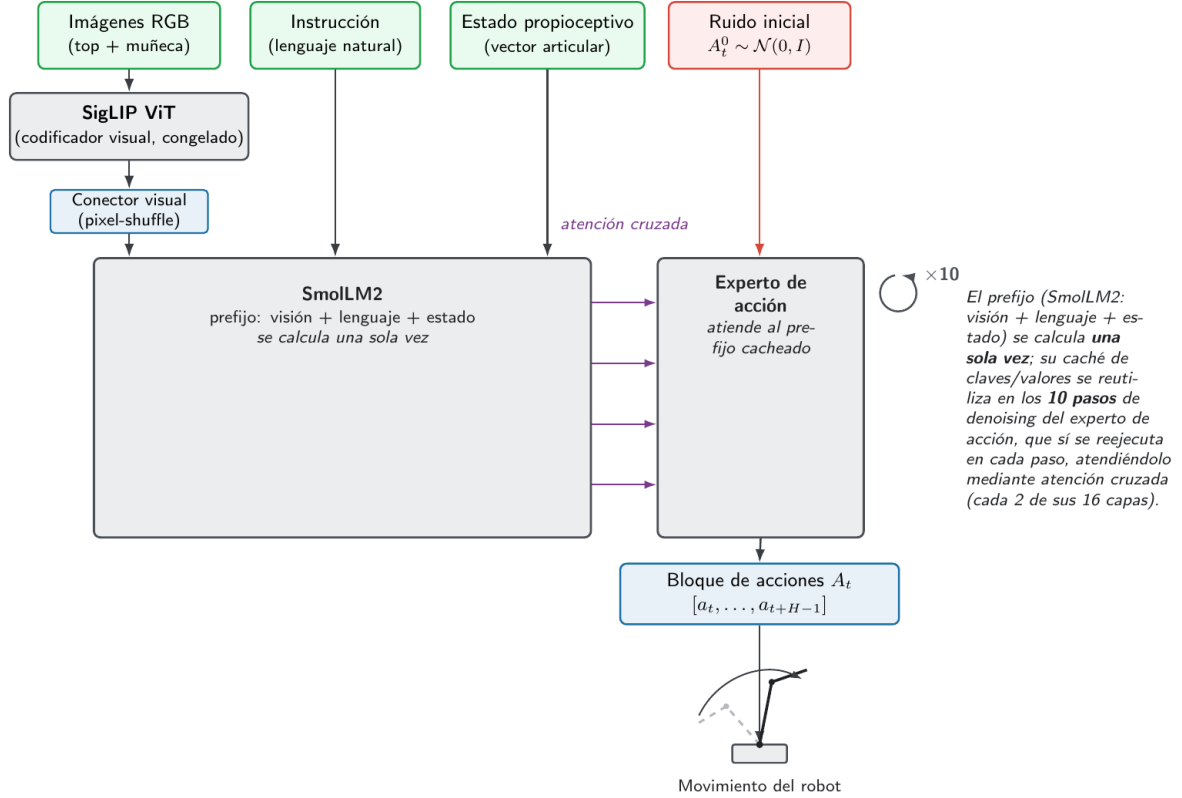


Figura 3.2: Arquitectura simplificada de SmolVLA. El modelo de visión y lenguaje procesa las imágenes RGB, la instrucción y la propiocepción para generar un contexto multimodal que condiciona de forma constante al experto de acción en la generación de acciones. Elaboración propia a partir de [10].

3.4. Representación tridimensional e inyección de características

Para estimar la profundidad de la escena en este trabajo se utiliza una cámara estereó, la cual captura la escena desde dos puntos de vista ligeramente separados. Tras alinear ambas imágenes, se busca la posición de un mismo punto en la vista izquierda y en la derecha. La diferencia horizontal entre ambas posiciones se denomina disparidad.

La idea es similar a observar un objeto alternando entre un ojo y otro: los elementos cercanos parecen desplazarse más que los lejanos. Por tanto, una disparidad grande

indica que el punto está próximo a la cámara, mientras que una disparidad pequeña indica que se encuentra más lejos.

Conociendo esta disparidad y los parámetros de calibración de la cámara, se calcula la distancia de cada punto. El resultado se transforma en una nube de puntos tridimensional, donde cada punto representa una posición de la escena. Esta nube se filtra para eliminar valores incorrectos y se reduce antes de ser procesada por un codificador.

El codificador convierte la nube en características compactas que representan la geometría del entorno. Estas características complementan a las obtenidas mediante las imágenes RGB: mientras la imagen aporta información visual y semántica, la nube de puntos describe con precisión la profundidad y la disposición espacial de los objetos, evitando errores de trayectoria durante la manipulación [14].

Para incorporar esta información al modelo, las características tridimensionales se adaptan a la dimensión interna del experto de acción y se añaden a sus representaciones mediante una suma residual:

$$\tilde{h}^{(l)} = h^{(l)} + \alpha_l r_{3D}^{(l)}, \quad (3.4)$$

donde $h^{(l)}$ representa las características originales del bloque, $r_{3D}^{(l)}$ contiene la información geométrica adaptada y α_l controla su contribución.

De este modo, la información tridimensional se introduce como una señal adicional sin sustituir el procesamiento visual y lingüístico original. Este principio, inspirado en PointVLA, permite añadir profundidad a SmoIVLA y estudiar si una representación explícita de la geometría mejora la generación de acciones [14].

3.5. Memoria visual de corto plazo

Una política de control exclusivamente reactiva genera sus acciones basándose únicamente en la observación del instante actual. Sin embargo, este planteamiento resulta insuficiente cuando el entorno es parcialmente observable, como ocurre cuando un objeto queda oculto de forma temporal por otro elemento de la escena o por la propia pinza del robot. En estas situaciones, una sola imagen no contiene información suficiente para determinar el estado de la tarea, ya que dos observaciones visualmente idénticas pueden corresponder a momentos diferentes de la ejecución y requerir acciones completamente distintas.

Para resolver este problema, la memoria visual de corto plazo amplía la entrada del modelo guardando una secuencia con las observaciones más recientes. Matemáticamente, esta ventana de memoria $O_t^{K,\Delta}$ en el instante actual t se define como:

$$O_t^{K,\Delta} = [o_{t-(K-1)\Delta}, \dots, o_{t-\Delta}, o_t], \quad (3.5)$$

donde o_t es la imagen del instante actual, K es el número total de fotogramas que guardamos en la memoria y Δ es el salto o separación temporal entre ellos (por ejemplo, tomar una imagen cada cierto número de pasos o cada ciertos milisegundos).

Al mantener esta pequeña secuencia de imágenes pasadas, el robot puede recordar dónde estaba un objeto que acaba de tapar con su propia mano, comprobar si un agarrar ha tenido éxito o interpretar hacia dónde se estaba moviendo un elemento. Para que el modelo aproveche estas observaciones, las representaciones de los distintos fotogramas se conectan mediante un mecanismo de atención temporal, permitiendo a la red aprender a prestar atención únicamente a los momentos pasados que son realmente relevantes para decidir el movimiento actual.

Por tanto, la función de esta memoria de corto plazo no consiste en almacenar la historia completa de una tarea prolongada, sino en conservar detalles visuales y de movimiento durante un intervalo reducido de pocos segundos. Esta capacidad resulta especialmente útil para superar pérdidas temporales de visibilidad y problemas de autooclusión [12].

De este modo, la dimensión temporal se incorpora de manera modular sobre la representación del entorno, respetando el procesamiento original de visión y lenguaje. Esta integración permite extender el modelo base de SmolVLA para estudiar si la retención de características recientes favorece el aprendizaje de políticas de control más robustas [12].

Capítulo 4

Metodología

En este capítulo se detalla la metodología desarrollada en este trabajo. En primer lugar, se describe el entorno experimental, tanto físico como digital. A continuación, se explica el proceso de recolección de datos, se presentan las cuatro variantes de SmolVLA desarrolladas y cómo se entrenaron. Finalmente, se describe tanto el test de ablación sobre las capas del experto de acción como la evaluación final. Los resultados de estas evaluaciones se presentan y analizan en el Capítulo 5.

4.1. Entorno experimental

Para este trabajo se ha utilizado una misma plataforma física, tanto para la recolección de datos como para las evaluaciones finales. Esta plataforma hace uso del brazo robótico SO-101¹, desarrollado por The Robot Studio en colaboración con Hugging Face. Se trata de un robot de 6 grados de libertad totalmente integrado en el ecosistema LeRobot, lo que facilita el despliegue de SmolVLA al contar con soporte nativo y un modelo base ya preentrenado sobre este hardware. En la Figura 4.1 se observa cómo se ven este robot junto a su brazo líder de teleoperación.

En cuanto a los sensores, se dispone de tres cámaras que se pueden ver en la Figura 4.2. La configuración consta de dos cámaras activas por experimento: una cámara superior fija que observa toda la escena, y una cámara montada en la muñeca del robot. En la muñeca se alterna entre una cámara RGB monocular, utilizada para el test de ablación explicado en la Subsección 4.4.1, y una cámara estéreo para la evaluación final, descrita en la Subsección 4.4.2.

Para la tarea se plantea un escenario delimitado por las dimensiones de un papel A3. En él se encuentran cuatro figuras que el robot debe manipular, un cubo para depositar las estrellas y una zona marcada en el papel para depositar los cubos. La zona de trabajo queda dividida en nueve posiciones delimitadas, tal y como se muestra en la Figura 4.3, enumerando las columnas del 1 al 3 y las filas con las letras A, B y C. La instrucción que se le pasa al modelo no cambia en ningún momento y se enuncia como:

Put the stars in the bin and put the cubes on the marked area.

¹<https://github.com/TheRobotStudio/SO-ARM100>

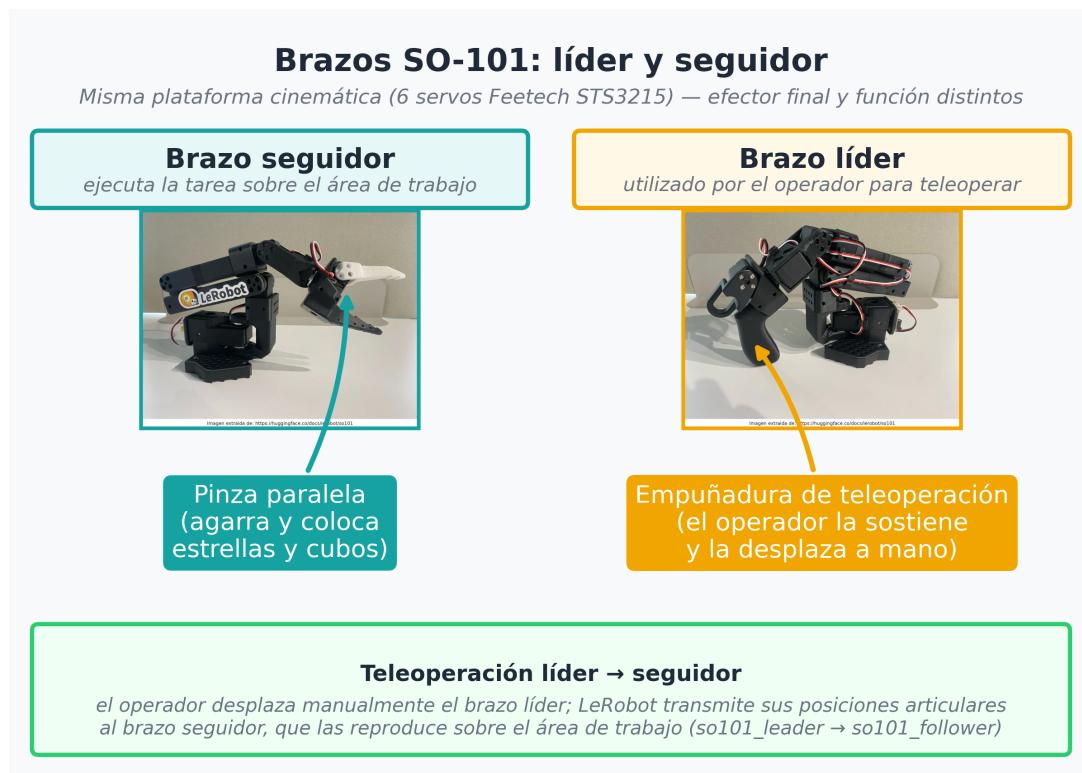


Figura 4.1: Brazos SO-101 para teleoperación: seguidor con pinza (izquierda) y líder (derecha). Imágenes de <https://huggingface.co/docs/lerobot/so101>.



Figura 4.2: Cámaras del sistema: RGB superior fija (izquierda), RGB monocular de muñeca (centro) y estéreo SVPRO con su calibración y captura side-by-side (derecha).

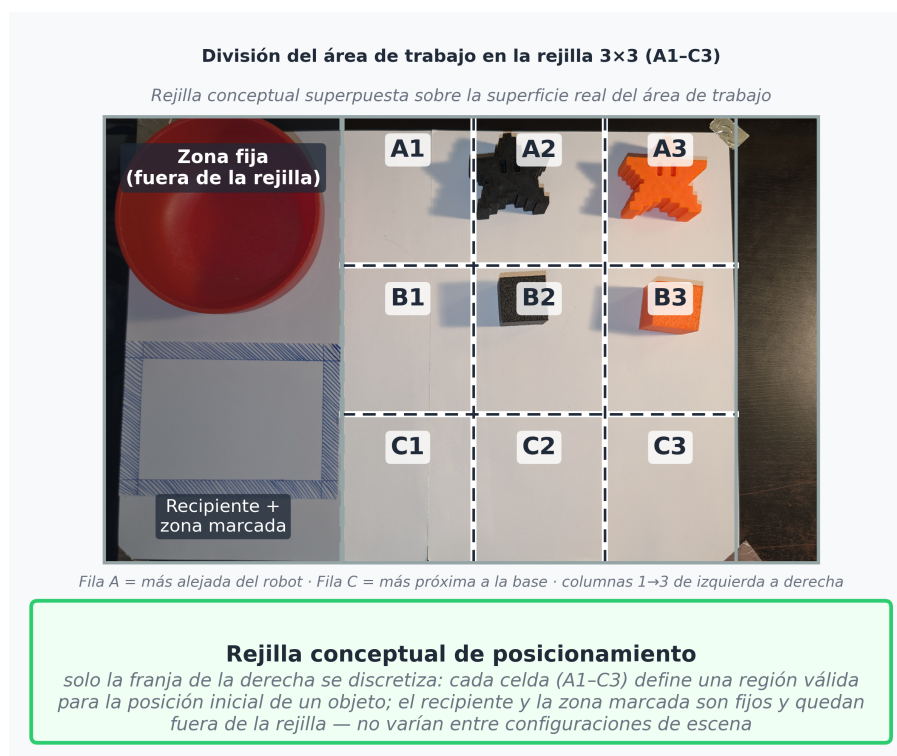


Figura 4.3: Discretización del área de trabajo: rejilla 3×3 (posiciones A1-C3) para origen de objetos y zona fija de destino.

Esta configuración busca, por un lado, estructurar el área de trabajo para estandarizar las posiciones iniciales de los objetos en la escena durante las evaluaciones y, por otro, favorecer una mayor probabilidad de autooclusión al situar el brazo robótico deliberadamente cerca de la zona de manipulación.

Para el registro de demostraciones y evaluaciones se emplea la propia librería LeRobot. El almacenamiento de datasets y checkpoints se gestiona a través de Hugging Face, mientras que la monitorización del entrenamiento, junto con el registro de métricas y pérdidas, se realiza en Weights & Biases.

4.2. Toma y preparación de datos

Para este trabajo se han confeccionado dos conjuntos de datos principales: uno utilizado en el test de ablación y otro en la evaluación final. Ambos cuentan con un total de 120 episodios con configuraciones distintas cada uno, variando la posición, rotación y cercanía de las figuras para aportar suficiente variabilidad al conjunto. Para la recolección, se utilizó la plataforma experimental controlando manualmente los movimientos a través del brazo líder.

La diferencia principal entre ambos datasets reside en la cámara de muñeca empleada y el tiempo total de grabación: el dataset de ablación utiliza la cámara monocular RGB (acumulando unos 103 minutos), mientras que el dataset final utiliza la cámara estéreo monocromo (unos 87 minutos). A partir de este segundo conjunto se generó una variante con profundidad, cuyo preprocesado se muestra en la Figura 4.4.

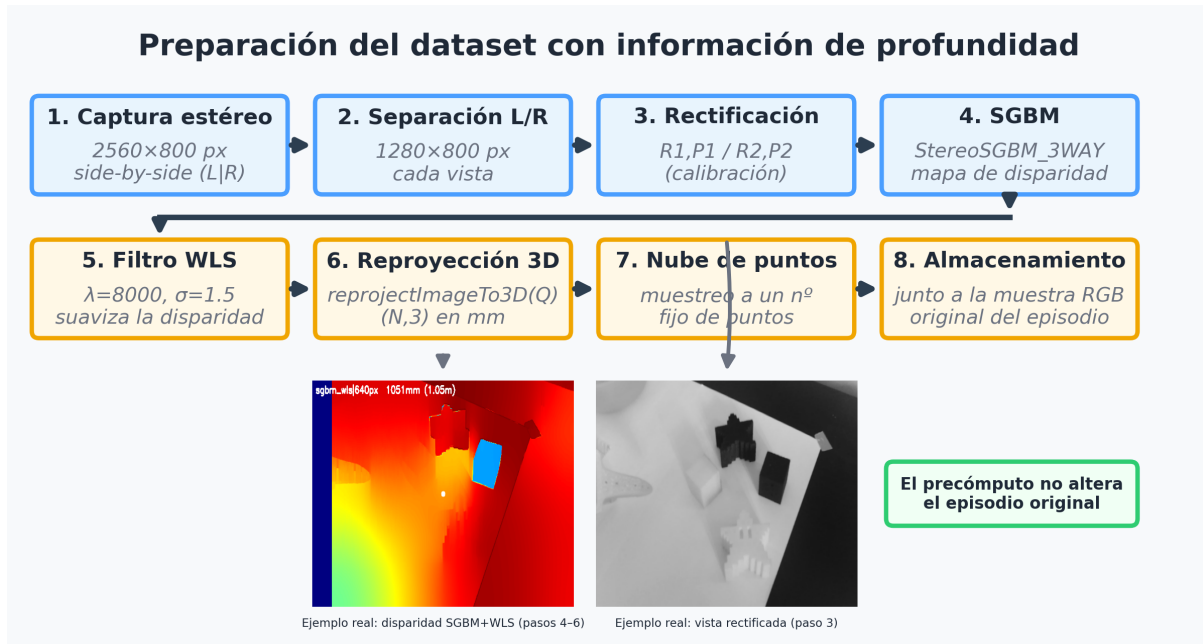


Figura 4.4: Flujo de procesamiento para el dataset con profundidad: rectificación de vistas estéreo, cálculo del mapa de disparidad y reproyección a nube de puntos.

4.3. Variantes de SmolVLA

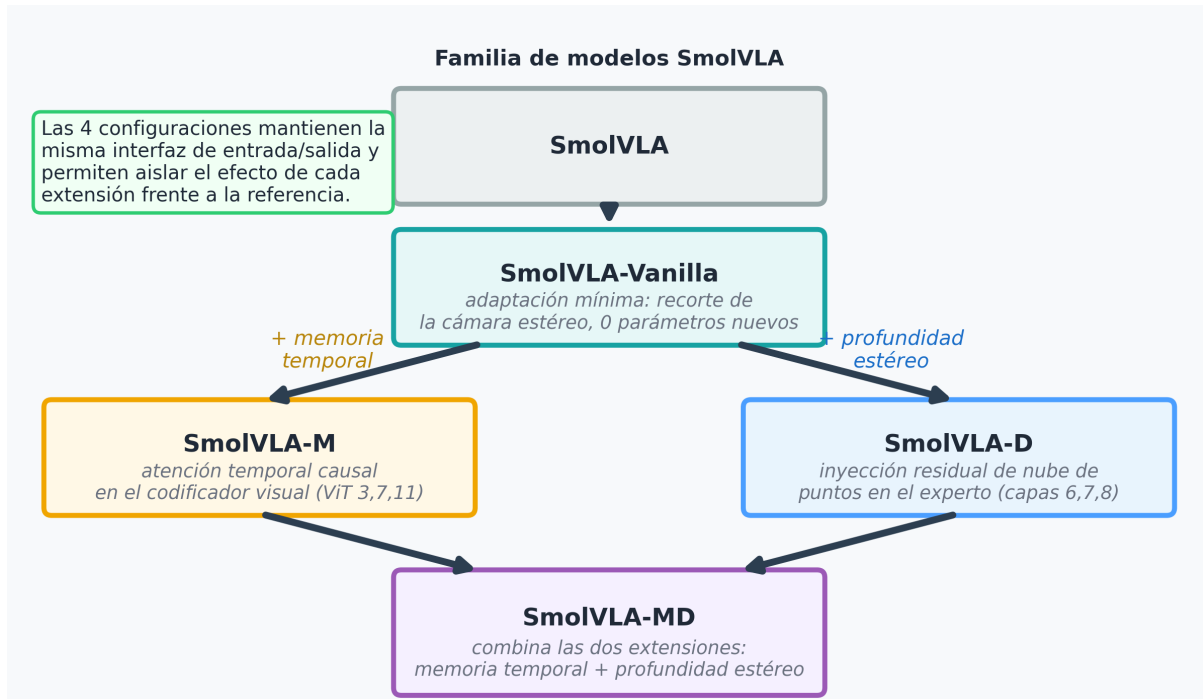


Figura 4.5: Relación y jerarquía de la familia de modelos SmolVLA desarrollada a partir del checkpoint base.

Todas las arquitecturas desarrolladas en este trabajo parten del modelo base SmolVLA disponible en la librería LeRobot. Para el estudio de ablación sobre las capas del

experto de acción (Subsección 4.4.1) se utiliza este modelo original sin modificar la red, evaluando únicamente la eliminación progresiva de sus capas.

Por el contrario, para la evaluación final se diseñan cuatro variaciones que introducen modificaciones estructurales específicas para abordar problemas de memoria y percepción espacial. La Figura 4.5 resume la relación jerárquica entre el modelo base y las cuatro configuraciones desarrolladas (SmolVLA-Vanilla, SmolVLA-M, SmolVLA-D y SmolVLA-MD).

4.3.1. SmolVLA-Vanilla

Esta variante se concibe como la arquitectura de referencia para la evaluación final. Conserva intacta la estructura original de SmolVLA sin añadir parámetros adicionales, garantizando que el rendimiento medido corresponda exclusivamente a la capacidad base del modelo.

La única modificación radica en un módulo de preprocesado diseñado para gestionar la entrada visual de la cámara de muñeca. Puesto que este sensor estéreo entrega una captura continua con las vistas izquierda y derecha concatenadas, el módulo recorta automáticamente la imagen y selecciona únicamente la mitad derecha para alimentar al codificador visual. De este modo, se mantiene el mismo hardware físico en todos los experimentos sin alterar el formato de entrada monocular que requiere la red original.

Como se observa en la Figura 4.6, esta adaptación mínima constituye una modificación común a las cuatro variantes desarrolladas. Para simplificar los diagramas de los modelos posteriores, esta caja de recorte no se muestra de forma explícita en el resto de figuras, aunque se mantiene activa en todos los experimentos para garantizar unas condiciones homogéneas y comparables entre sí.

4.3.2. SmolVLA-M

SmolVLA-M incorpora memoria visual a corto plazo mediante una adaptación del codificador de vídeo propuesto en MEM [12]. Como se detalla en la Sección 3.5, en lugar de procesar únicamente el fotograma actual, el codificador visual recibe una secuencia de $K = 3$ imágenes con un stride temporal s de aproximadamente 2,5 segundos. Esta configuración conforma una ventana temporal total de 5 segundos que preserva la cobertura de MEM, aunque con una resolución temporal menor debido a las restricciones de VRAM durante el entrenamiento.

Las representaciones visuales se combinan mediante atención temporal causal en las capas 3, 7 y 11 del codificador visual. Esta distribución replica el patrón de MEM al intercalar una operación temporal cada cuatro bloques del ViT, permitiendo integrar el historial en distintos niveles de abstracción sin transformar la totalidad de las capas en operaciones espacio-temporales.

La salida de cada bloque modificado se calcula mediante la siguiente conexión residual:

$$\tilde{H}^{(l)} = H_{\text{esp}}^{(l)} + \alpha_l H_{\text{temp}}^{(l)}, \quad l \in \{3, 7, 11\}, \quad (4.1)$$

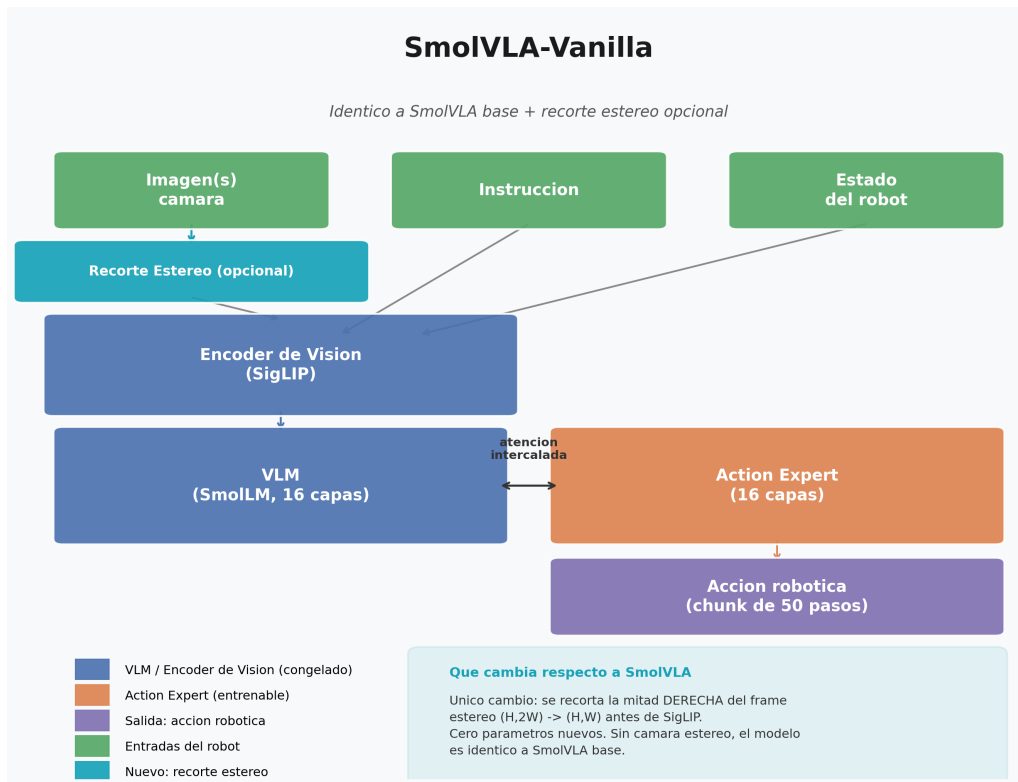


Figura 4.6: Vista general de la arquitectura de SmolVLA-Vanilla.

donde $H_{\text{esp}}^{(l)}$ representa la activación espacial original y $H_{\text{temp}}^{(l)}$ la información extraída de las observaciones pasadas. El esquema conceptual de esta arquitectura se muestra en la Figura 4.7.

4.3.3. SmolVLA-D

Como se detalla en la Sección 3.4, las nubes de puntos 3D se obtienen a partir de la captura de la cámara estéreo. Para inyectar esta información espacial en el experto de acción de SmolVLA-D, seguimos la estrategia de PointVLA [14] dividiendo el proceso en dos módulos.

En primer lugar, un PointCloudEncoder procesa las coordenadas tridimensionales para extraer una representación compacta que resume la geometría de la escena. Después, el DepthAdapter se encarga de proyectar esta representación para ajustar su dimensión a la del experto de acción. Finalmente, la señal resultante se inyecta mediante una suma residual únicamente en los bloques 6, 7 y 8 de la red.

La elección de las capas 6, 7 y 8 se decide tras realizar el estudio de ablación, identificando así qué bloques son más susceptibles de incorporar información tridimensional. La Figura 4.8 muestra la arquitectura general de SmolVLA-D, la rama estéreo y el punto exacto de inyección dentro del experto de acción.

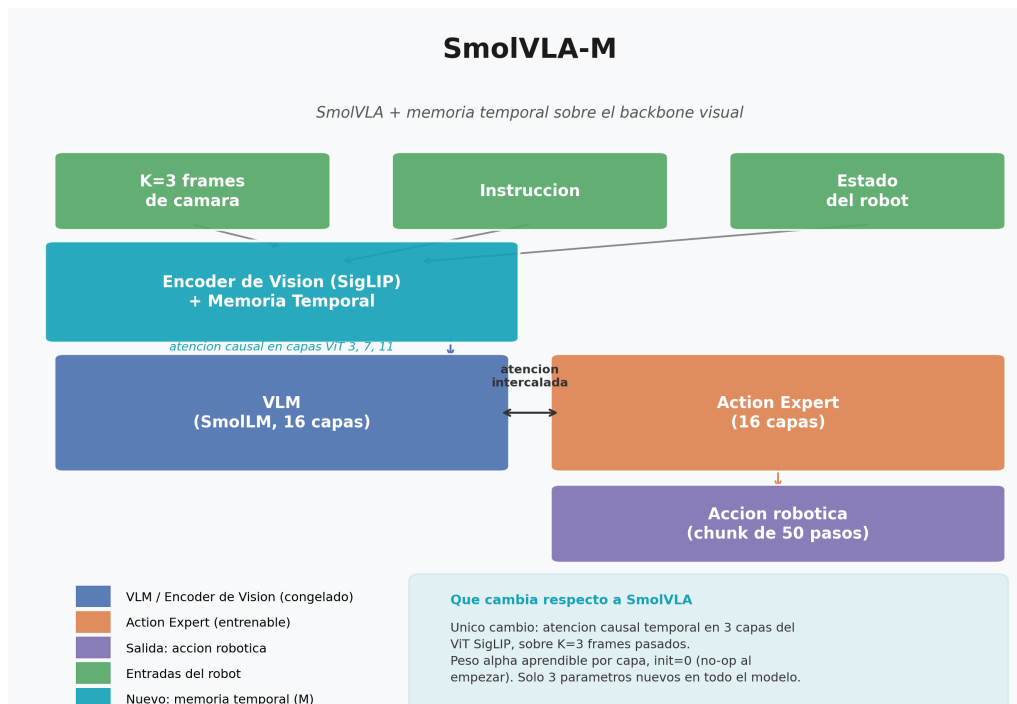


Figura 4.7: Arquitectura de SmoIVLA-M: integración de atención temporal causal en las capas 3, 7 y 11 del codificador visual.

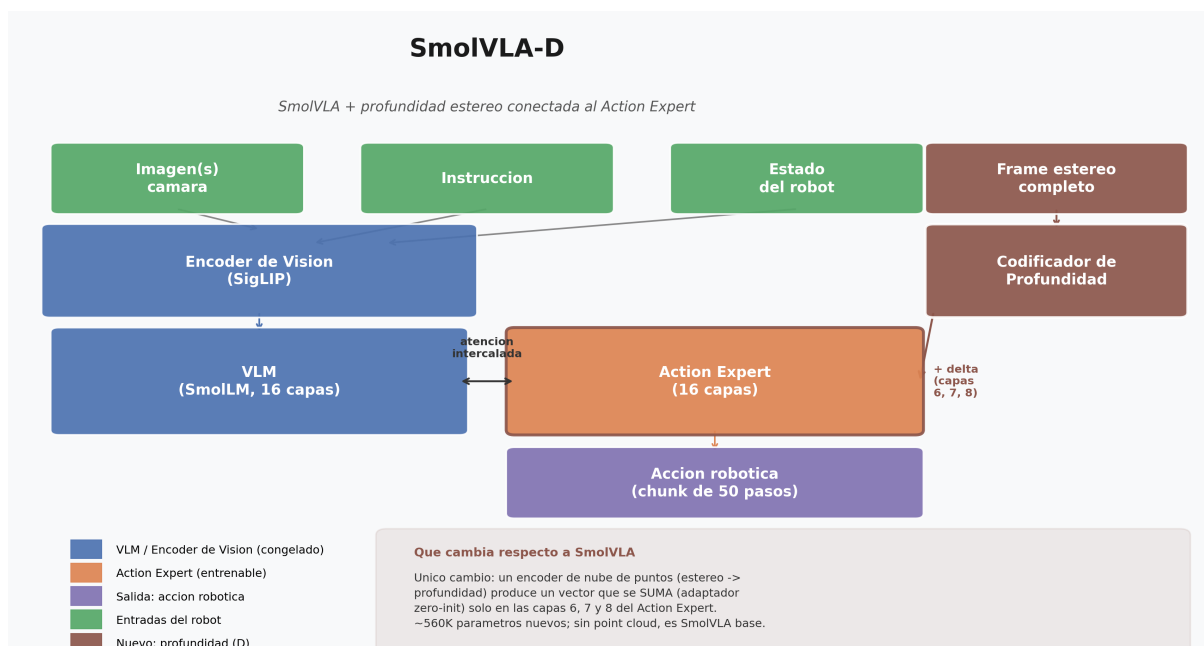


Figura 4.8: Arquitectura de SmoIVLA-D.

4.3.4. SmolVLA-MD

SmolVLA-MD integra de manera simultánea las mejoras desarrolladas en SmolVLA-M y SmolVLA-D. Dado que la memoria temporal actúa en el codificador visual (bloques 3, 7 y 11) y la información tridimensional se inyecta en el experto de acción (bloques 6, 7 y 8), ambas extensiones son completamente independientes y no se solapan.

Con esta variante buscamos comprobar si unir la memoria temporal y la información 3D mejora la destreza del robot frente a usarlas por separado. La Figura 4.9 muestra cómo se integran ambas rutas en el modelo.

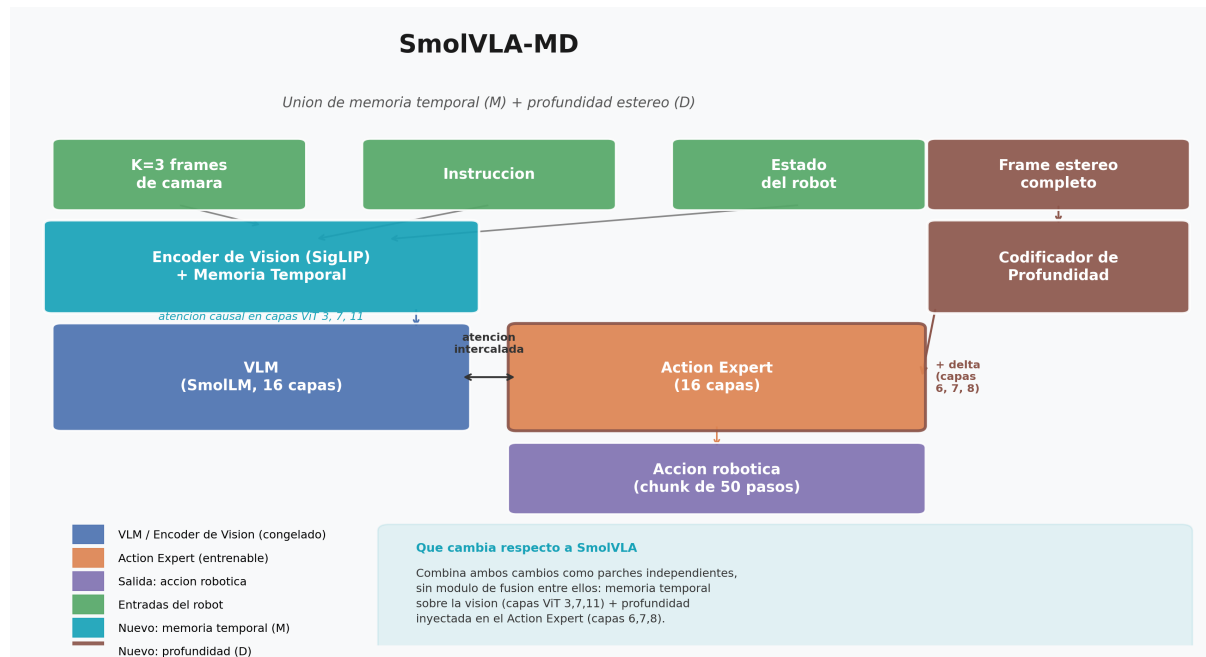


Figura 4.9: Arquitectura de SmolVLA-MD.

4.3.5. Entrenamientos

Los modelos se entrenan a partir de los pesos base de SmolVLA disponibles en LeRobot, inicializando a cero únicamente las nuevas extensiones. Durante el proceso se congelan el ViT y el modelo de lenguaje, de modo que solo se optimizan el experto de acción y los módulos adicionales (el codificador de nubes de puntos y el adaptador 3D en SmolVLA-D, y los tres escalares de atención temporal en SmolVLA-M). Se mantiene la función objetivo original basada en flow matching sin pérdidas auxiliares, por lo que todo el aprendizaje proviene de la predicción de acciones.

Para la optimización se utiliza AdamW con tasa de aprendizaje máxima de 10^{-4} (calentamiento lineal de 1000 pasos y decaimiento coseno hasta $2,5 \times 10^{-6}$), decaimiento de pesos de 10^{-10} y recorte de gradiente en 10. Además, se desactiva la opción de data augmentation sobre las imágenes para no alterar el recorte de la cámara estéreo.

Los cuatro modelos se entrenaron secuencialmente en una GPU NVIDIA H100 (80 GB) fijando un presupuesto objetivo de 1 440 000 muestras y la semilla 42. Para evitar la dependencia exclusiva de la geometría en las variantes con profundidad (D y

Metodología

MD), se aplica un descarte aleatorio del 20 % de los datos de profundidad. El Cuadro 4.1 resume la configuración y estado final de las ejecuciones.

Cuadro 4.1: Resumen de la configuración y ejecución de los entrenamientos de los modelos finales (NVIDIA H100 80GB).

Modelo	Batch	Pasos (R/P)	Épocas	Tiempo	Checkpoint	Estado
SmoIVLA-Vanilla	64	22 500/22 500	11,01	4 h 47 min	Paso 22 500	Completo
SmoIVLA-D	64	22 500/22 500	11,01	5 h 21 min	Paso 22 500	Completo
SmoIVLA-M	32	24 000/45 000	5,87	8 h 34 min	Paso 24 000	Interrumpido
SmoIVLA-MD	32	24 000/45 000	5,87	7 h 25 min	Paso 24 000	Interrumpido

Debido al coste computacional de procesar tres fotogramas por muestra en las variantes con memoria (M y MD), su tamaño de lote se redujo a 32 e incrementó el número de pasos previstos. Como muestra la Figura 4.10, SmoIVLA-M y SmoIVLA-MD se interrumpieron en el paso 24 000 por restricciones de presupuesto. Aunque la pérdida continuaba descendiendo suavemente, la ganancia marginal no justificaba el coste de completar el ciclo programado, seleccionándose el último checkpoint disponible para las evaluaciones.

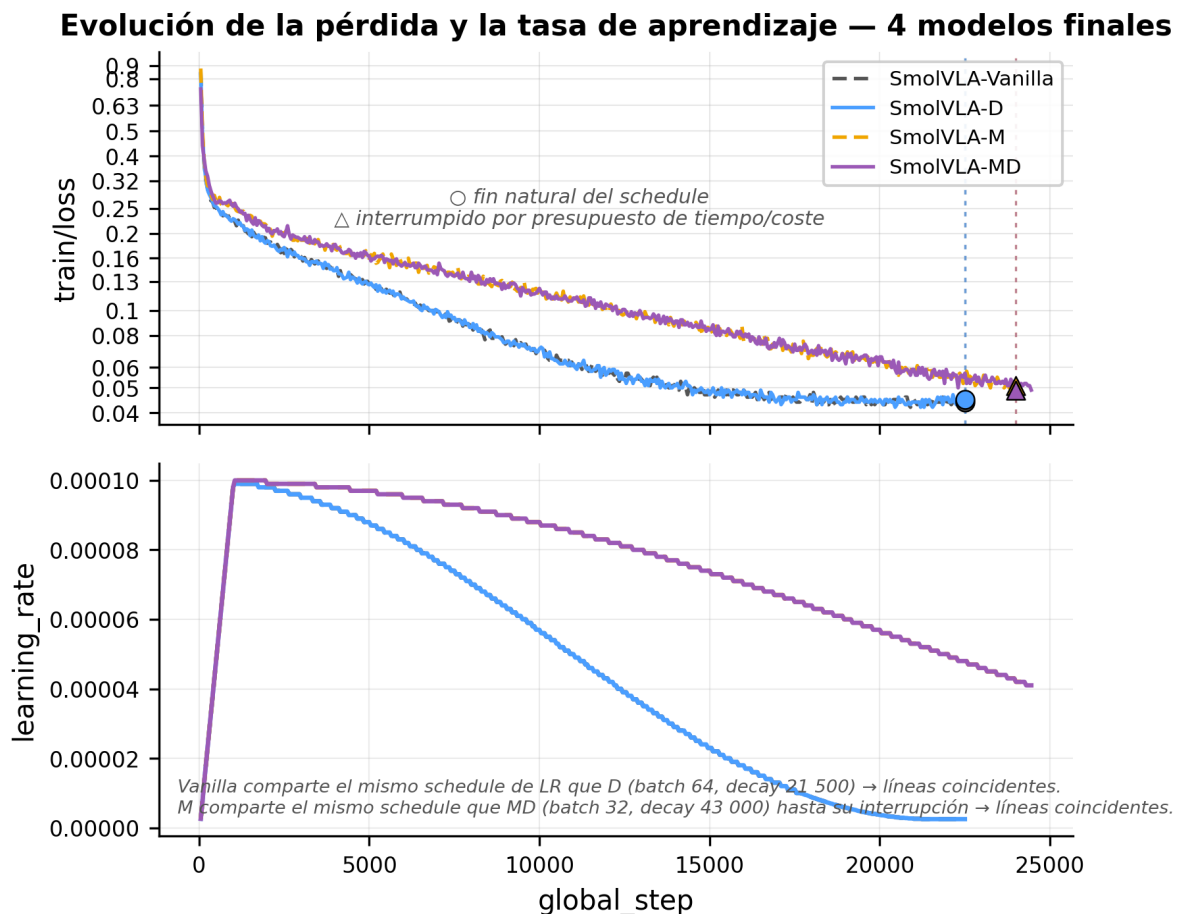


Figura 4.10: Evolución de la pérdida y de la tasa de aprendizaje durante el entrenamiento de las cuatro configuraciones finales. Las líneas verticales indican el punto de finalización o interrupción de cada ejecución.

4.4. Estrategia experimental

4.4.1. Test de ablación de capas

Como ya se explicó en la Subsección 4.3.3, el mecanismo de inyección de información de profundidad se inspira en la estrategia de PointVLA [14]. De igual forma, se plantea un estudio de ablación sobre el experto de acción antes de desarrollar las variantes SmolVLA-D y SmolVLA-MD. El objetivo de este experimento es identificar en qué bloques del experto de acción la inyección de nueva información resulta menos disruptiva, asegurando que la señal tridimensional se integre sin degradar las capacidades previamente aprendidas por el modelo.

El análisis se realiza exclusivamente sobre las 16 capas del experto de acción, manteniendo congelados e intactos el resto de componentes de SmolVLA. Para llevarlo a cabo, se implementó una ligera modificación en la clase original del modelo en LeRobot que permite desactivar un número determinado de capas. En este contexto, desactivar una capa consiste en hacer que devuelva exactamente el embedding de entrada mediante una operación identidad, anulando su efecto en la red sin alterar las dimensiones de los tensores.

Cuadro 4.2: Configuración del entrenamiento utilizado como base para la ablación.

Parámetro	Valor
Modelo base	lerobot/smolvla_base
Lote / Muestras	16 / 1 440 000 (7,71 épocas)
Pasos (Total / Warmup / Decay)	90 000 / 1 000 / 86 000
Parámetros entrenables	Experto de acción (VLM congelado)
Hardware / Tiempo	NVIDIA RTX 5060 Ti (16 GB) / 14 h 13 min

Para evaluar el comportamiento del robot de forma equitativa en cada experimento, se diseñó una prueba bajo condiciones idénticas para todas las configuraciones. El test consta de tres disposiciones de objetos con dificultad creciente, evaluando cinco repeticiones por escena para mitigar la aleatoriedad y con un tiempo límite de 120 segundos por ejecución.

El test de ablación se ejecutó en dos fases. En la primera, se omitió individualmente cada una de las 16 capas del experto de acción para medir su degradación en comparación con el modelo completo y así identificar las regiones de mayor tolerancia a modificaciones. En la segunda, tras identificar la región de mayor estabilidad, se evaluó la omisión simultánea de bloques contiguos mediante las configuraciones {6, 7}, {6, 7, 8}, {7, 8, 9} y {9, 10, 11}. Esta segunda fase resulta imprescindible para comprobar el grado de redundancia de la red y determinar la extensión máxima de la ventana sobre la cual inyectar el adaptador de profundidad sin desestabilizar la política de control.

Para evaluar cada ejecución se utilizó una rúbrica común de 0 a 9 puntos, asignando 2 puntos por cada una de las 4 figuras (agarre y colocación) y un punto adicional por finalizar la prueba limpiamente. A partir de este resultado se obtiene una puntuación normalizada entre 0 y 1 para cada intento. Posteriormente, se generan gráficas que recogen la tasa de éxito absoluto (alcanzar los 9 puntos), la media de las 15 ejecuciones y la dispersión por experimento, junto con métricas diagnósticas adicionales

como los modos de fallo, la fase de manipulación o los bloqueos de la política.

Para el estudio de ablación se entrenó un modelo SmolVLA a partir de los pesos base originales utilizando el optimizador AdamW. El Cuadro 4.2 resume la configuración detallada de este entrenamiento y la Figura 4.11 muestra la evolución de la pérdida en relación con la tasa de aprendizaje.

Evolucion de la perdida y la tasa de aprendizaje — modelo de ablacion por capas

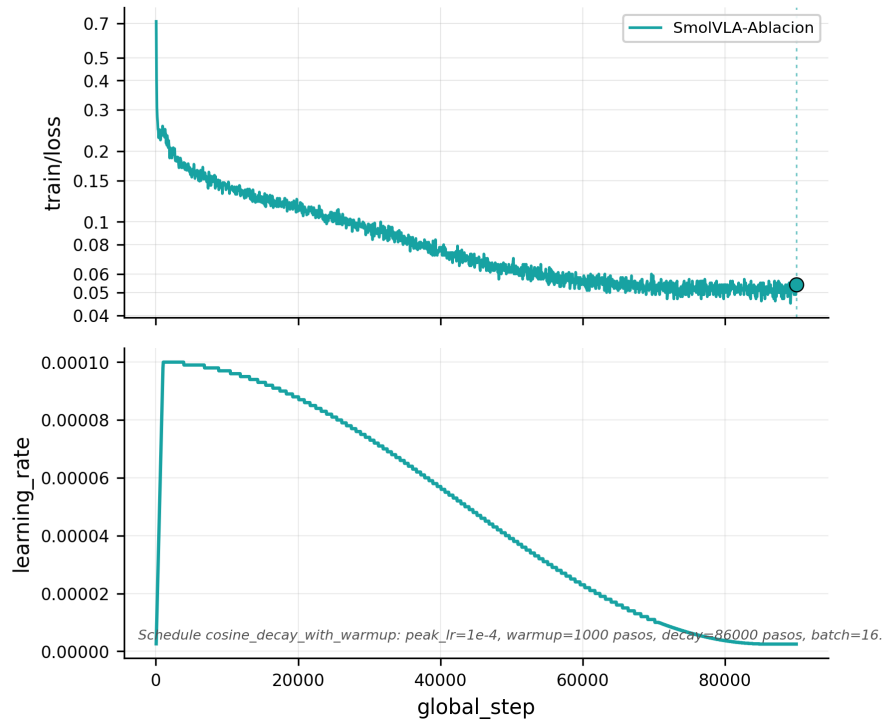


Figura 4.11: Evolución de la pérdida de entrenamiento y de la tasa de aprendizaje durante el ajuste del modelo base para el estudio de ablación.

4.4.2. Evaluación final

El objetivo de la evaluación final es analizar el impacto de las modificaciones de memoria y profundidad descritas en la Sección 4.3 en comparación con la arquitectura de referencia SmolVLA-Vanilla, explicada en la Subsección 4.3.1.

Para evaluar la memoria temporal y la discriminación tridimensional, el experimento introduce dos condiciones del escenario: pruebas estándar de manipulación y pruebas que incorporan una fotografía impresa de uno de los objetos como distractor visual 2D. En un diseño de 12 ejecuciones por modelo sobre una disposición fija del área de trabajo (véase la Figura 4.3 como referencia), los objetos reales se distribuyen estratégicamente para forzar que el propio brazo del robot los ocuya durante los agarres. Las primeras 6 ejecuciones evalúan esta autooclusión para medir si la memoria temporal permite recordar la posición de un objeto tras quedar oculto. En las 6 ejecuciones restantes, se mantiene la misma configuración espacial y se añade la fotografía impresa como elemento OOD, analizando si la percepción 3D permite discernir la imagen plana sin volumen de los objetos tridimensionales reales.

La evaluación de cada ejecución se estructura a través de una rúbrica que mide la destreza básica del robot, así como sus capacidades de memoria y percepción 3D. La prueba otorga inicialmente 2 puntos por cada una de las 4 figuras (agarre y colocación) y un punto extra por finalizar la tarea de forma limpia. Sobre esta base, se otorgan 2 puntos de memoria en todas las ejecuciones (retomar el objeto tras la oclusión y completarlo) y, en las 6 pruebas con fotografía, 2 puntos adicionales de percepción 3D (dirigir la primera intención al objeto real y evitar intentar agarrar la imagen impresa). Para permitir la comparación directa entre escenarios, el resultado total se normaliza entre 0 y 1 dividiendo la puntuación obtenida entre el máximo posible del episodio (11 puntos sin fotografía y 13 con ella), tal y como se sintetiza en El Cuadro 4.3. Finalmente, se registran métricas diagnósticas auxiliares como los modos de fallo, la fase de manipulación o los bloqueos de la política de control.

Cuadro 4.3: Desglose de la rúbrica de evaluación según la condición del escenario.

Categoría de evaluación	Sin fotografía	Con fotografía
Destreza básica (4 objetos + finalización limpia)	9 pts	9 pts
Memoria temporal (tras autoclusión)	2 pts	2 pts
Percepción 3D (resistencia a distractor 2D)	—	2 pts
Puntuación máxima	11 pts	13 pts

Una ejecución válida se considera como éxito completo únicamente cuando alcanza la puntuación máxima del escenario (11 o 13 puntos según la ejecución), considerándose éxito parcial en cualquier otro caso en que se complete solo una parte de la tarea (véase la manipulación correcta de al menos una figura). A partir de estas ejecuciones se generan gráficas y métricas descriptivas, como la puntuación media, la dispersión o la tasa de éxito, junto con el análisis de los modos de fallo, sirviendo de base para la discusión de los resultados en el Capítulo 5.

Capítulo 5

Resultados y discusión

Este capítulo presenta, en primer lugar, los resultados obtenidos en el test de ablación de capas del experto de acción (Subsección 4.4.1) junto con la decisión de diseño que se deriva de estos. A continuación, se detallan los resultados y la discusión de la evaluación final sobre el robot real, cuyo protocolo de experimentación se describió en la Subsección 4.4.2.

5.1. Resultados del test de ablación de capas

Esta sección analiza la sensibilidad de las capas del experto de acción para identificar el lugar óptimo donde inyectar la información de profundidad. El estudio se dividió en dos fases: el análisis de capas individuales y la ablación de grupos de capas contiguas.

En la primera fase de sensibilidad de capas individuales, la Figura 5.1 muestra el impacto de desactivar cada capa por separado frente al rendimiento del modelo original. En los resultados se observan dos comportamientos claramente diferenciados. Por un lado, las capas destacadas en rojo resultan ser críticas, ya que desactivarlas degrada drásticamente la puntuación media por debajo de 0,25 y reduce a cero la tasa de éxito. Por otro lado, existe una zona de tolerancia entre las capas 6 y 11 donde la arquitectura mantiene una puntuación media superior a 0,40. Esta franja continua se identifica como la región candidata para realizar la inyección de la señal 3D.

En la segunda fase de ablación de conjuntos contiguos, la Figura 5.2 compara el rendimiento observado frente al esperado teóricamente al eliminar múltiples capas simultáneas. A medida que la ventana de desactivación se desplaza hacia el final de la red, la penalización sobre la política se incrementa. Tras este ensayo, los conjuntos {6, 7} y {6, 7, 8} resultan ser los más estables de la prueba, conservando los mejores niveles de rendimiento medio.

Para tomar la decisión definitiva entre los candidatos {6, 7} y {6, 7, 8}, se examinó la distribución de errores presentada en la Figura 5.3. Aunque el conjunto {6, 7} mantiene una tasa de éxito completo residual superior a cero, genera una alta proporción de bloqueos severos en la política. En cambio, la región {6, 7, 8} presenta una puntuación media prácticamente idéntica y reduce de forma significativa los bloqueos,

5.1. Resultados del test de ablación de capas

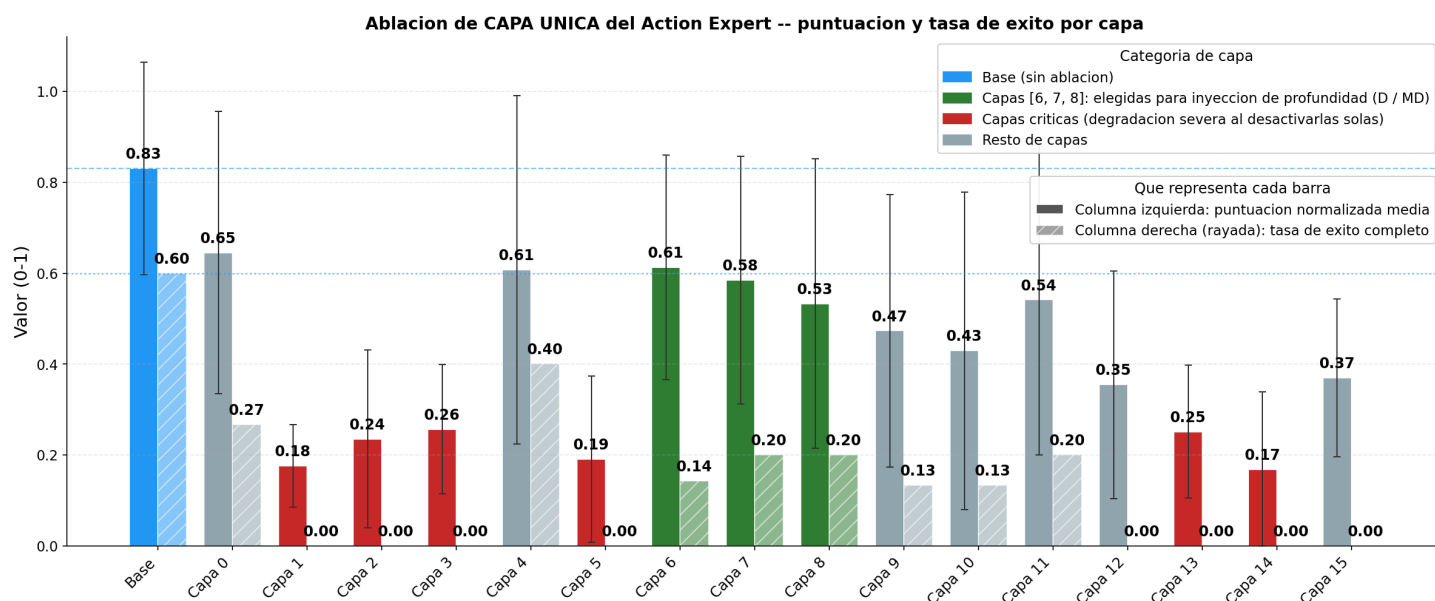


Figura 5.1: Puntuación normalizada media y tasa de éxito completo obtenidas al desactivar individualmente cada capa del experto de acción. Las barras de error representan la desviación típica de la puntuación.

La degradacion combinada es SUPER-ADITIVA (peor de lo esperado) y empeora fuera de 6-8

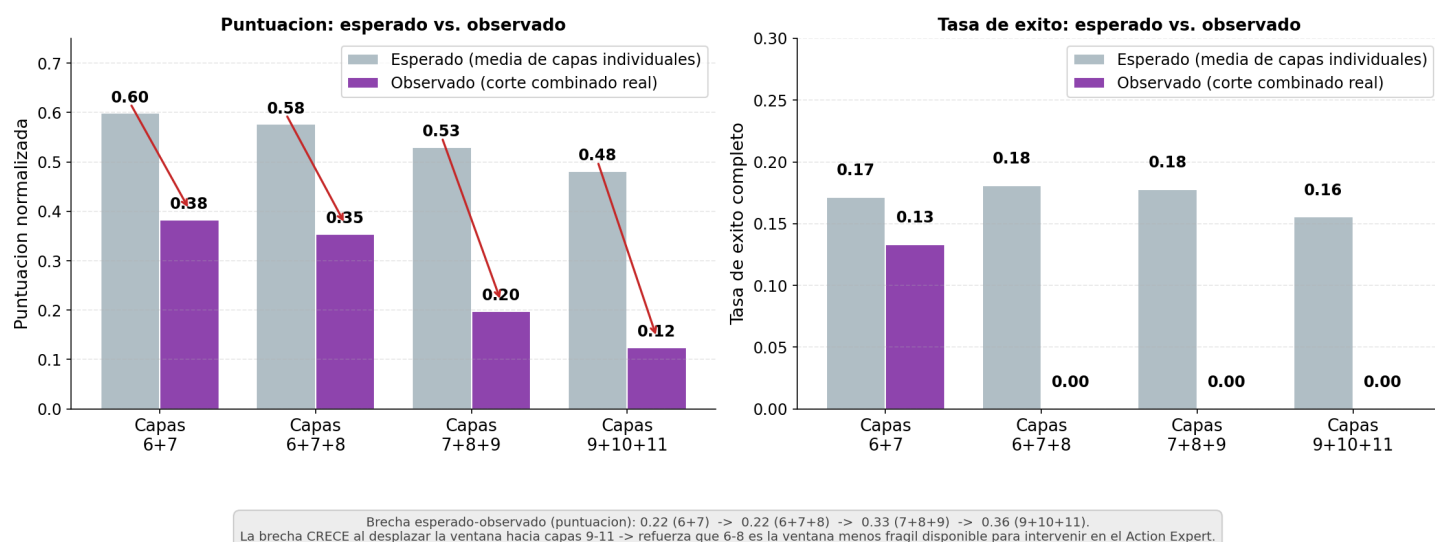


Figura 5.2: Comparación entre el rendimiento esperado a partir de las ablaciones individuales y el observado al desactivar varias capas contiguas.

Resultados y discusión

mostrando fallos más suaves como aproximaciones o agarres imprecisos. Por ello, se selecciona la región {6, 7, 8} para realizar la inyección de profundidad, al ofrecer un espacio más amplio para el adaptador y un comportamiento dinámico menos propenso al colapso.

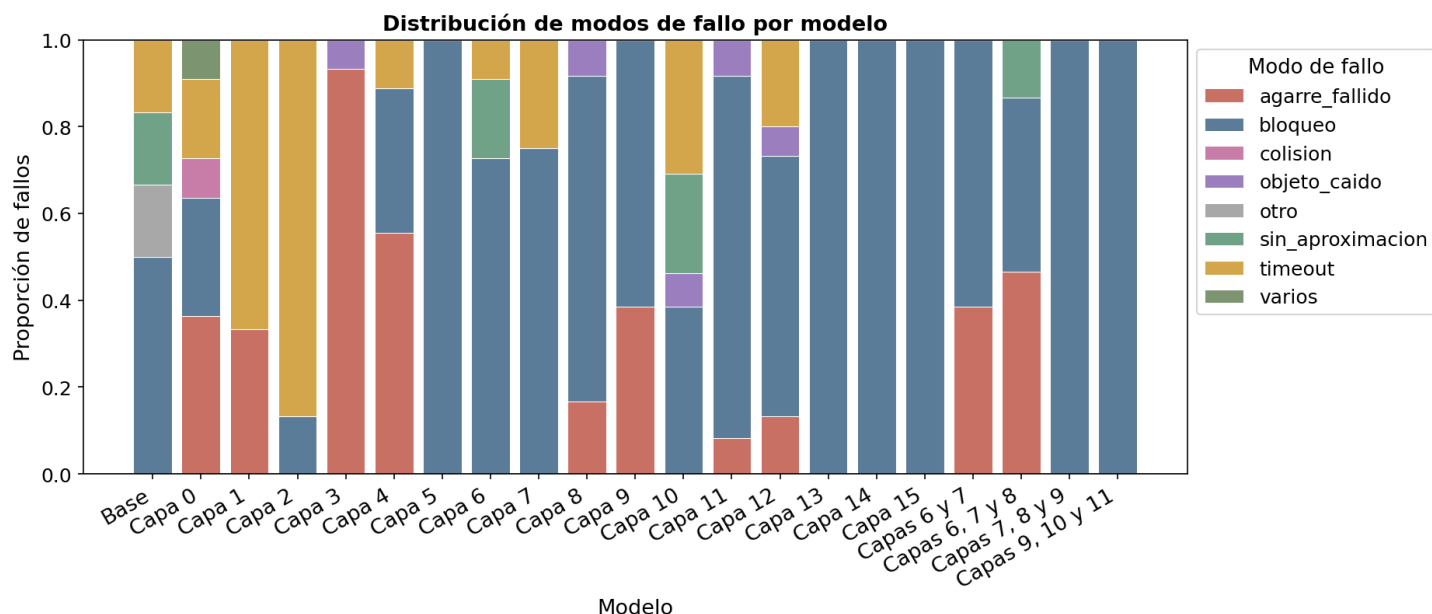


Figura 5.3: Distribución de los modos de fallo observados en el modelo base, las ablaciones individuales y las combinaciones de capas contiguas.

5.2. Resultados de la evaluación final

Para analizar el rendimiento práctico de las arquitecturas propuestas sobre la plataforma física, la Figura 5.4 sintetiza la puntuación normalizada media de cada variante en ambas condiciones experimentales (con y sin fotografía como distractor). Por su parte, la Figura 5.5 desglosa el comportamiento individual en cada uno de los 12 episodios por modelo, permitiendo observar la estabilidad y la dispersión de las ejecuciones.

Al analizar ambas gráficas se observan dos conclusiones clave sobre el rendimiento general.

Por un lado, la fotografía distrae a todas las variantes por igual. Al incluir la foto en la escena, la puntuación media de las distintas propuestas cae drásticamente (Figura 5.4) y los resultados se vuelven muy inestables de una prueba a otra (Figura 5.5). Salvo en algún caso aislado, la presencia de la foto impide que el robot complete la tarea con normalidad.

Por otro lado, SmolVLA-MD no funciona y falla por completo. En los ensayos normales sin foto (episodios 1 al 6 en la Figura 5.5), SmolVLA-D y SmolVLA-M rinden igual o mejor que la arquitectura base SmolVLA-Vanilla, alcanzando SmolVLA-D las puntuaciones más altas de la prueba. Sin embargo, al combinar ambas mejoras en SmolVLA-MD, la propuesta obtiene una puntuación prácticamente nula en casi todos

5.2. Resultados de la evaluación final

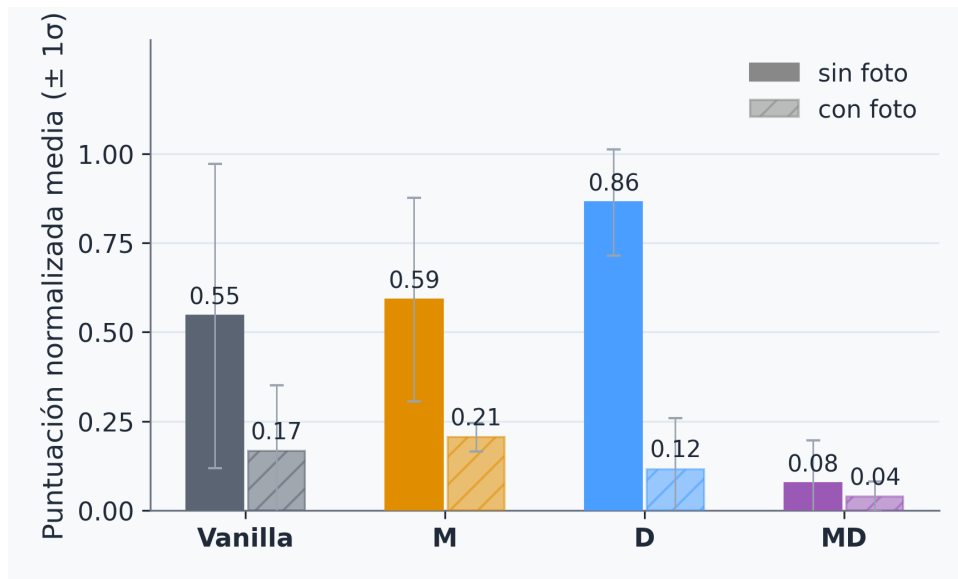


Figura 5.4: Comparación de la puntuación normalizada media por modelo y condición experimental (sin foto y con foto).



Figura 5.5: Puntuación normalizada de cada ejecución física por modelo. La línea vertical separa los episodios sin fotografía de los episodios con fotografía. La línea horizontal representa la media de ese conjunto de ejecuciones

Resultados y discusión

los intentos.

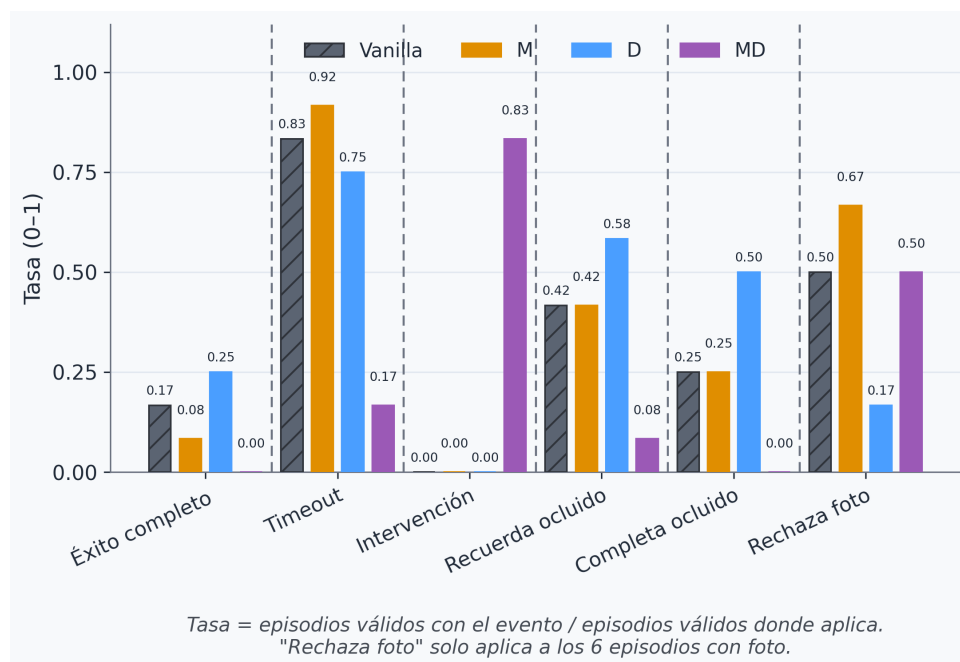


Figura 5.6: Métricas conductuales de la evaluación final: éxito completo, timeout, intervención de emergencia, recuerdo del objeto ocluido, finalización sobre el objeto ocluido y rechazo de la fotografía. Derivadas de las dinámicas de ejecución y modos de fallo, estas tasas permiten identificar cómo se comporta y por qué falla cada modelo en detalle.

Esta disparidad se comprende al examinar los indicadores conductuales de la Figura 5.6. Mientras que SmolVLA-Vanilla, SmolVLA-M y SmolVLA-D completan sus ejecuciones sin requerir paradas de seguridad, SmolVLA-MD acumula un elevado porcentaje de intervenciones de emergencia. Durante sus pruebas, la política sufrió un colapso completo de la política caracterizado por temblores, rotaciones sin control y movimientos marcadamente erráticos y violentos que obligaron a detener manualmente el brazo para evitar daños mecánicos (llegando a registrarse la rotura del soporte de la cámara de la muñeca en uno de los episodios).

En las pruebas sin fotografía, SmolVLA-M rinde de forma muy similar a SmolVLA-Vanilla en las métricas de memoria. Por su parte, SmolVLA-D es la que mejor funciona en estas escenas limpias. Sin embargo, al añadir la foto, SmolVLA-D es la que más empeora de las tres variantes funcionales: en la Figura 5.6 se ve que casi nunca logra ignorar la foto, quedando en puntuación media solo por encima de SmolVLA-MD.

5.3. Discusión de la evaluación final

Los resultados obtenidos en la sección anterior permiten analizar cómo han afectado las variantes desarrolladas en este trabajo al modelo original SmolVLA y ofrecer hipótesis sobre por qué han ocurrido los comportamientos observados.

Empezando por SmolVLA-D, su desempeño superior en las escenas sin distractor confirma la hipótesis de que la inyección de información de profundidad en el experto

de acción proporciona una señal relevante para la manipulación. En ausencia de un distractor OOD, la representación geométrica mitiga las ambigüedades de una imagen únicamente RGB sin información 3D, mejorando la estimación de distancias relativas y la precisión en el agarre, en consonancia con los hallazgos descritos en PointVLA [14]. Asimismo, esta referencia tridimensional explica su mejor desempeño en la resolución de oclusiones parciales.

Sin embargo, la drástica caída de SmolVLA-D ante la fotografía muestra que la información 3D no garantiza por sí sola la discriminación entre objetos reales y fotos. Aunque la cámara estéreo detecta una superficie plana, la coincidencia de textura confunde al codificador RGB, haciendo que la red priorice la imagen sobre la geometría real. Como resultado, la red no logra rechazar la imagen, lo que indica que es necesario reforzar la prioridad de la señal geométrica sobre la apariencia 2D en la arquitectura.

Por su parte, la variante SmolVLA-M demostró un comportamiento estable, pero sin una mejora sustancial en situaciones de autooclusión o pérdida de visibilidad frente a la variante de referencia. La hipótesis de que retener observaciones pasadas permitiría mantener la precisión al perder de vista el objetivo se vio limitada por la propia implementación. A diferencia de MEM [12], en SmolVLA-M se integró únicamente una memoria a corto plazo con menor resolución temporal. Sin pérdidas auxiliares ni una atención más compleja, la representación latente del historial aportó estabilidad motora, pero no la guía espacial necesaria para recuperar la trayectoria en momentos de oclusión.

El colapso funcional de SmolVLA-MD constituye el hallazgo más interesante de las evaluaciones. La hipótesis inicial asumía que la combinación de ambas mejoras arquitectónicas daría lugar a un rendimiento superior al resto de variantes. Sin embargo, la concatenación e inyección simultánea de ambas modalidades en el experto de acción derivó en un comportamiento errático y en bloqueos severos de la política de control.

Este fenómeno responde a una interferencia dentro del espacio latente de la red. Al introducir el vector de memoria y el adaptador de profundidad al mismo tiempo, se incrementó la complejidad sobre un número de capas reducido, dificultando la convergencia del modelo. Aunque esta variante contó con un menor tiempo de entrenamiento, la comparación con SmolVLA-M (que operó bajo restricciones similares sin colapsar) demuestra que la causa principal radica en una incompatibilidad al integrar ambas señales en la arquitectura.

Finalmente, la incapacidad de los modelos para corregir trayectorias erráticas al interactuar con el distractor (así como los bloqueos al insistir en agarrar la fotografía) radica directamente en la naturaleza del conjunto de entrenamiento. La mayoría de las demostraciones consistieron en ejecuciones limpias de la tarea. Aunque se incluyeron episodios con correcciones tras agarres fallidos sobre un mismo objetivo, el dataset no contenía ningún ejemplo en el que el robot abandonase un objeto tras un intento infructuoso para dirigirse a uno distinto.

Como consecuencia, cuando la política se aproximaba a la fotografía o sufría una desviación, carecía de vectores de acción que guiaran al modelo a cambiar de objetivo. Esto provocaba que el robot agotara el tiempo del episodio en infinitos reintentos sobre la imagen. Este comportamiento evidencia que la robustez en modelos VLA no

Resultados y discusión

depende únicamente de la riqueza sensorial de la arquitectura, sino también de la representatividad y variedad del conjunto de entrenamiento.

Capítulo 6

Conclusiones y trabajo futuro

6.1. Conclusiones

Este Trabajo Fin de Máster ha abordado el diseño, entrenamiento y evaluación de extensiones de percepción 3D y memoria visual a corto plazo sobre el modelo VLA ligero SmolVLA. A través de un procedimiento que ha abarcado la recolección de datos, un estudio de ablación para seleccionar la arquitectura óptima y la evaluación experimental en un robot real, se pueden establecer las siguientes conclusiones.

Por un lado, en los resultados del test de ablación se seleccionó el conjunto de capas {6, 7, 8} para inyectar la percepción de profundidad, el cual fue elegido por tener la menor degradación de las opciones candidatas de la Fase 2 con más capas quitadas (ver Figura 5.2). Podemos concluir que el objetivo operativo de este estudio fue exitoso debido a los resultados positivos en la parte de manipulación de SmolVLA-D, obteniendo una puntuación de 0,86 en escenarios sin fotografía y un pleno en el recuerdo de oclusiones (ver Figuras 5.5 y 5.6). Con esto podemos concluir como extra en este trabajo que, de forma aislada, es posible inyectar nueva información dentro de un experto de acción en un VLA sin colapsar la política.

Asimismo, las evidencias presentadas y discutidas en el Capítulo 5 ofrecen un respaldo desigual a las hipótesis planteadas al inicio de este trabajo:

Por su parte, la hipótesis de la memoria temporal H1 tiene un respaldo bastante limitado. La ventana temporal de SmolVLA-M funcionó en la práctica como un suavizador de movimientos en el mejor de los casos, aportando estabilidad a la trayectoria y logrando una puntuación de 0,59 con 5 de 6 aciertos en la memoria de oclusiones en el escenario sin fotografía, pero sin llegar a ofrecer una verdadera mejora a la hora de guiar la política cuando el objetivo se pierde de vista durante demasiado tiempo, cayendo a 0,21 de puntuación y finalizando 11 de los 12 episodios totales por agotamiento de tiempo (ver Figuras 5.5 y 5.6).

En cuanto a la hipótesis de percepción espacial H2, solo se sostiene en escenarios limpios. Añadir profundidad en SmolVLA-D sirvió para afinar el agarre y resolver oclusiones sencillas, logrando la puntuación más alta del estudio en el bloque sin fotografía (0,86), pero se quedó corta en cuanto entró en juego un distractor fuera de distribución; la red terminó priorizando la apariencia 2D del RGB frente a la geometría

real de la escena, cayendo a una puntuación de 0,12 en el bloque con fotografía y logrando evitar el agarre de la foto solo en 1 de los 6 episodios (ver Figuras 5.5 y 5.6).

Por último, la hipótesis de sinergia H3 queda refutada. La combinación de ambas modalidades en SmolVLA-MD no solo no sumó lo mejor de cada variante, obteniendo puntuaciones residuales de 0,08 en escenarios sin fotografía y 0,04 en escenarios con fotografía, sino que terminó saturando las capas del experto de acción e introduciendo movimientos inestables que obligaron a realizar 10 paradas de emergencia manuales sobre 12 intentos. Esto deja claro que acoplar ramas perceptivas sin un mecanismo que coordine cómo se mezclan las señales, arruina la estabilidad de la política de control (ver Figura 5.4).

A su vez podemos concluir que la recolección de datos es claramente importante, no siendo tan importante la cantidad como la calidad, pues con los resultados en mente, si nos fijamos en la Figura 4.10 que muestra la gráfica con los entrenamientos podemos ver como las variantes SmolVLA-D y Vanilla sí llegaron a una meseta donde no disminuía la pérdida, y las demás propuestas obtenían una pérdida comparable. Esto indica que el volumen de muestras era suficiente para entrenar los modelos, por lo que los fallos responden más a una falta de diversidad en las demostraciones. En concreto, faltaron secuencias con formas alternativas de completar la tarea, como abandonar un objeto para pasar a otro, o corregir la ejecución si una pieza queda mal colocada durante el proceso.

Estos resultados deben interpretarse dentro de las limitaciones propias del trabajo. En primer lugar, existe una restricción en el alcance experimental al haber evaluado sobre un conjunto concreto de objetos y una tarea específica de manipulación, lo que limita la capacidad de generalizar las conclusiones a otros escenarios de mayor complejidad. En segundo lugar, la señal perceptiva de profundidad estuvo condicionada por el ruido inherente a una cámara estéreo de bajo coste que no se pudo calibrar para conseguir una precisión subpíxel, degradando potencialmente la calidad de la representación geométrica inyectada. En tercer lugar, el módulo de memoria visual se limitó por diseño a un horizonte temporal corto con poca resolución (5 segundos en el pasado con un stride de 2,5 segundos) para mantener la eficiencia, resultando insuficiente para representar eventos prolongados o dependencias de largo alcance.

Por otro lado, existen condicionantes arquitectónicos y computacionales relevantes. Las extensiones desarrolladas se incorporaron de forma acotada y localizada dentro de la red, inyectando la memoria antes del backbone visual y la profundidad dentro del experto de acción. Si bien esta decisión permitió preservar el conocimiento previamente adquirido por la arquitectura base y reducir el coste de adaptación, también restringió la interacción directa y cruzada entre las distintas mejoras perceptivas. Finalmente, el elevado coste computacional asociado a los modelos VLA limitó la exploración de hiperparámetros, el entrenamiento con diferentes semillas iniciales y la repetición de ensayos, haciendo además que los modelos con extensión de memoria no pudieran completar la totalidad de su ciclo de entrenamiento por cuestiones estrictas de tiempo y presupuesto.

En definitiva, este trabajo muestra que enriquecer la percepción de un modelo VLA ligero no es tan simple como conectar nuevas señales a la red. Para que la información tridimensional o la memoria temporal se traduzcan en una mayor robustez de la política, la arquitectura debe incorporar mecanismos explícitos que coordinen cómo se ponderan estas modalidades dentro del modelo, requiriendo además un conjunto

de datos suficientemente variado para que la red aprenda a explotar dichas mejoras.

6.2. Trabajo futuro

Una primera línea de trabajo futuro sería estudiar a fondo cómo se mezclan las señales de las ampliaciones sensoriales en este tipo de modelos. En lugar de la inyección directa empleada en este trabajo, convendría explorar diferentes pérdidas auxiliares y métodos de fusión para que el modelo aprenda a apoyarse en esta información extra solo cuando sea necesario, evitando que los gradientes de los distintos módulos compitan o se cancelen entre sí durante el entrenamiento.

A la hora de abordar las limitaciones identificadas en este estudio, no todas las posibles líneas de actuación contribuyen de la misma manera ni requieren la misma inversión. Las acciones con mejor relación entre coste y beneficio son la mejora en la diversidad de los datos y la integración arquitectónica, ya que permitirían aumentar sustancialmente el desempeño de las variantes sin disparar los recursos. En un escalón intermedio se situaría la revisión del mecanismo de memoria y la calidad de la cámara, las cuales implican un coste notable en VRAM y en equipamiento físico para obtener mejoras predominantemente perceptivas. Por último, la exploración sistemática de hiperparámetros mediante reentrenamientos sucesivos representa la vía más cara en tiempo y cómputo, resultando la opción menos prioritaria si no se corrigen antes los factores principales.

Finalmente, sería muy interesante evaluar estas mismas extensiones sobre arquitecturas VLA más complejas y de mayor capacidad que SmolVLA, el cual no deja de ser un modelo pensado para bajo coste y despliegue en hardware de consumo, así como extender las pruebas experimentales a tareas de mayor complejidad y variabilidad funcional.

Bibliografía

- [1] M. Shridhar, L. Manuelli y D. Fox, «A Multi-Task Transformer for Robotic Manipulation,» en, sep. de 2022. dirección: <https://arxiv.org/abs/2209.05451>.
- [2] T. Z. Zhao, V. Kumar, S. Levine y C. Finn, *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*, en, arXiv:2304.13705 [cs], abr. de 2023. DOI: 10.48550/arXiv.2304.13705. dirección: <http://arxiv.org/abs/2304.13705>.
- [3] A. Brohan et al., *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*, en, arXiv:2307.15818 [cs], jul. de 2023. DOI: 10.48550/arXiv.2307.15818. dirección: <http://arxiv.org/abs/2307.15818>.
- [4] O. M. Team et al., *Octo: An Open-Source Generalist Robot Policy*, en, arXiv:2405.12213 [cs], mayo de 2024. DOI: 10.48550/arXiv.2405.12213. dirección: <http://arxiv.org/abs/2405.12213>.
- [5] K. Black et al., π_0 : A Vision-Language-Action Flow Model for General Robot Control, en, arXiv:2410.24164 [cs], ene. de 2026. DOI: 10.48550/arXiv.2410.24164. dirección: <http://arxiv.org/abs/2410.24164>.
- [6] M. J. Kim et al., *OpenVLA: An Open-Source Vision-Language-Action Model*, en, arXiv:2406.09246 [cs], sep. de 2024. DOI: 10.48550/arXiv.2406.09246. dirección: <http://arxiv.org/abs/2406.09246>.
- [7] NVIDIA et al., *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots*, en, arXiv:2503.14734 [cs], mar. de 2025. DOI: 10.48550/arXiv.2503.14734. dirección: <http://arxiv.org/abs/2503.14734>.
- [8] P. Intelligence et al., $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization, en, arXiv:2504.16054 [cs], abr. de 2025. DOI: 10.48550/arXiv.2504.16054. dirección: <http://arxiv.org/abs/2504.16054>.
- [9] J. Wen et al., *TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation*, en, arXiv:2409.12514 [cs.RO], sep. de 2024. DOI: 10.48550/arXiv.2409.12514. dirección: <https://arxiv.org/abs/2409.12514>.
- [10] M. Shukor et al., *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics*, en, arXiv:2506.01844 [cs], jun. de 2025. DOI: 10.48550/arXiv.2506.01844. dirección: <http://arxiv.org/abs/2506.01844>.
- [11] C. Zhang, P. Hao, X. Cao, X. Hao, S. Cui y S. Wang, *VTLA: Vision-Tactile-Language-Action Model with Preference Learning for Insertion Manipulation*, en, arXiv:2505.09577 [cs], mayo de 2025. DOI: 10.48550/arXiv.2505.09577. dirección: <http://arxiv.org/abs/2505.09577>.
- [12] M. Torne et al., «MEM: Multi-Scale Embodied Memory for Vision Language Action Models,» en, mar. de 2026. dirección: <https://www.pi.website/download/Mem.pdf>.

- [13] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang y H. Xu, *3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations*, en, arXiv:2403.03954 [cs], sep. de 2024. DOI: 10.48550/arXiv.2403.03954. dirección: <http://arxiv.org/abs/2403.03954>.
- [14] C. Li, J. Wen, Y. Peng, Y. Peng e Y. Zhu, «PointVLA: Injecting the 3D World Into Vision-Language-Action Models,» en, *IEEE Robotics and Automation Letters*, vol. 11, n.º 3, págs. 2506-2513, mar. de 2026. DOI: 10.1109/LRA.2026.3653303. dirección: <https://ieeexplore.ieee.org/document/11346992/>.
- [15] P. Holderrieth y E. Erives, *An Introduction to Flow Matching and Diffusion Models*, MIT Class 6.S184: Generative AI With Stochastic Differential Equations, jul. de 2025. arXiv: 2506.02070 [cs.LG]. dirección: <https://arxiv.org/abs/2506.02070>.